# MightyAISG™ AI Security Platform

- **Powered by agentgateway and WireGuard**

**MightyLink**

(www.mightyconn. com)

Doc. Revision: 1.00

# Table of Contents





# 1. AI Security Gateway

**LLM, MCP, A2A Gateway**
**based on WireGuard and ZTNA**

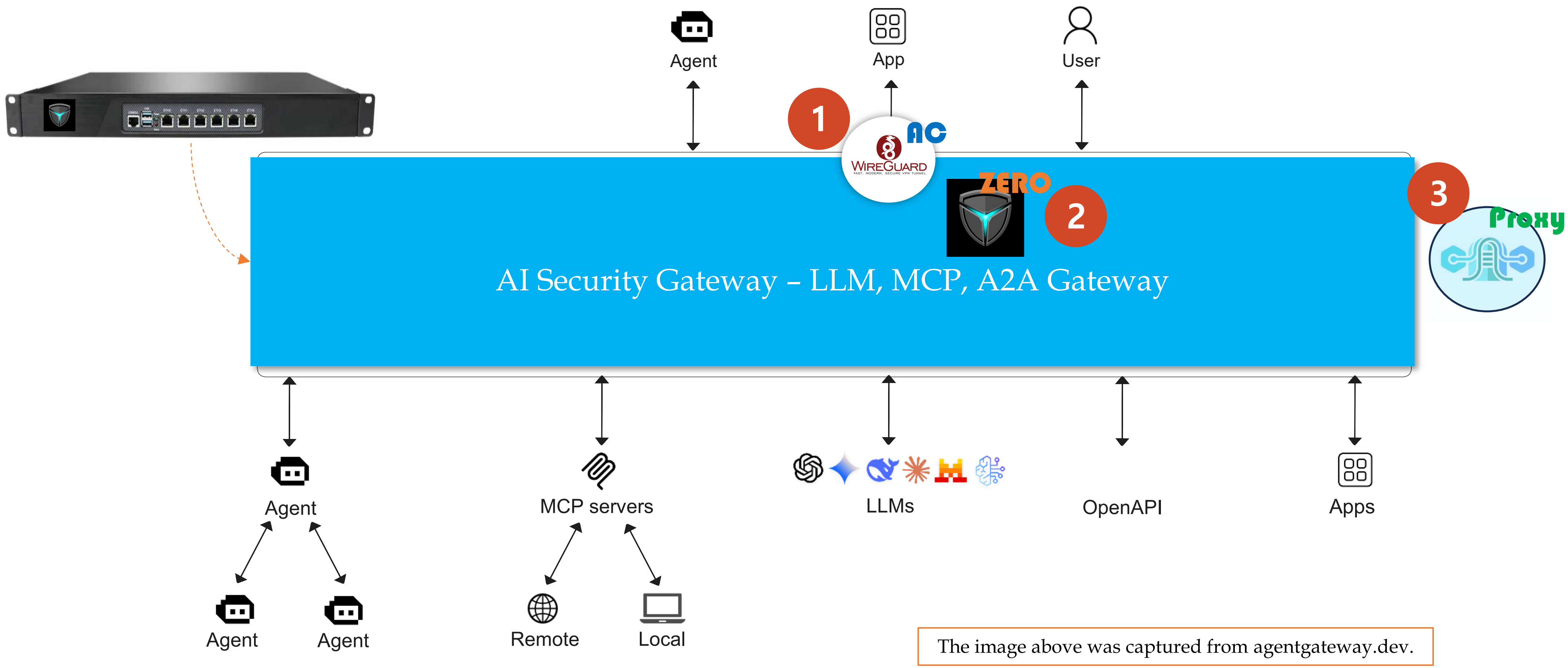
ZERO Trust

# 1. AI Security Gateway(1) – Overview(1)

The image below was captured from agentgateway.dev.


Ref) https://agentgateway.dev/

# 1. AI Security Gateway(1) – Overview(2)




The image above was captured from agentgateway.dev.

# 1. AI Security Gateway(1) – Overview(3)

WireGuard Tunnel

eBPF TC/XDP ZTNA

Application-Level Proxy

1

- 강력하고 빠른 VPN 터널 생성
- CLI/IDE, Agents, Apps 자체 보안 설정 불필요
- PQC(양자내성 암호알고리즘) 적용
- 자동 연결 기능 제공, Mesh DNS
- 다양한 OS 지원 & Mini Gateway 제공

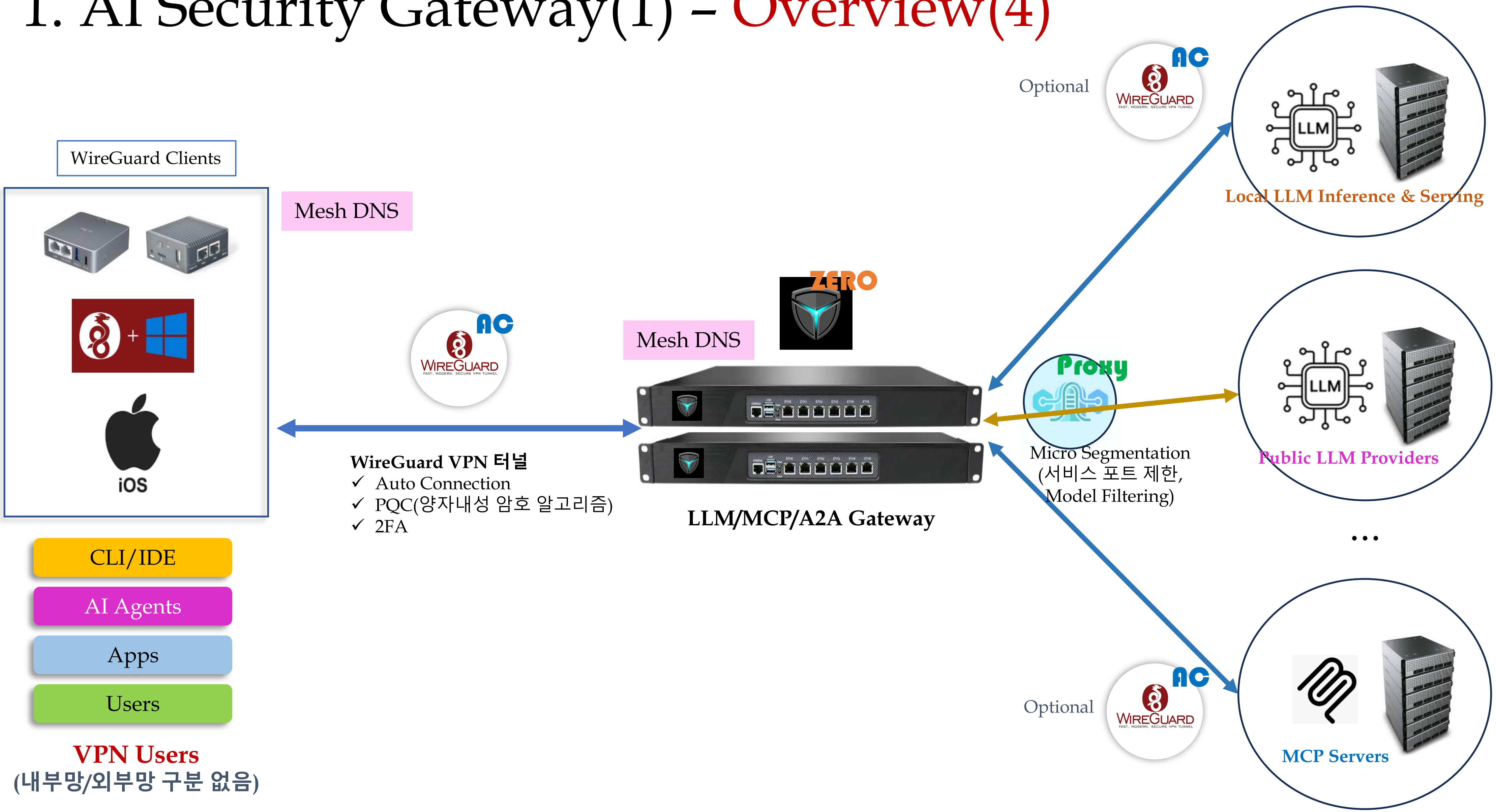
2

- Two-Factor Authentication
- Client User or Group 별 통제
- Resource(Dst IP, Port) 및 LLM 모델별 접근 제어
- eBPF 기반의 Policy Map - Fast Packet Filtering
- Mighty Gateway API Key 별 통제

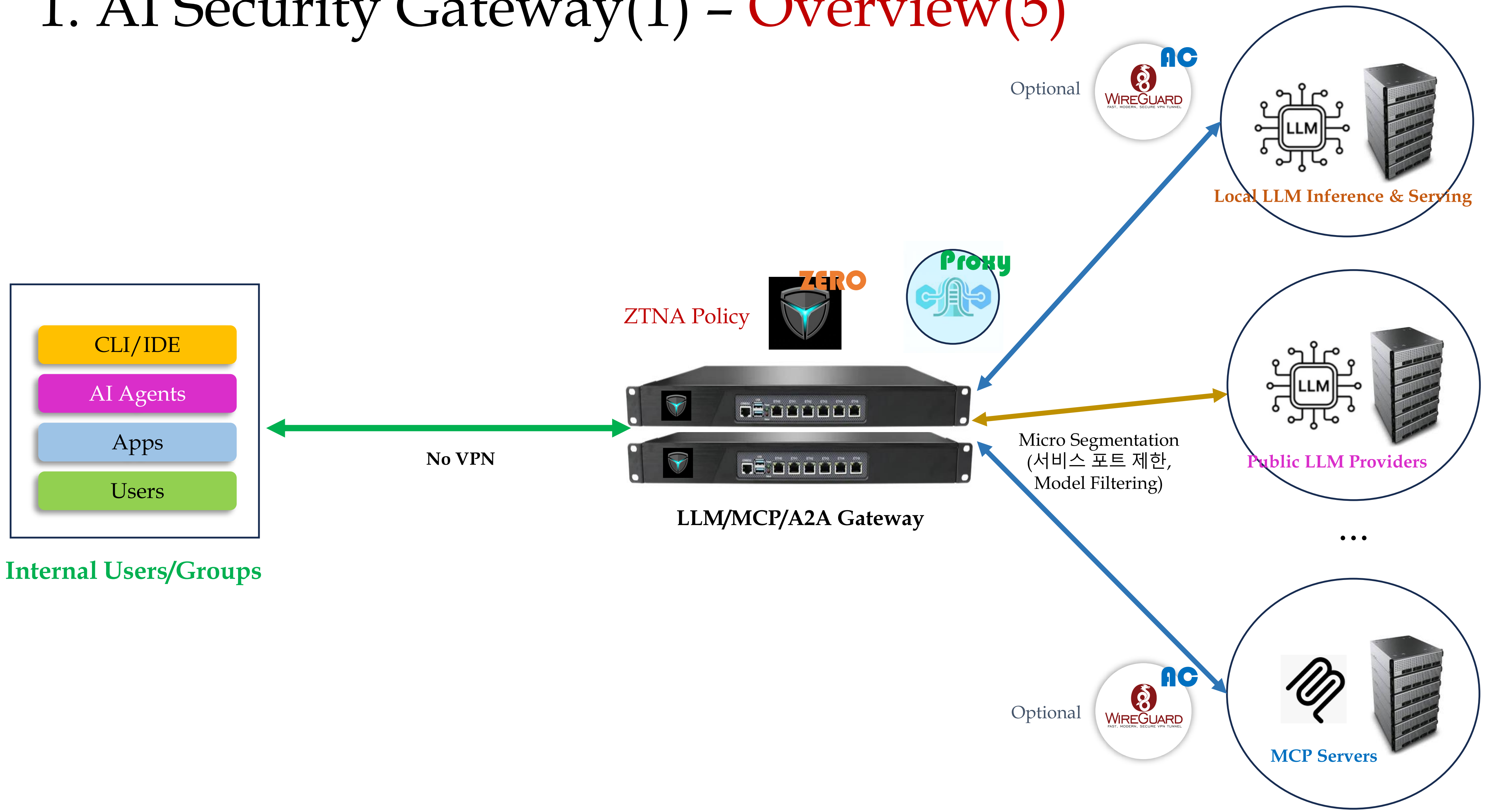
3

- LLM/MCP/A2A Connection
- Public & Local LLM Engine 연결 가능
- Load Balancing for LLM Provider
- DLP(Data Loss Prevention) & Rate Limiting

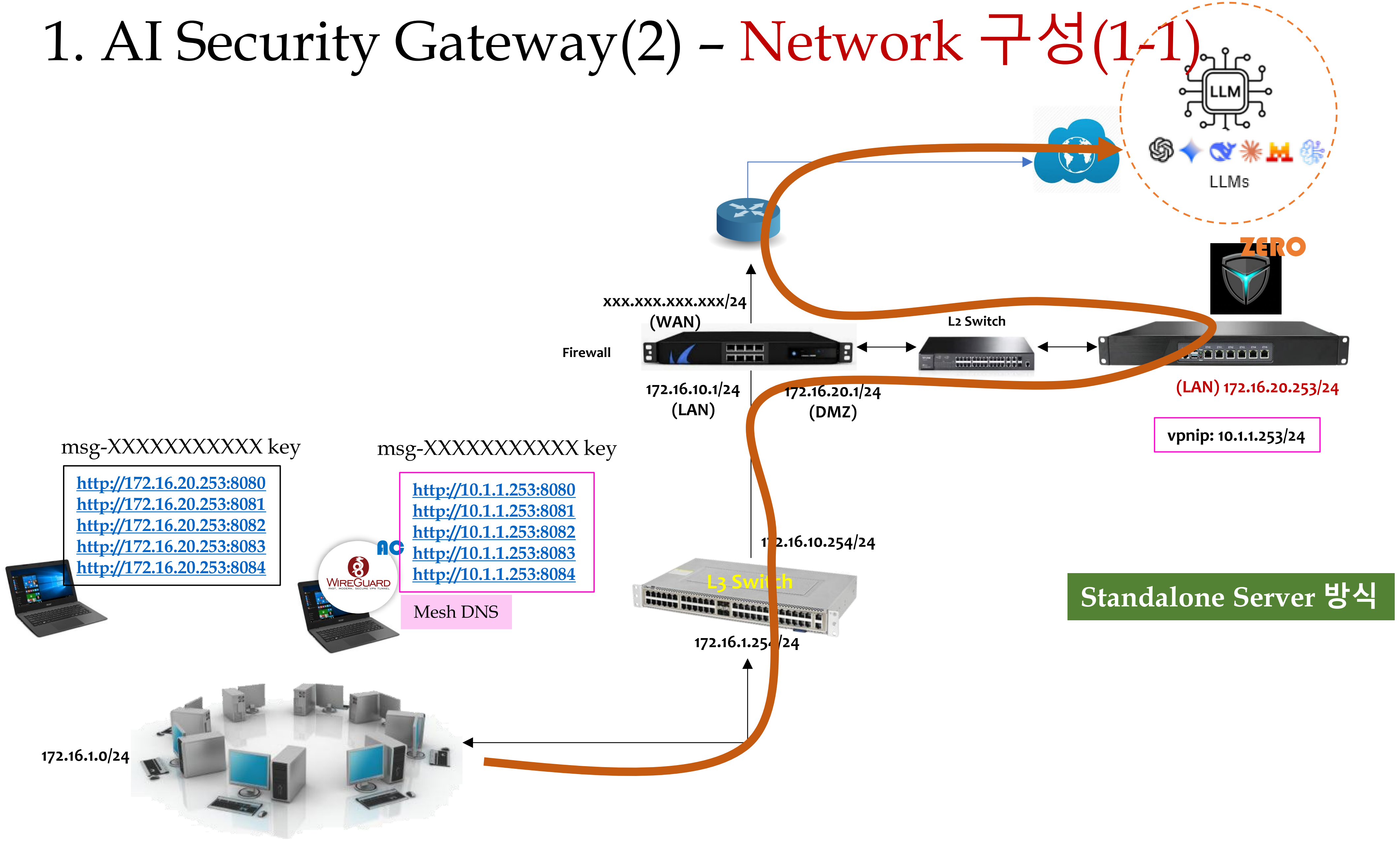
# 1. AI Security Gateway(1) – Overview(4)



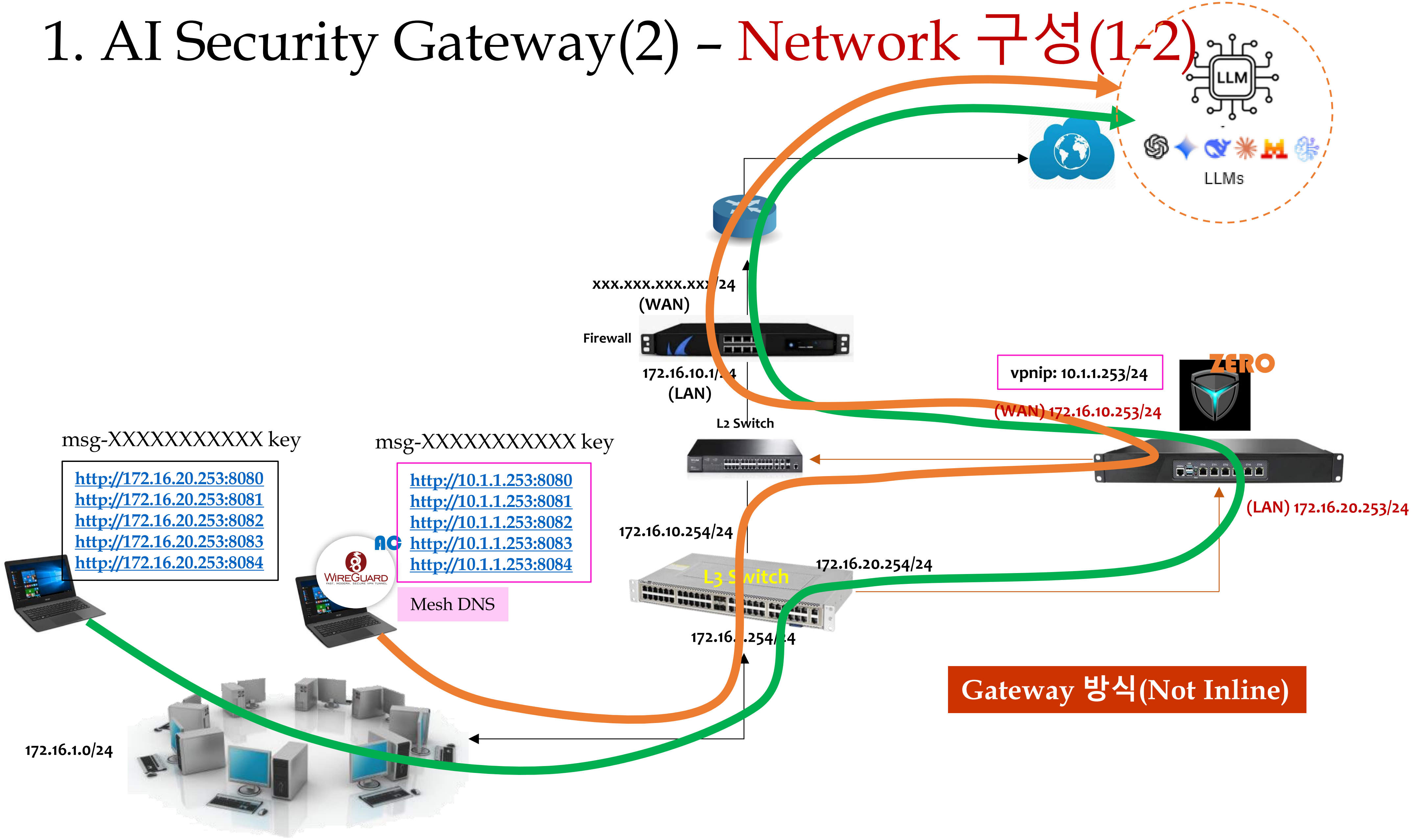
# 1. AI Security Gateway(1) – Overview(5)



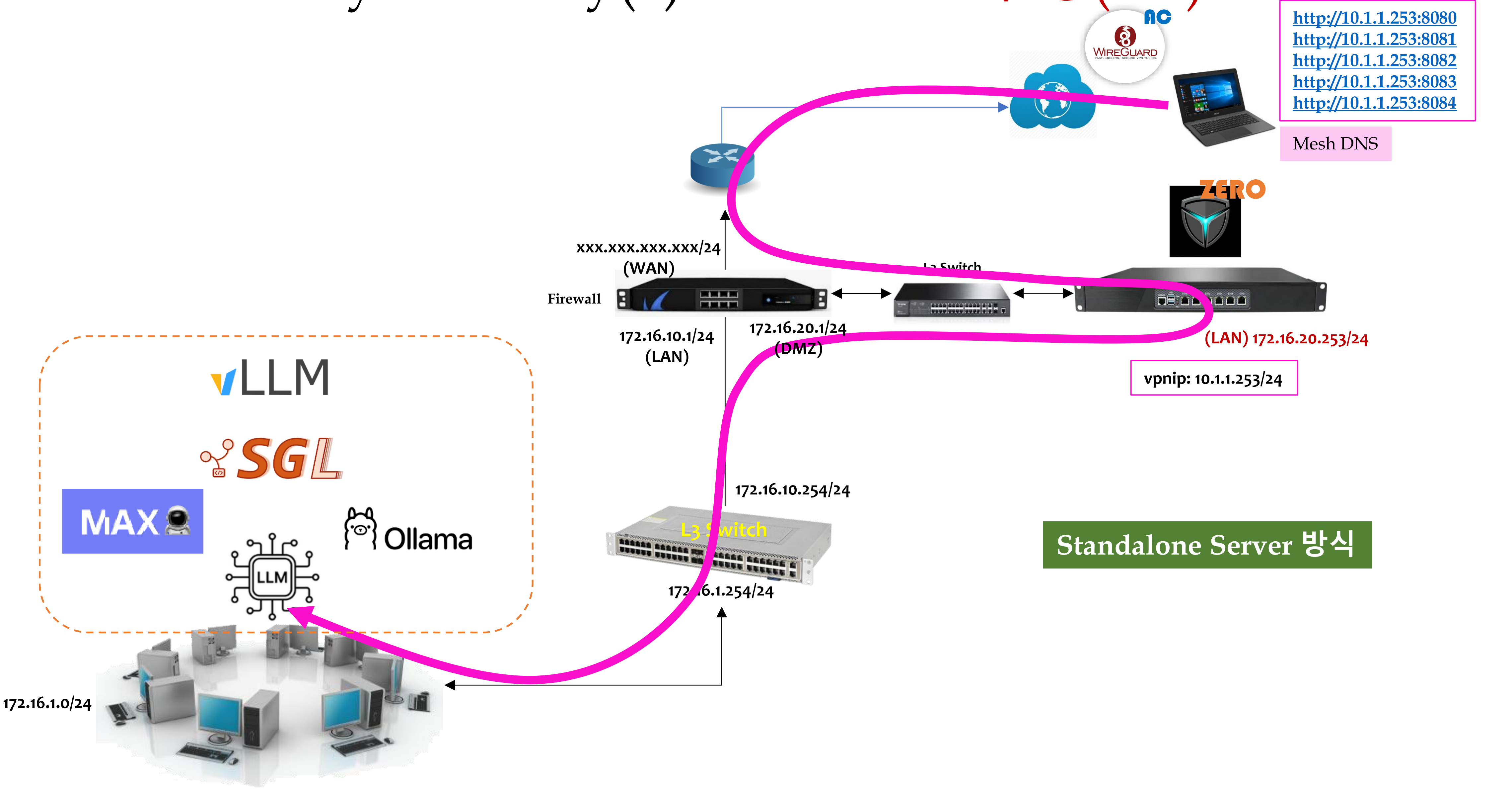
# 1. AI Security Gateway(2) – Network 구성(1-1)



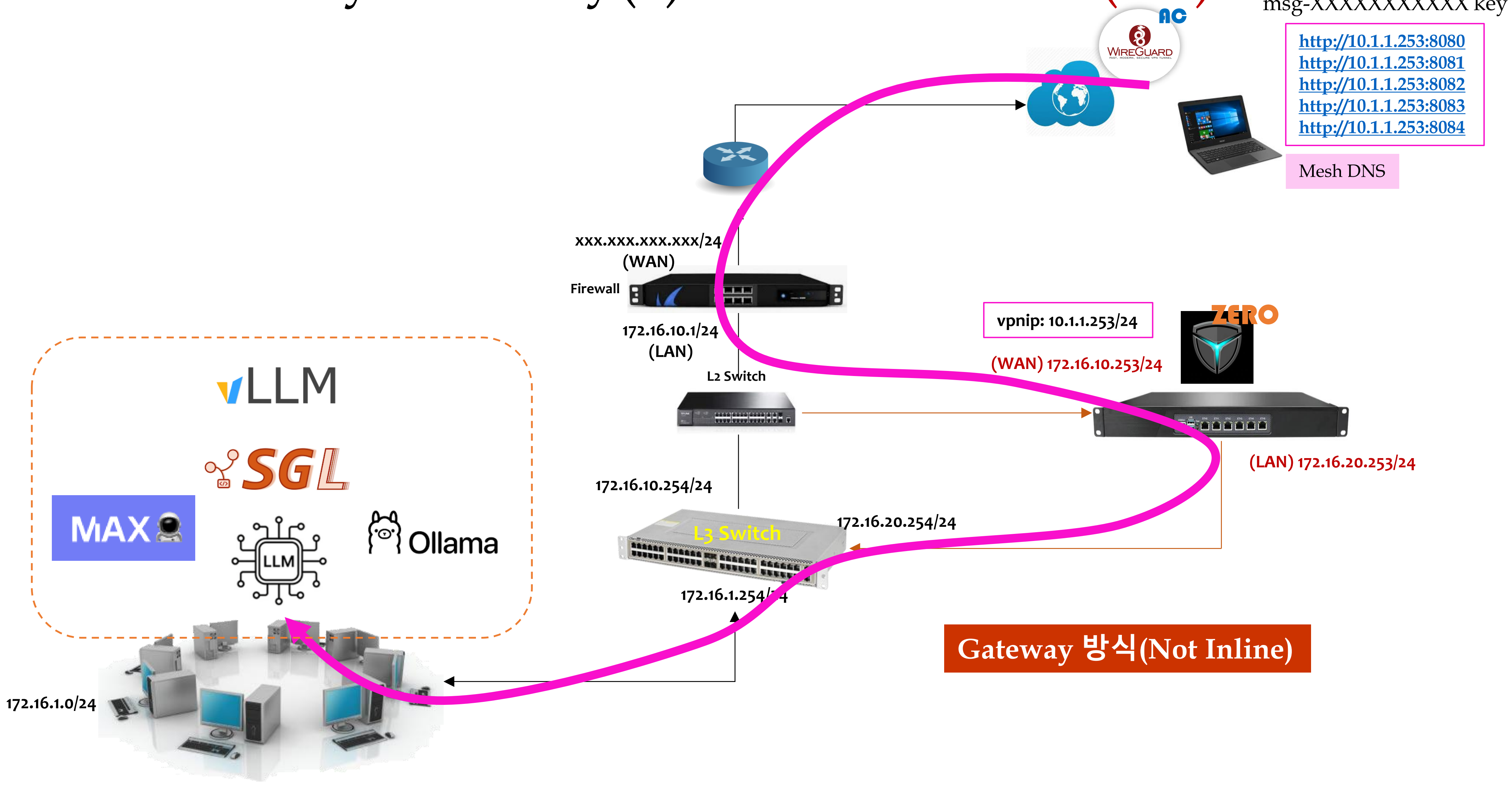
# 1. AI Security Gateway(2) – Network 구성(1-2)



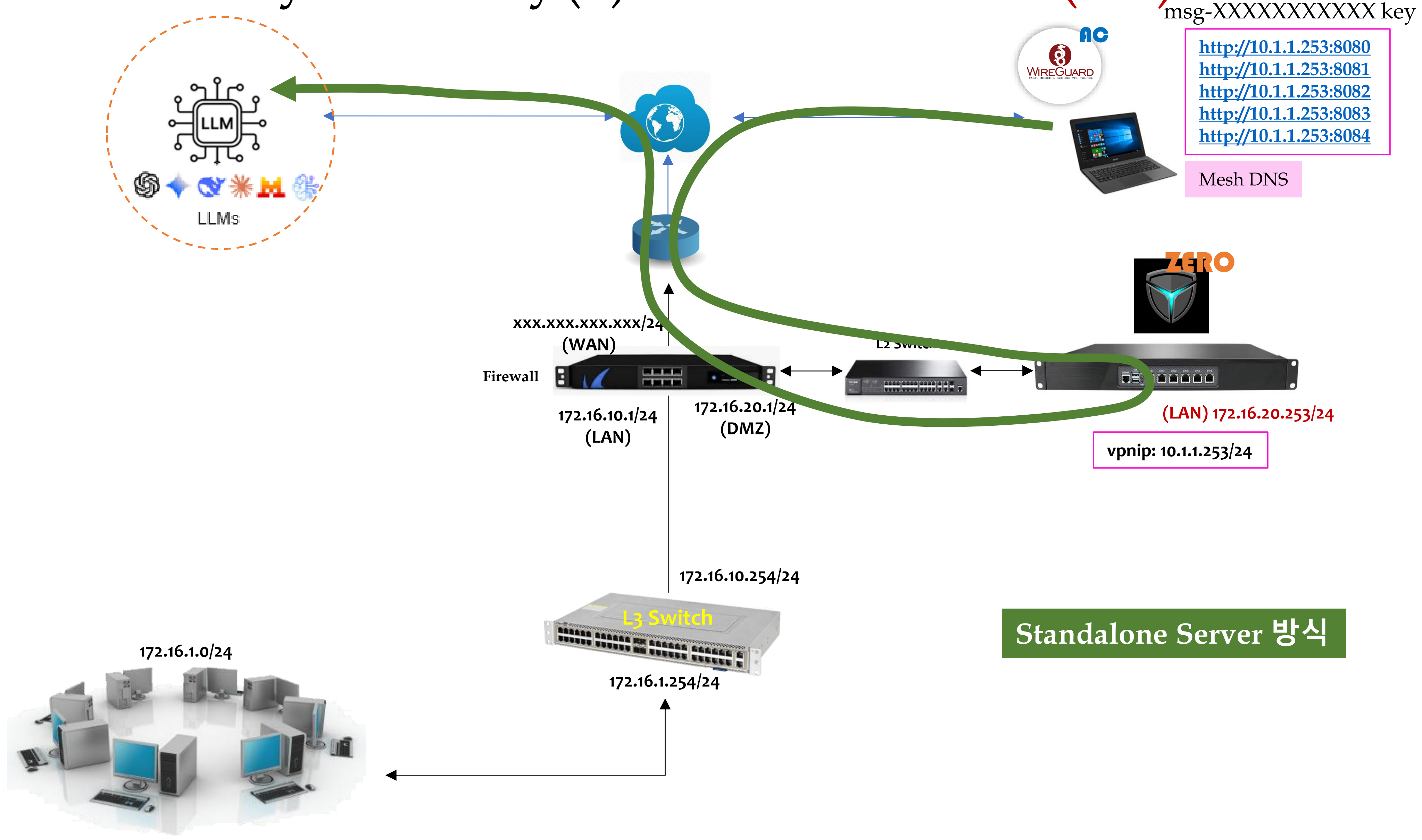
# 1. AI Security Gateway(2) – Network 구성(2-1)



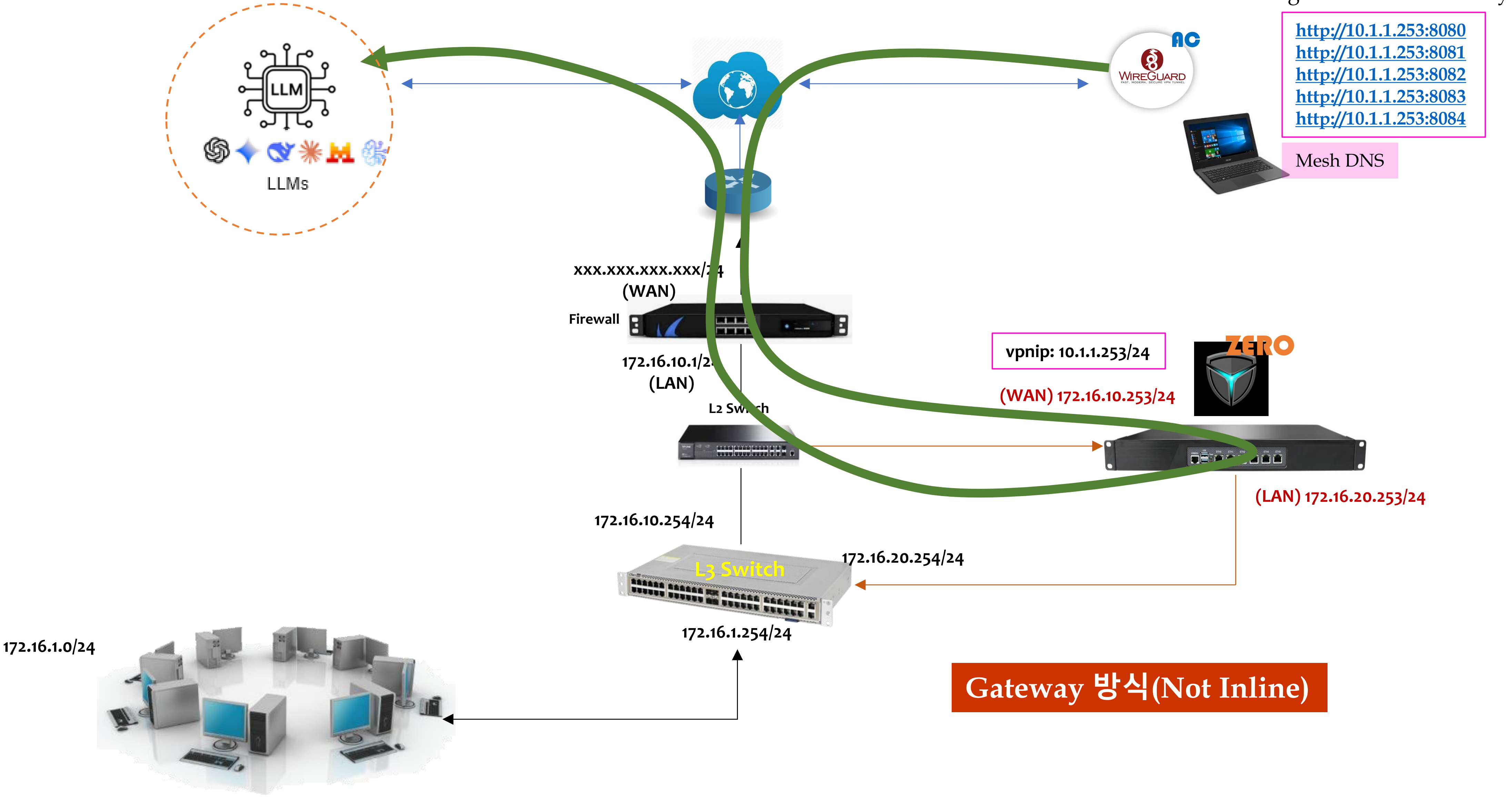
# 1. AI Security Gateway(2) – Network 구성(2-2)



# 1. AI Security Gateway(2) – Network 구성(3-1)

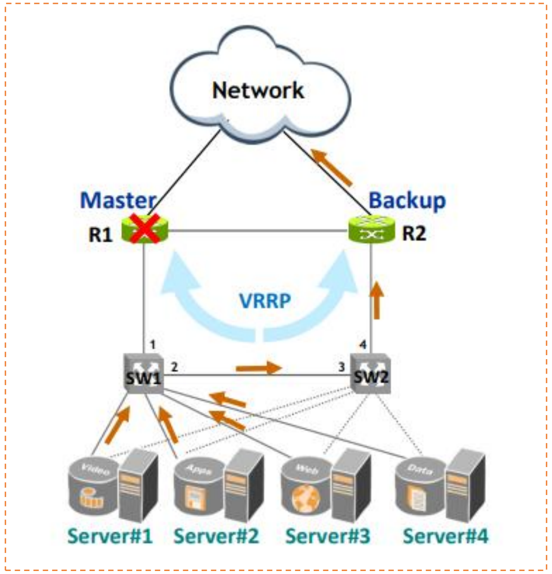


# 1. AI Security Gateway(2) – Network 구성(3-2)

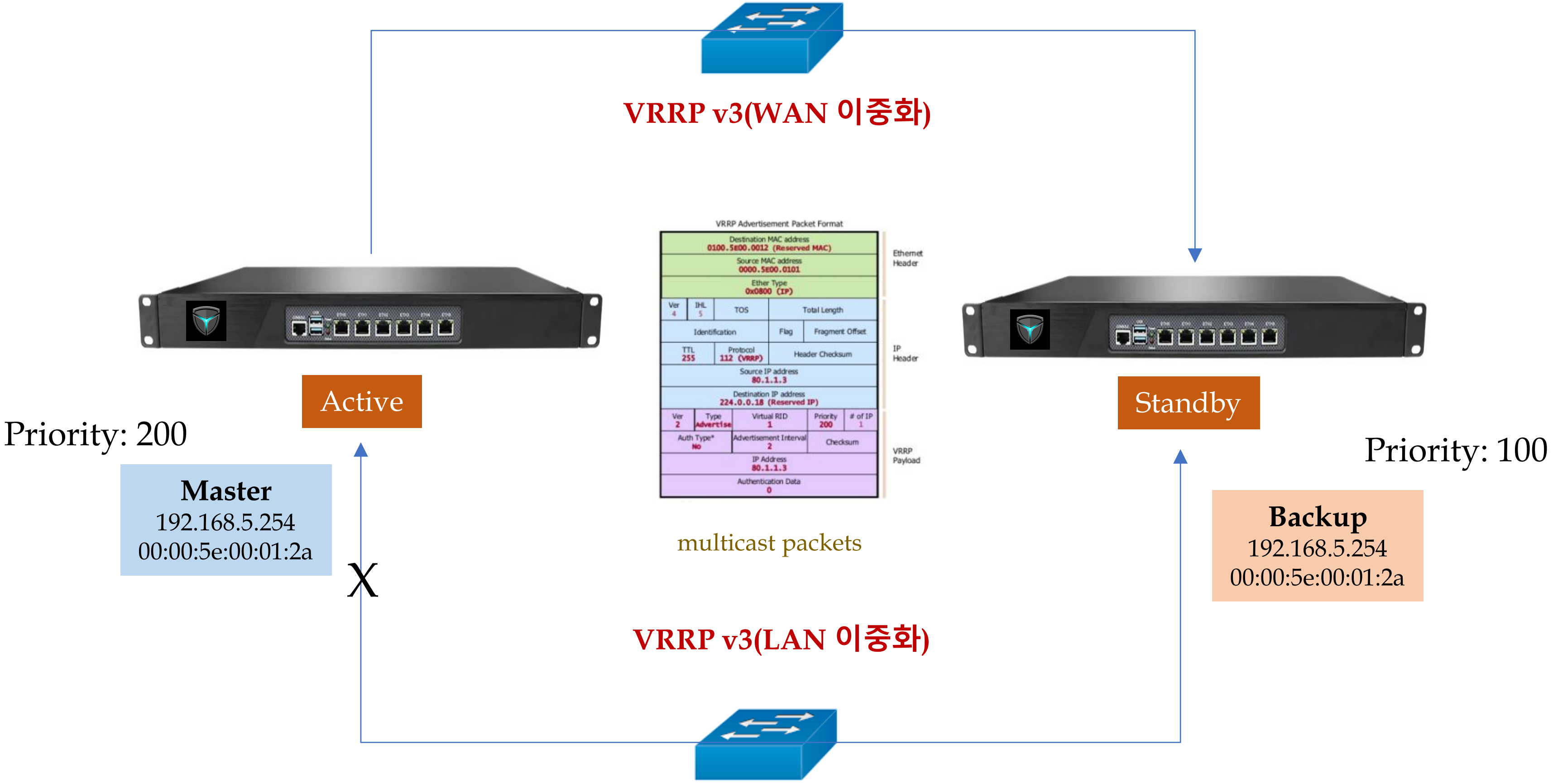



Gateway 방식(Not Inline)

# 1. AI Security Gateway(3) – Redundancy(1)

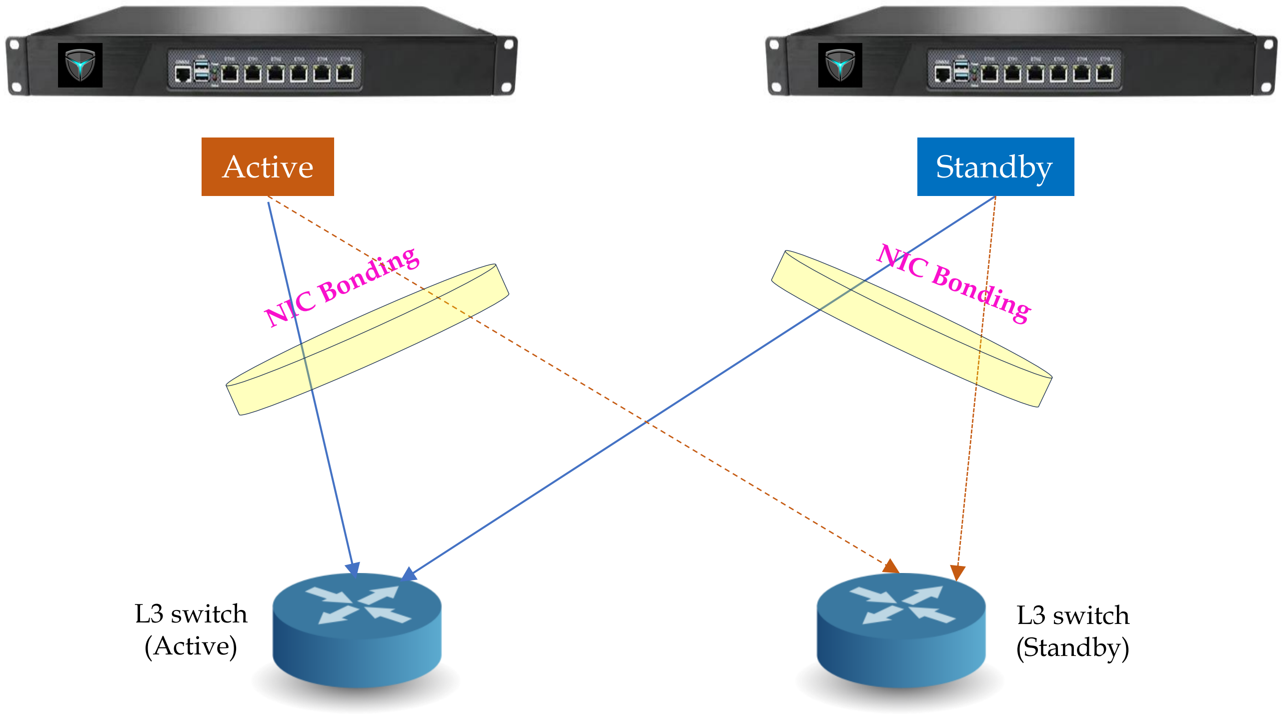



Ref) https://www.netmanias.com/



# 1. AI Security Gateway(3) – Redundancy(2)

**LAN & WAN NIC Bonding**




**<NIC Bonding Mode>**

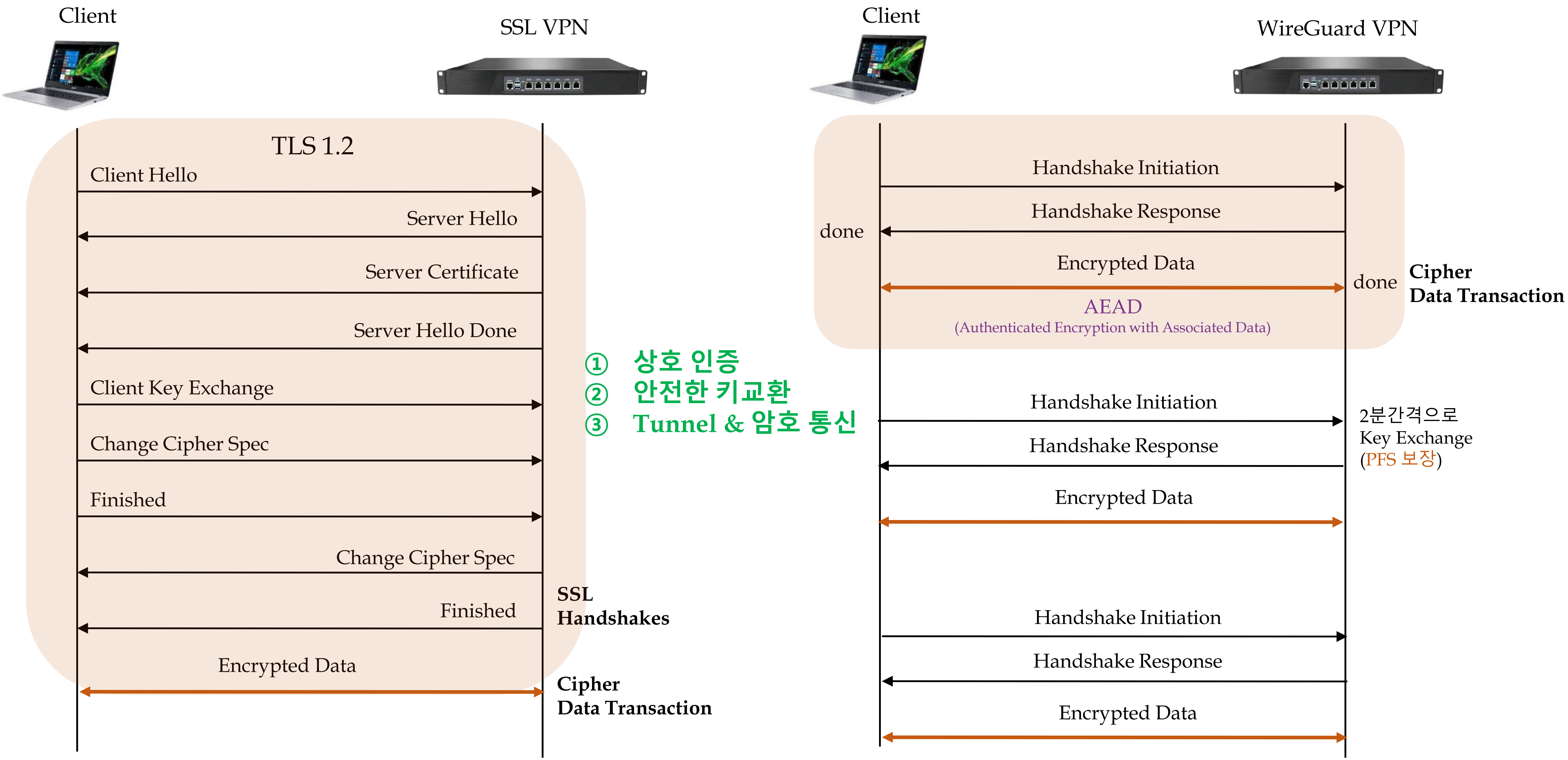
- balance-rr
- active-backup
- balance-xor
- broadcast
- 802.3ad
- …

# 2. WireGuard

## New Generation VPN Protocol

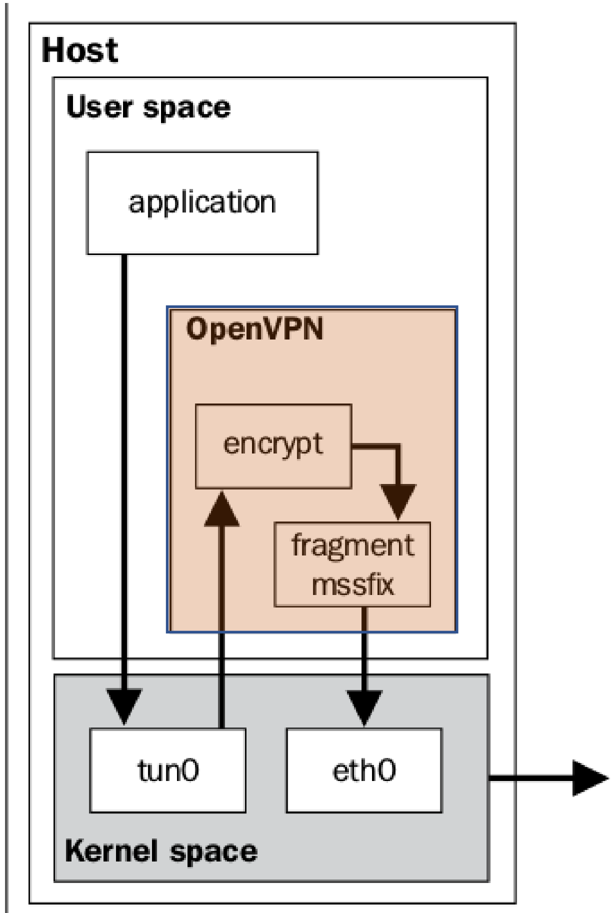
"WireGuard" and the "WireGuard" logo are registered trademarks of Jason A. Donenfeld

# 2. WireGuard(1) – Architecture(1)



# 2. WireGuard(1) - Architecture(2-1)

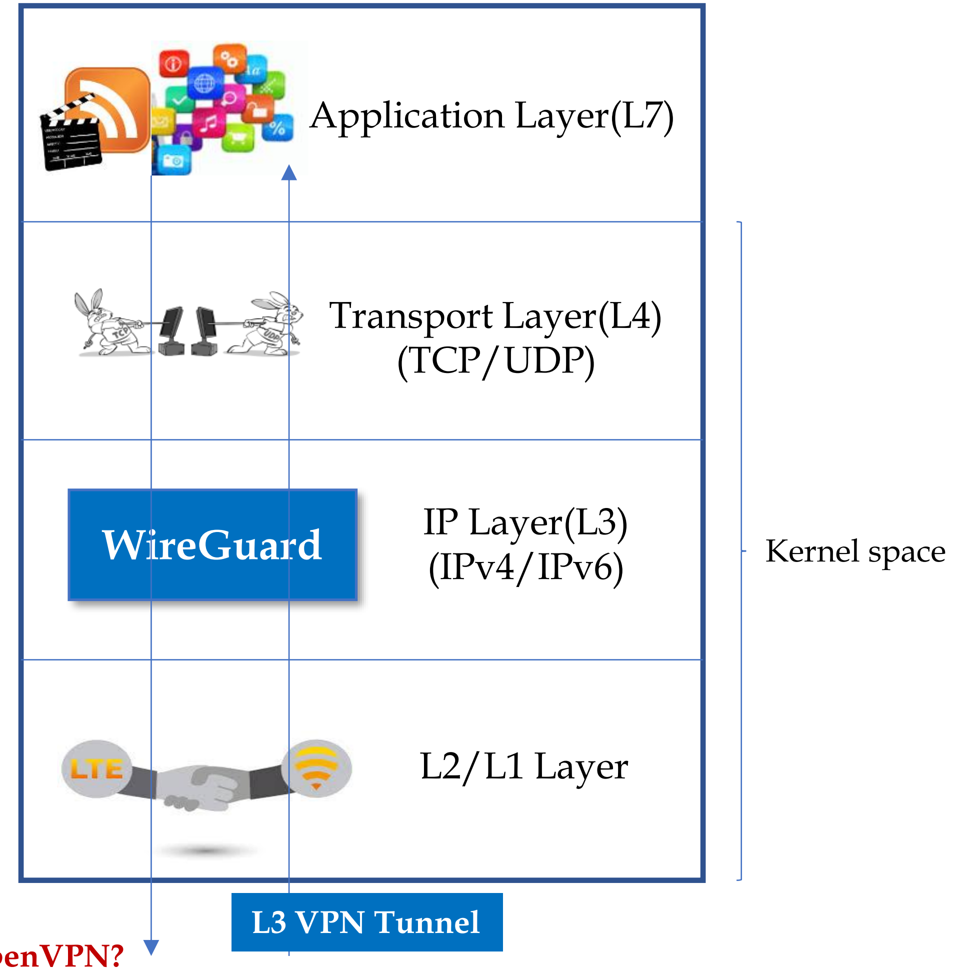
**Limitations of OpenVPN**




**SSL VPN Tunnel(Before DCO)**

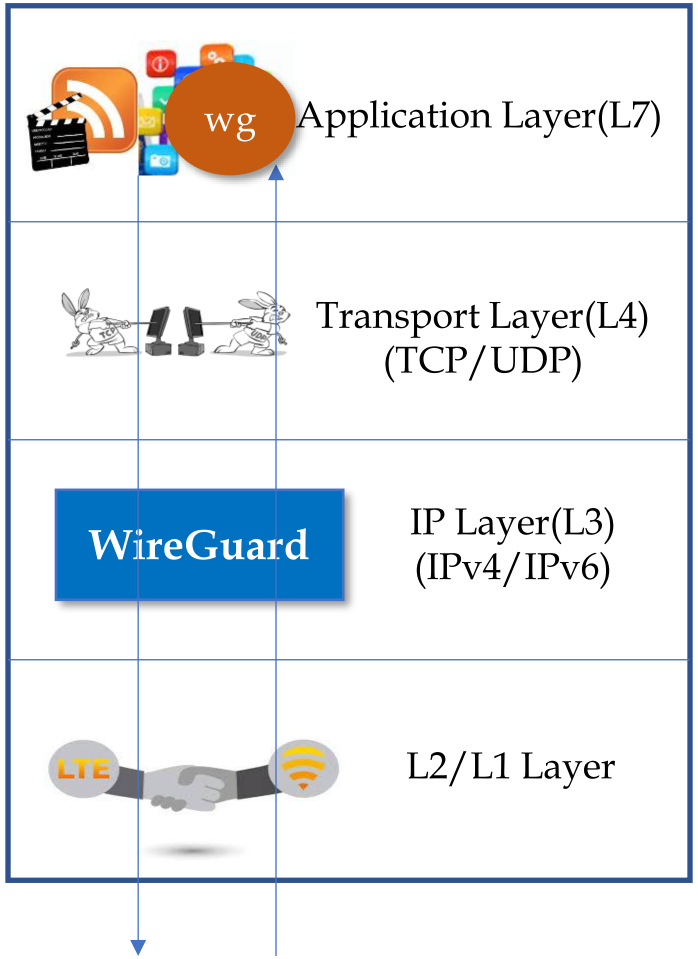
vs

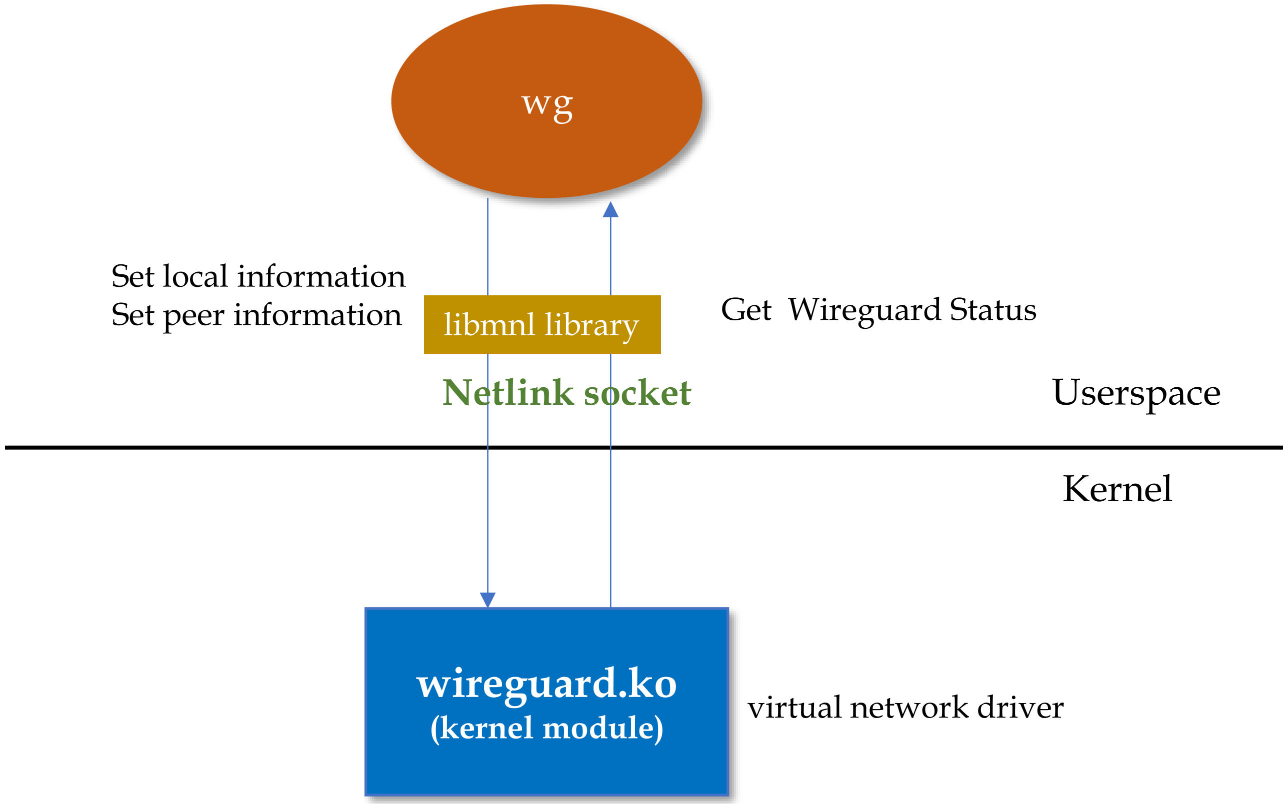
**No Limitation. Cryptokey Routing**



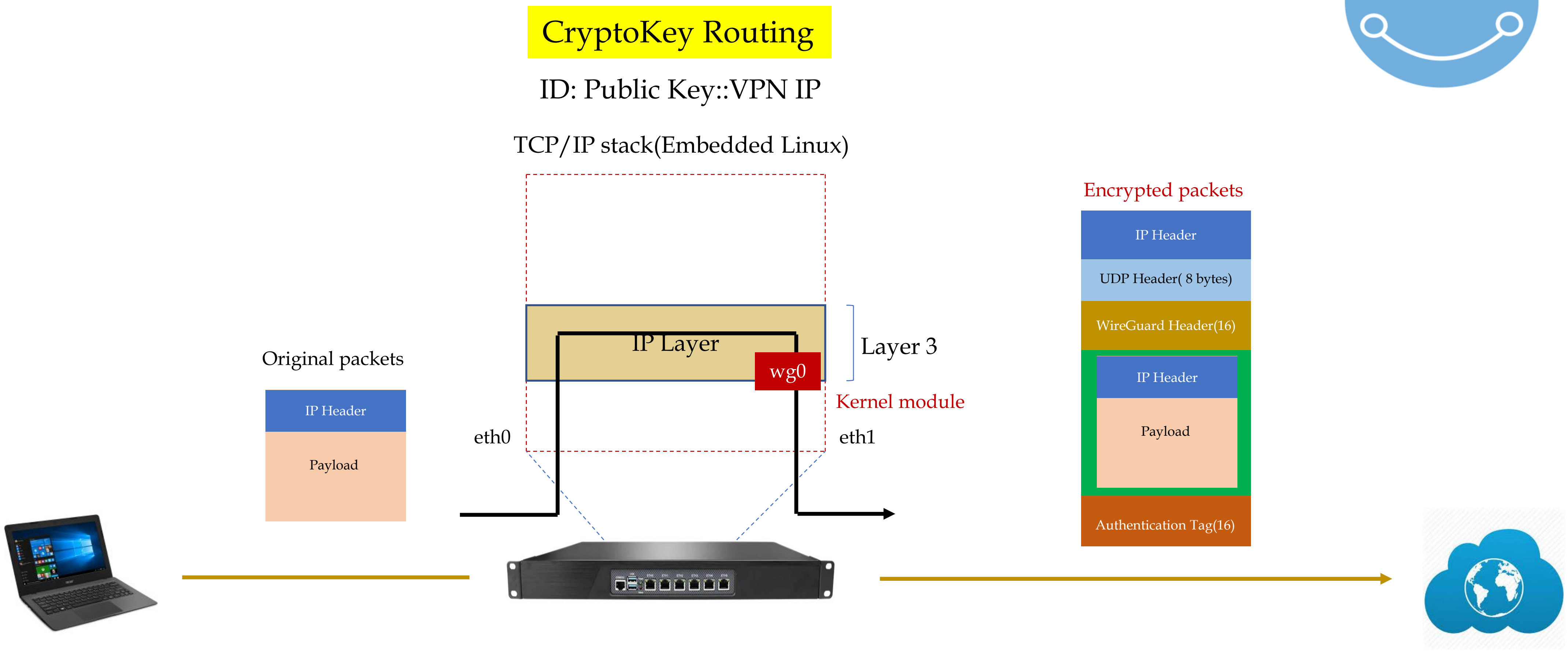

**Why is WireGuard faster than OpenVPN?**

# 2. WireGuard(1) - Architecture(2-2)

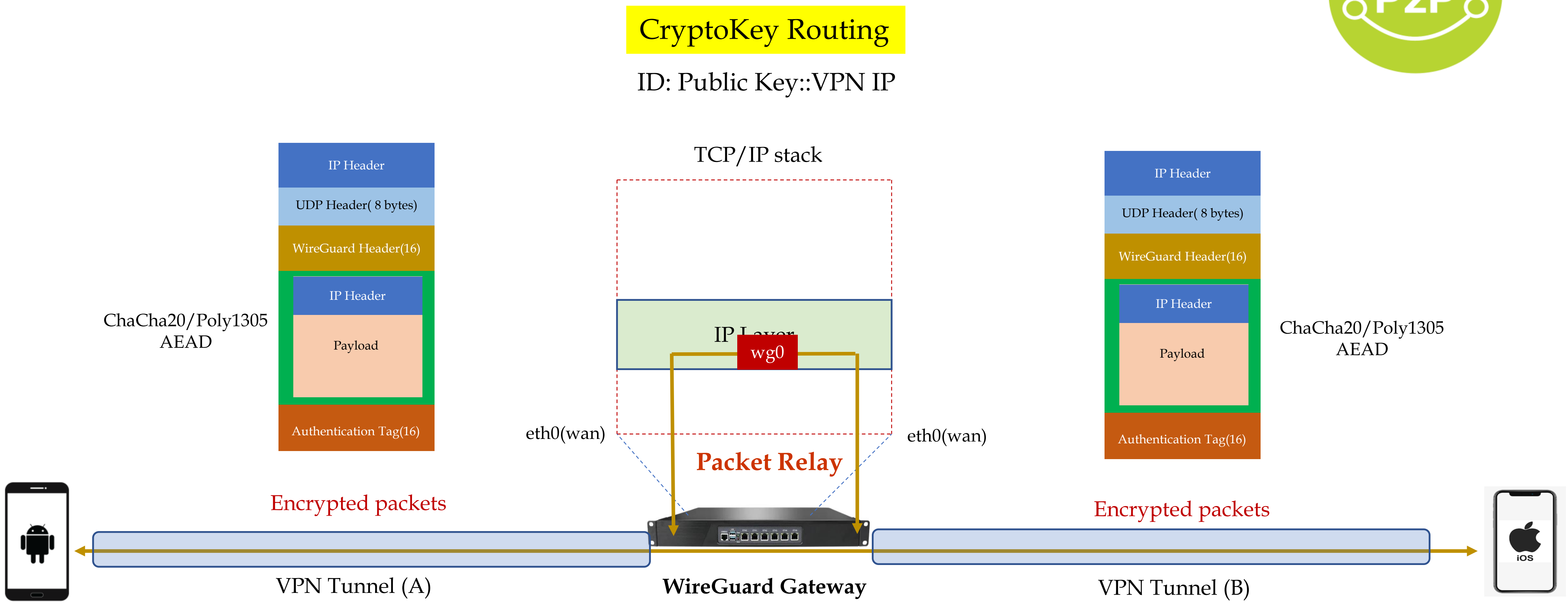


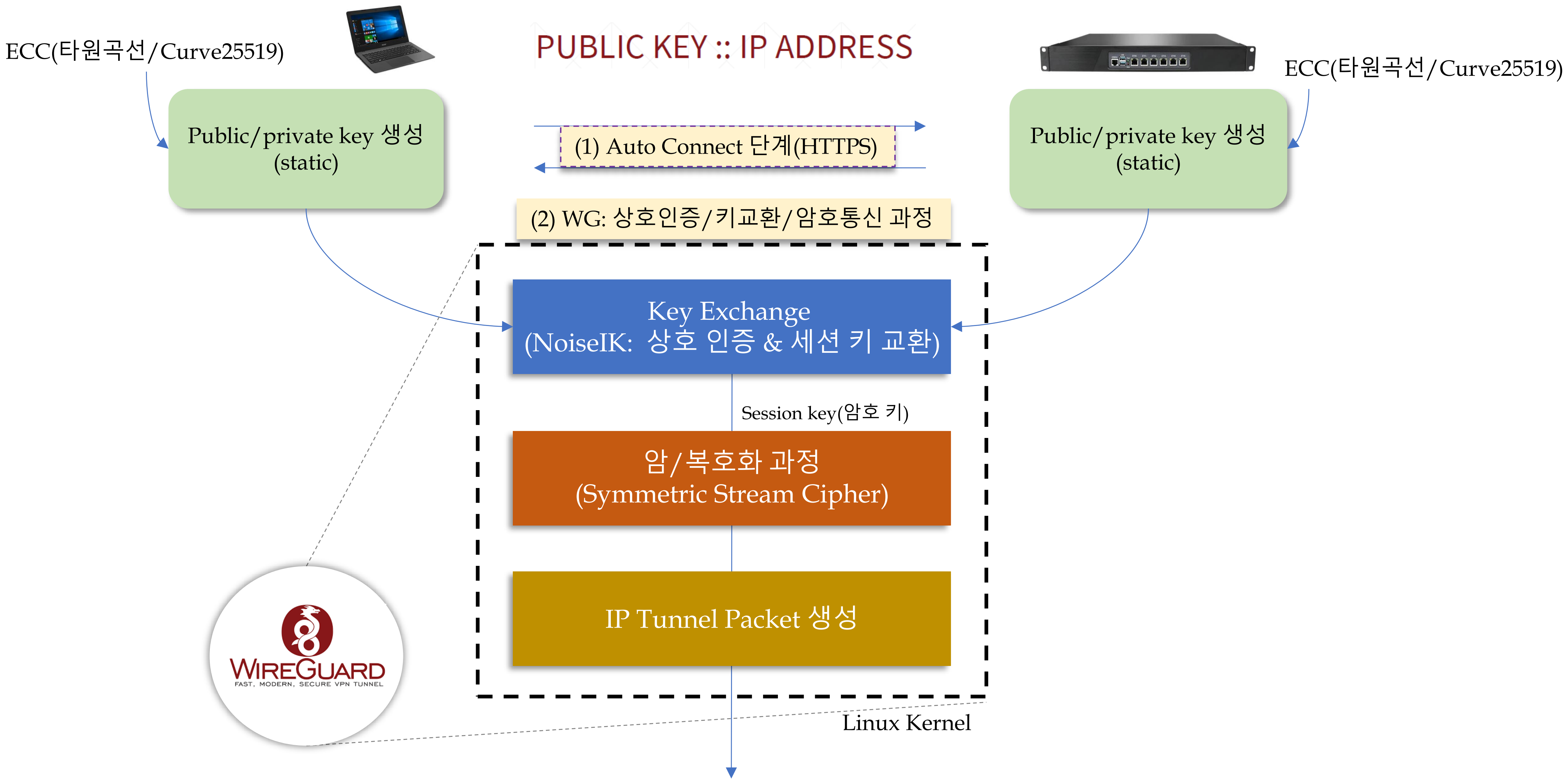
# 2. WireGuard(1) - Architecture(3-1)



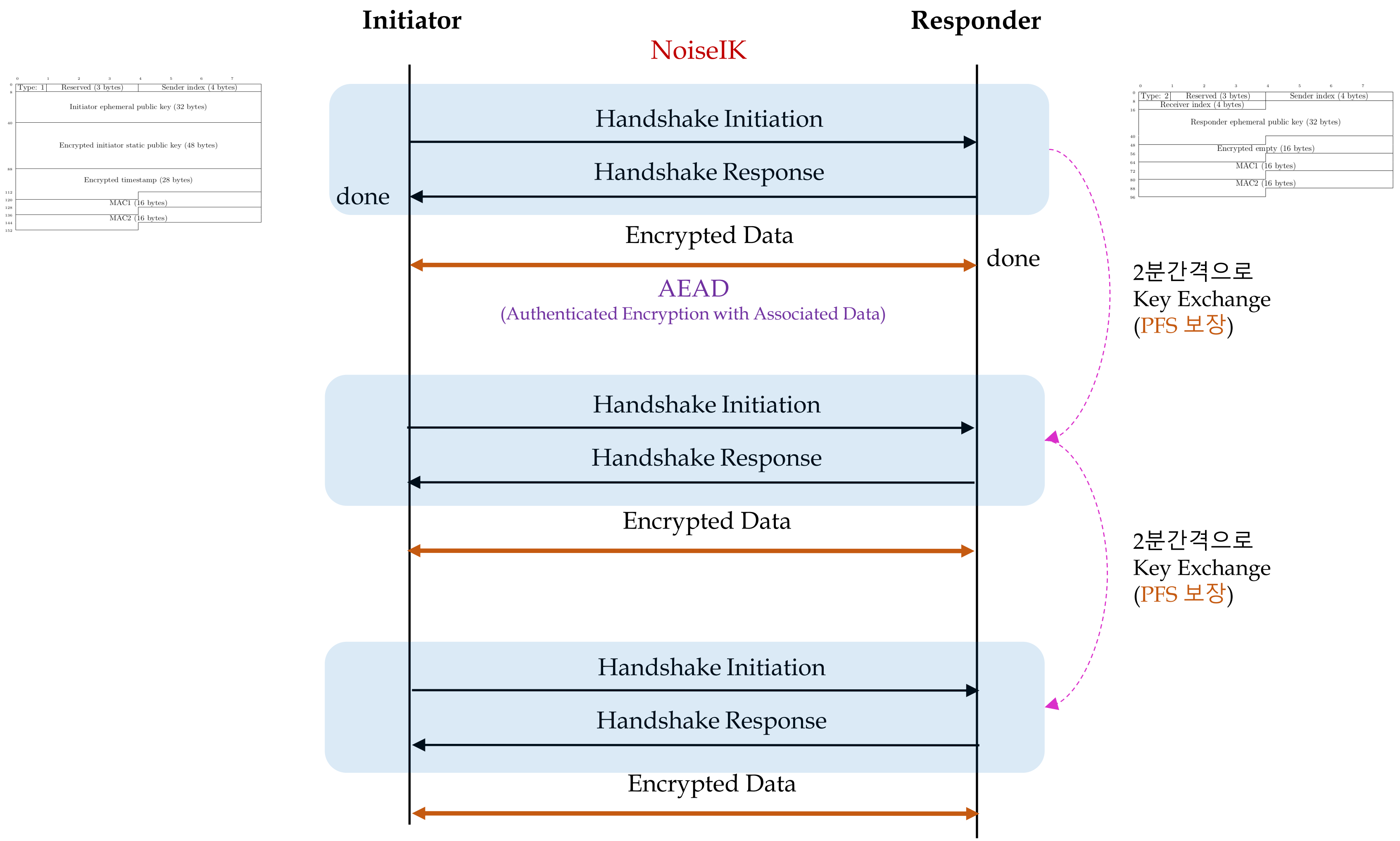
# 2. WireGuard(1) - Architecture(3-2)



# 2. WireGuard(2) - Protocol(1)

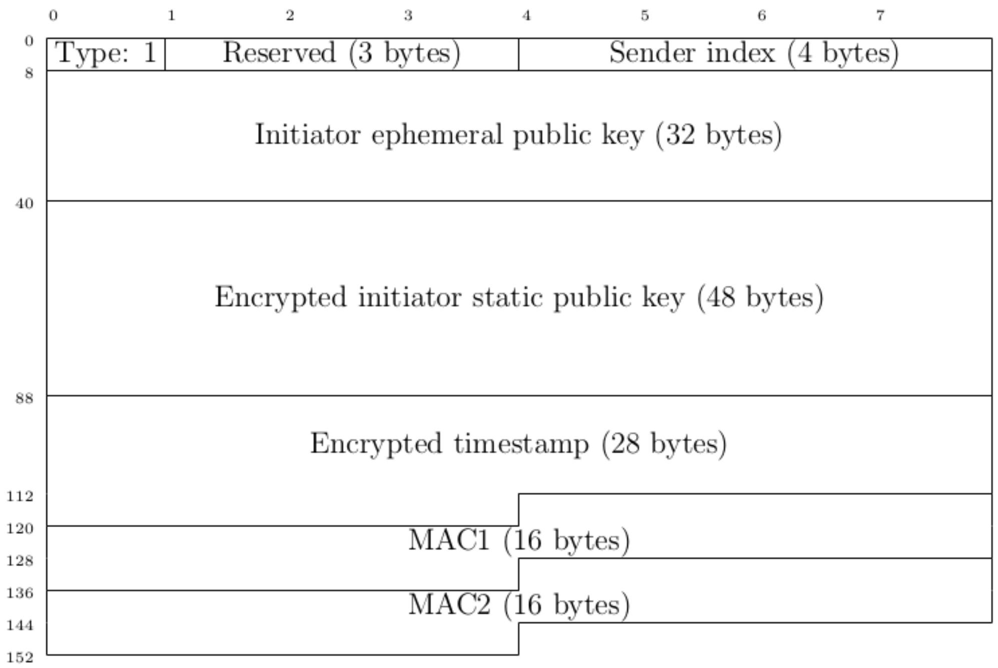


# 2. WireGuard(2) - Protocol(2)

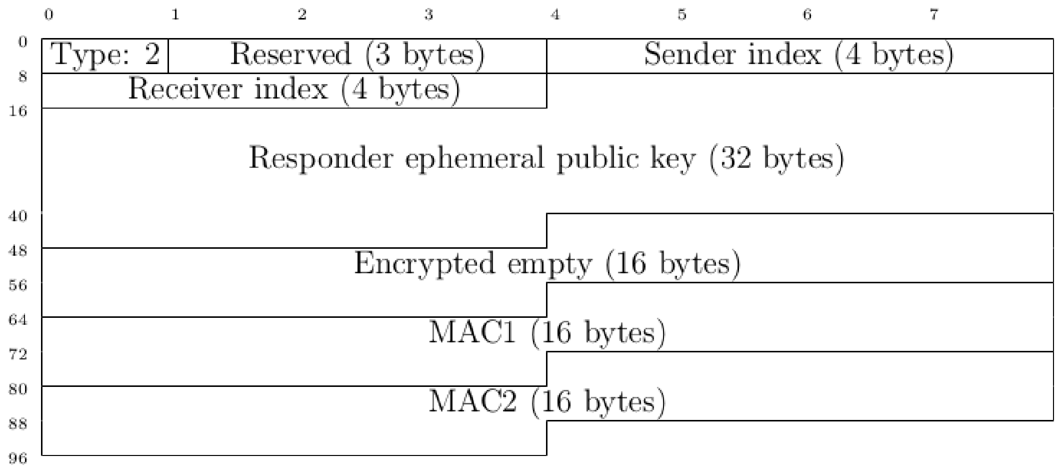


# 2. WireGuard(2) - Protocol(3)

Initiation Packet Format




Response Packet Format



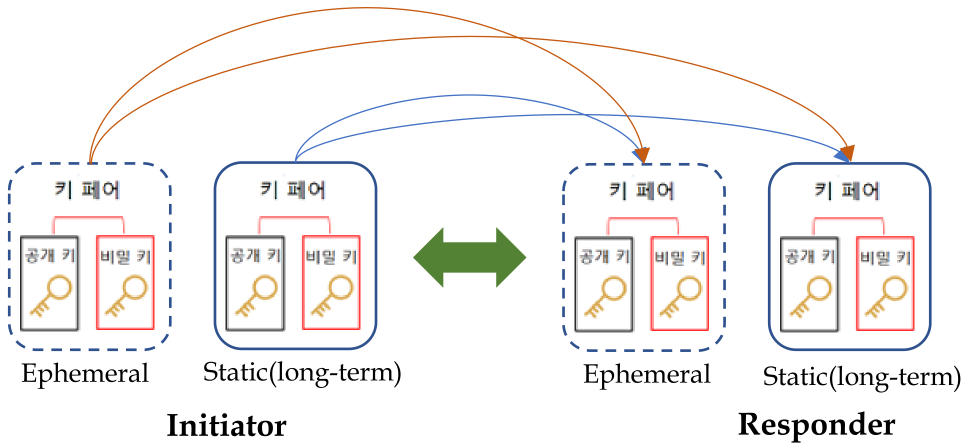

| Type | Message Name | Length |
|---|---|---|
| 1 | Handshake Initiation | 148 bytes |
| 2 | Handshake Response | 92 bytes |
| 3 | Cookie Reply | 64 bytes |
| 4 | Transport Data | $\geq$ 32 bytes |

# 2. WireGuard(2) - Protocol(4)

Session keys = **Noise**(
ECDH(ephemeral, static),
ECDH(static, ephemeral),
ECDH(ephemeral, ephemeral),
ECDH(static, static)
)

256bits(32bytes)


AEAD




상호 인증
및
키 교환

Initiator has long-term DH key-pair $(\mathbf{ssk}_i, \mathbf{spk}_i)$
Responder has long-term DH key-pair $(\mathbf{ssk}_r, \mathbf{spk}_r)$

| Initiator | | Responder |
|---|---|---|
| $(\mathbf{esk}_i, \mathbf{epk}_i) \leftarrow \mathrm{DH.Gen}()$ | | |
| | $\xrightarrow{\mathbf{epk}_i}$ | |
| | | $(\mathbf{esk}_r, \mathbf{epk}_r) \leftarrow \mathrm{DH.Gen}()$ |
| | $\xleftarrow{\mathbf{epk}_r}$ | |
| $k_1 \leftarrow \mathrm{DH.Shared}(\mathbf{esk}_i, \mathbf{spk}_r)$ | | $k_2 \leftarrow \mathrm{DH.Shared}(\mathbf{ssk}_r, \mathbf{epk}_i)$ |
| $k_2 \leftarrow \mathrm{DH.Shared}(\mathbf{ssk}_i, \mathbf{epk}_r)$ | | $k_3 \leftarrow \mathrm{DH.Shared}(\mathbf{esk}_r, \mathbf{spk}_i)$ |
| $k_3 \leftarrow \mathrm{DH.Shared}(\mathbf{esk}_i, \mathbf{epk}_r)$ | | $k_4 \leftarrow \mathrm{DH.Shared}(\mathbf{esk}_r, \mathbf{epk}_i)$ |
| $k_4 \leftarrow \mathrm{DH.Shared}(\mathbf{ssk}_i, \mathbf{spk}_r)$ | | $k_1 \leftarrow \mathrm{DH.Shared}(\mathbf{ssk}_r, \mathbf{spk}_i)$ |

Derive session key from $k_1$, $k_2$, $k_3$, and $k_4$

# 2. WireGuard(3) – Algorithms(1)

**Layer 3(IPv4, IPv6 지원) VPN Protocol**

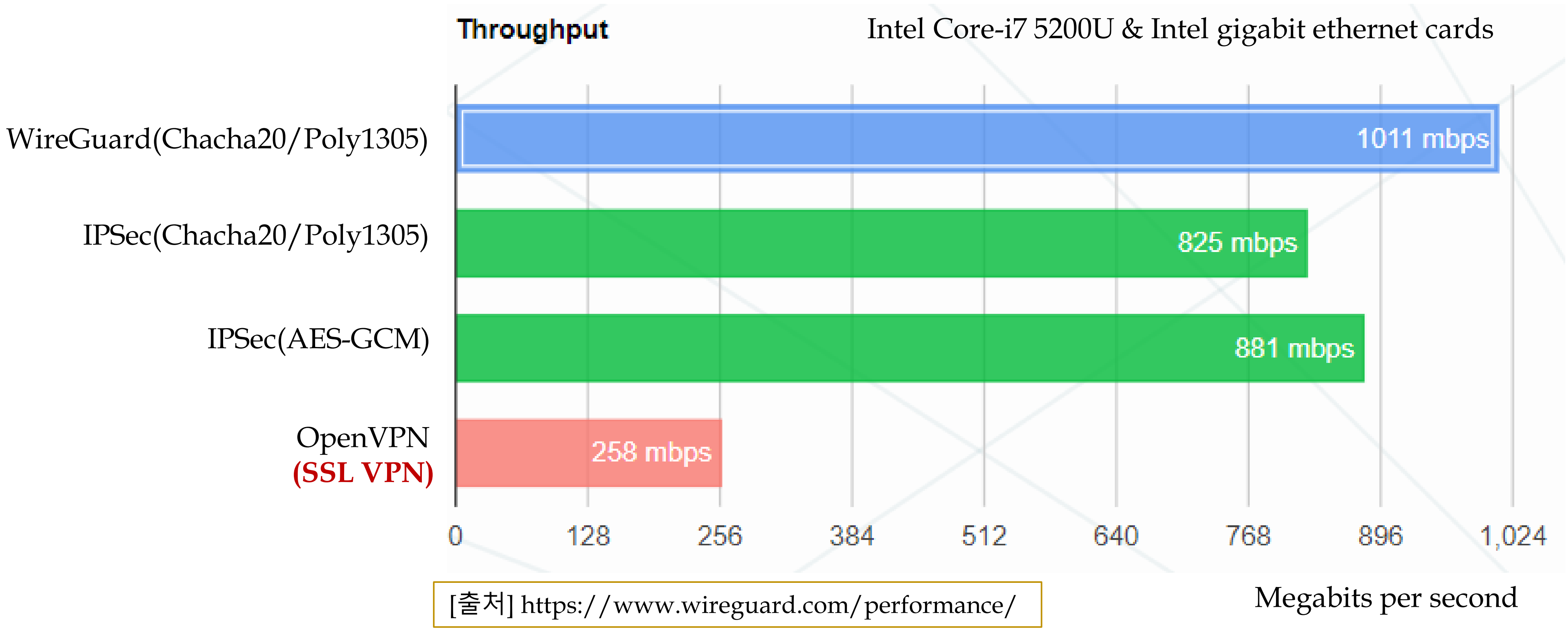
| | |
|---|---|
| handshake pattern | IKpsk2 **(Noise Protocol Framework 기반)** |
| DH function | X25519 |
| cipher function | ChaCha20-Poly1305 |
| hash function | BLAKE2s |
| prologue | `WireGuard v1 zx2c4 Jason@zx2c4.com` (34 bytes) |

# 2. WireGuard(3) - Algorithms(2)

<table>
<tr><th>보안 알고리즘</th><th>상세 내용</th></tr>
<tr><td>NoiseIK<br>Key 교환 방식<br>및<br>상호 인증</td><td>NoiseIK handshake 방식(Noise IKpsk2)<br>▪ ECDH(Diffie-Hellman) 기반<br>▪ Curve25519 Public key(32 byte)를 교환 후, 이를 통해 안전하게 shared secret 생성<br>▪ Static/Ephemeral private/public key(2 쌍) 이용<br>▪ Key 교환 시 아래 기능 보장<br>▪ Perfect forward secrecy 보장, replay attack 방지 기능<br>▪ Identity 감춤 기능 제공, DoS 공격 완화 기능(Cookie),<br>▪ PSK 사용 시, post-quantum security 보장<br>Hash 알고리즘<br>▪ BLAKE2 – fast secure hashing 알고리즘<br>▪ SHA series 보다 빠름. 즉 MD5 수준임. SHA-3 보안 강도.<br>PRF & MAC 및 Key Derivation 함수<br>▪ SipHash4 (Hash table lookup 용으로 사용)<br>▪ HKDF (HMAC-based Key Derivation Function) 사용</td></tr>
<tr><td rowspan="2">암호 알고리즘<br>Authenticator 알고리즘<br>(ChaCha20-Poly1305 AEAD)</td><td>ChaCha20 – 256 bits stream cipher(20 round cipher Salsa20 기반)<br>▪ Stream cipher는 일반적인 block cipher(예: AES-256-CBC)에 비해 속도가 빠름<br>▪ key(32 bytes)는 대칭키를 사용(즉, 암호화 용 키와 복호화 용 키 동일)<br>▪ Video/Audio 등 stream 암호화에 적합</td></tr>
<tr><td>Poly1305 - message authentication code 알고리즘(16 bytes output 생성)</td></tr>
</table>

**Noise_IKpsk2_25519_ChaChaPoly_BLAKE2s**

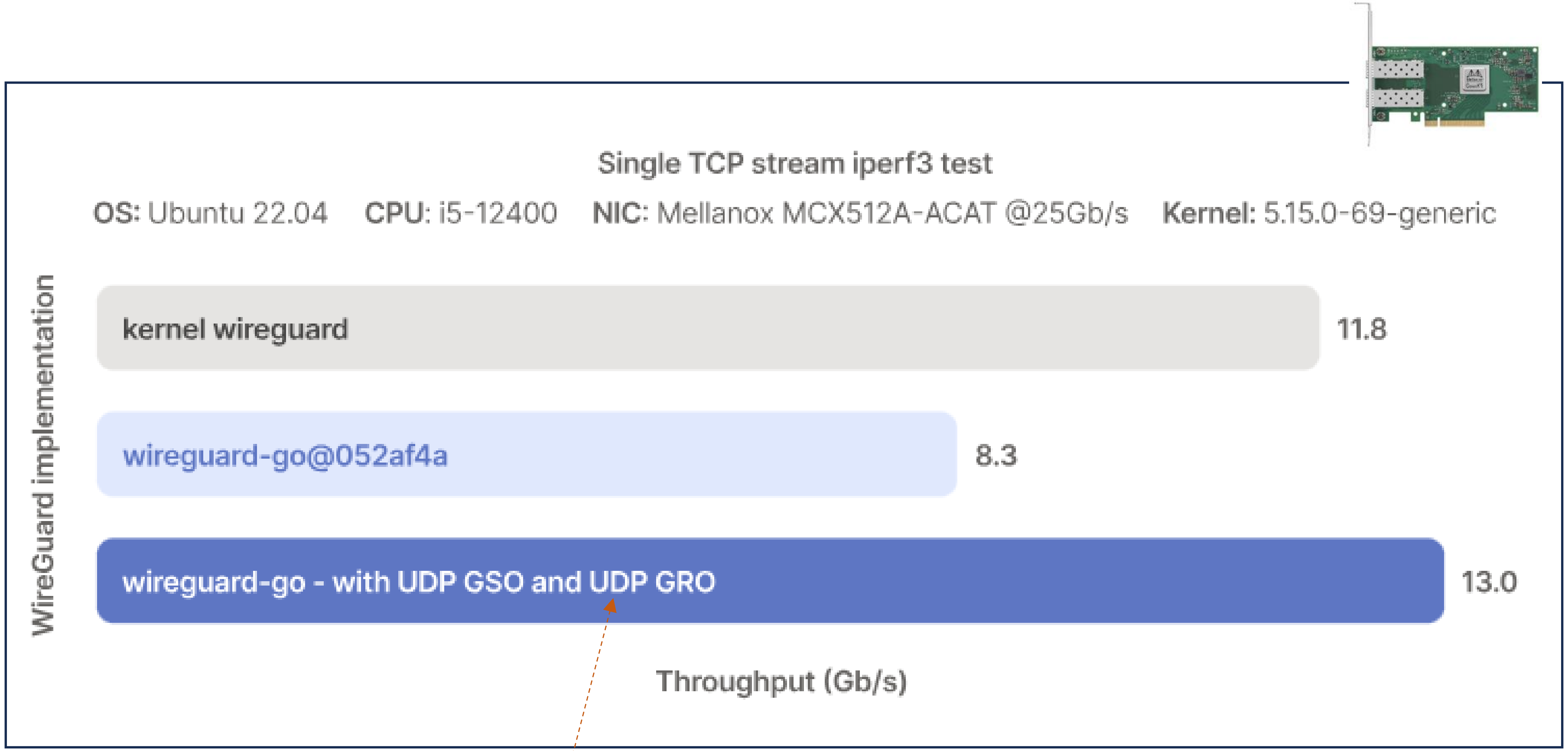
# 2. WireGuard(4) – Performance(1)




[출처] https://www.wireguard.com/performance/

[주의] OpenVPN DCO(Data Channel Offload)는 위의 그림보다는 훨씬 빠르다.

# 2. WireGuard(4) - Performance(2)




Packet Vector Processing
UDP GRO/GSO
Checksum Offload

그림 출처: https://tailscale.com/blog/more-throughput

# 2. WireGuard(4) - Performance(3)

- **WireGuard VPN의 우수한 성능 (Multi-Core 환경)**

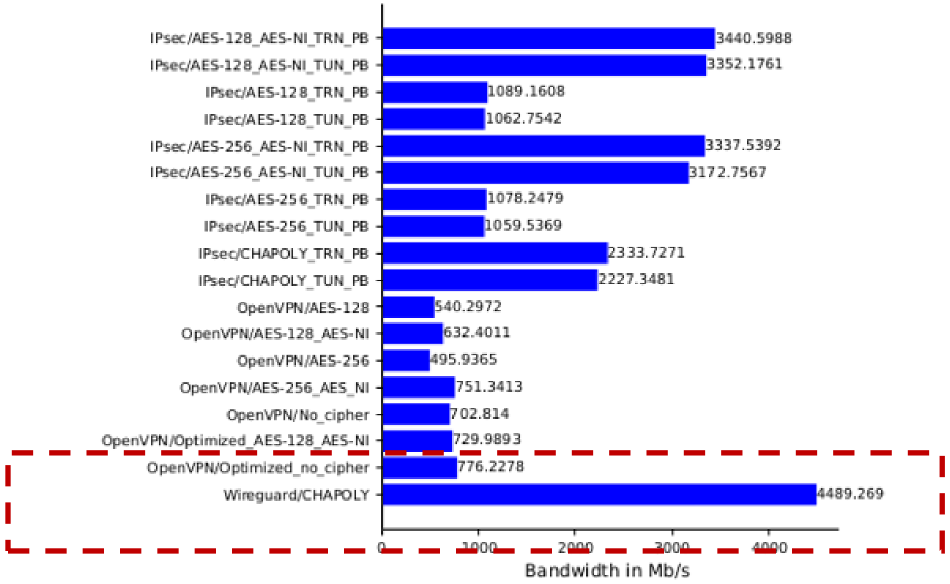
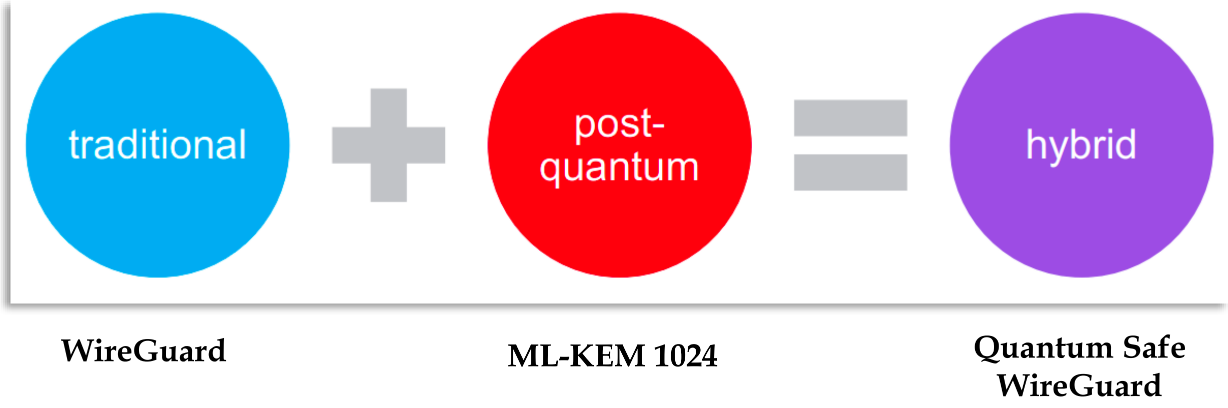
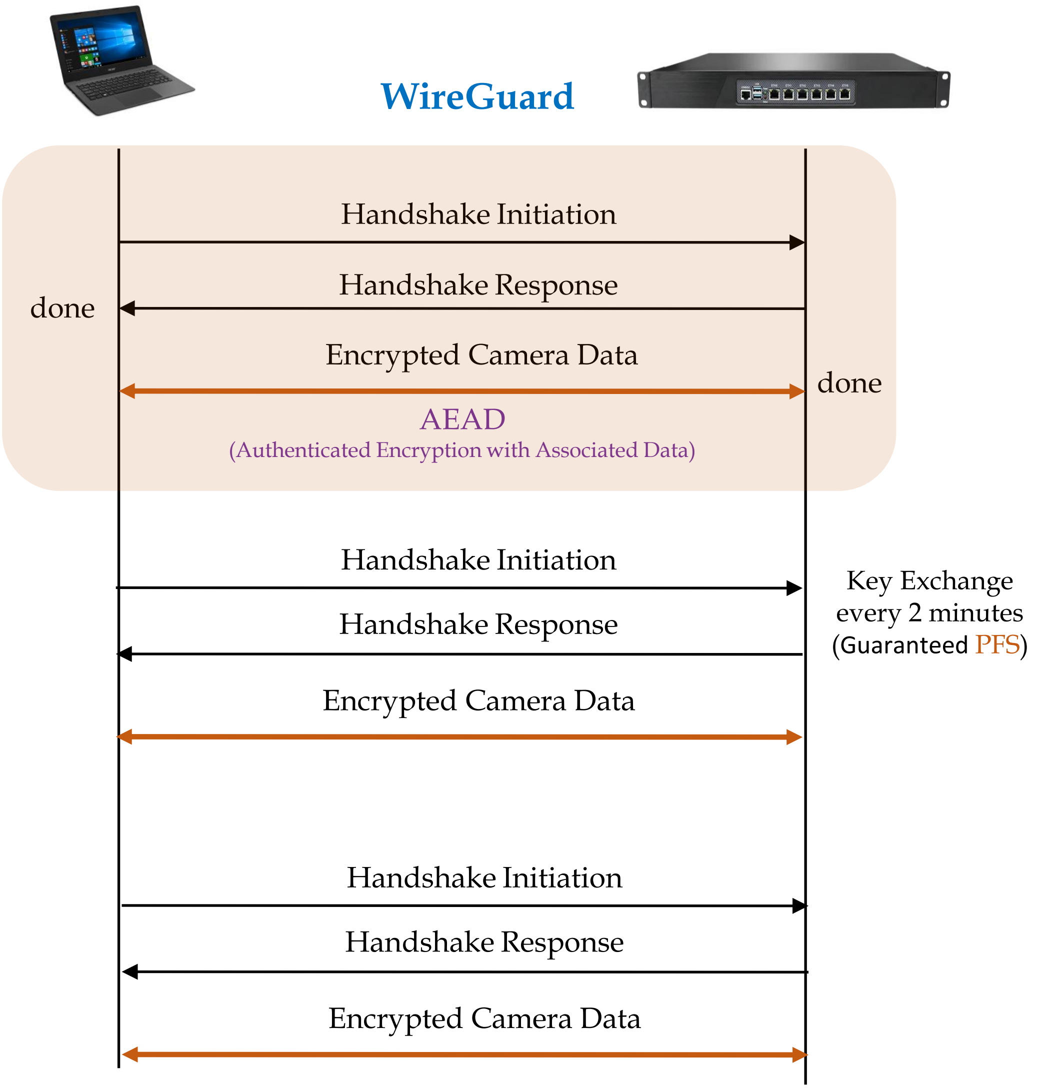



그림 출처: Performance Comparison of VPN Solutions, Lukas Osswald

- **Signature 없이 인증 & 키 교환하는 AKE 기법 도입**
- **최신 암호 알고리즘 및 암호 가속 기능으로 무장**
  - Curve25519, Blake2s, ChaCha20, Poly1305, SipHash4
  - X86_64 AVX2, AVX512
- **Perfect Forward Secrecy 보장(2분 간격 Handshaking)**
  - 1-RTT만에 Handshaking 완료
- **Replay Attack 방지 기능(Counter 필드 유지/관리)**
- **Identity Hiding 기능**
- **DoS 공격 완화 기능(Cookie 활용)**

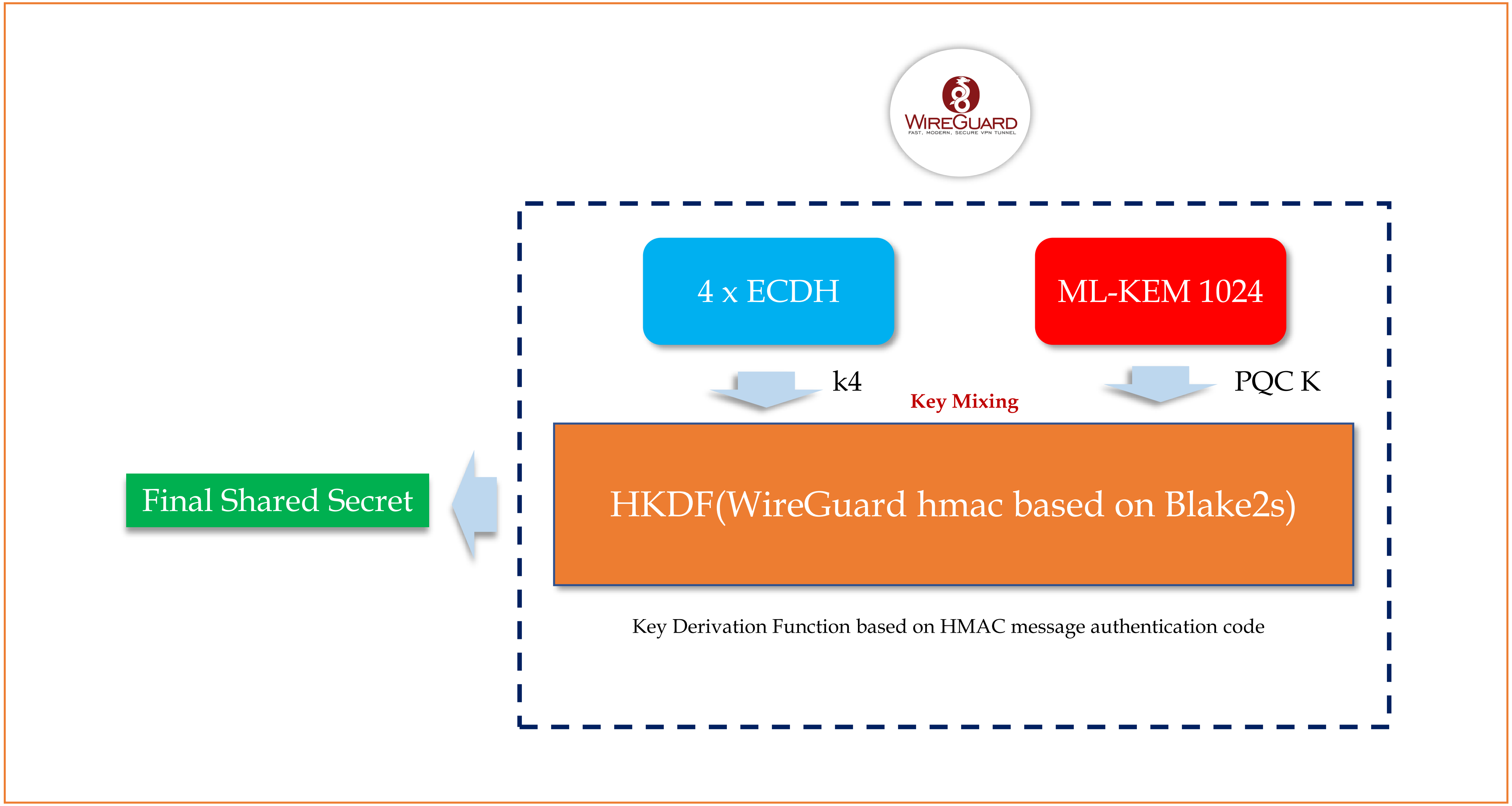
# 2. WireGuard(5) – PQC(1)



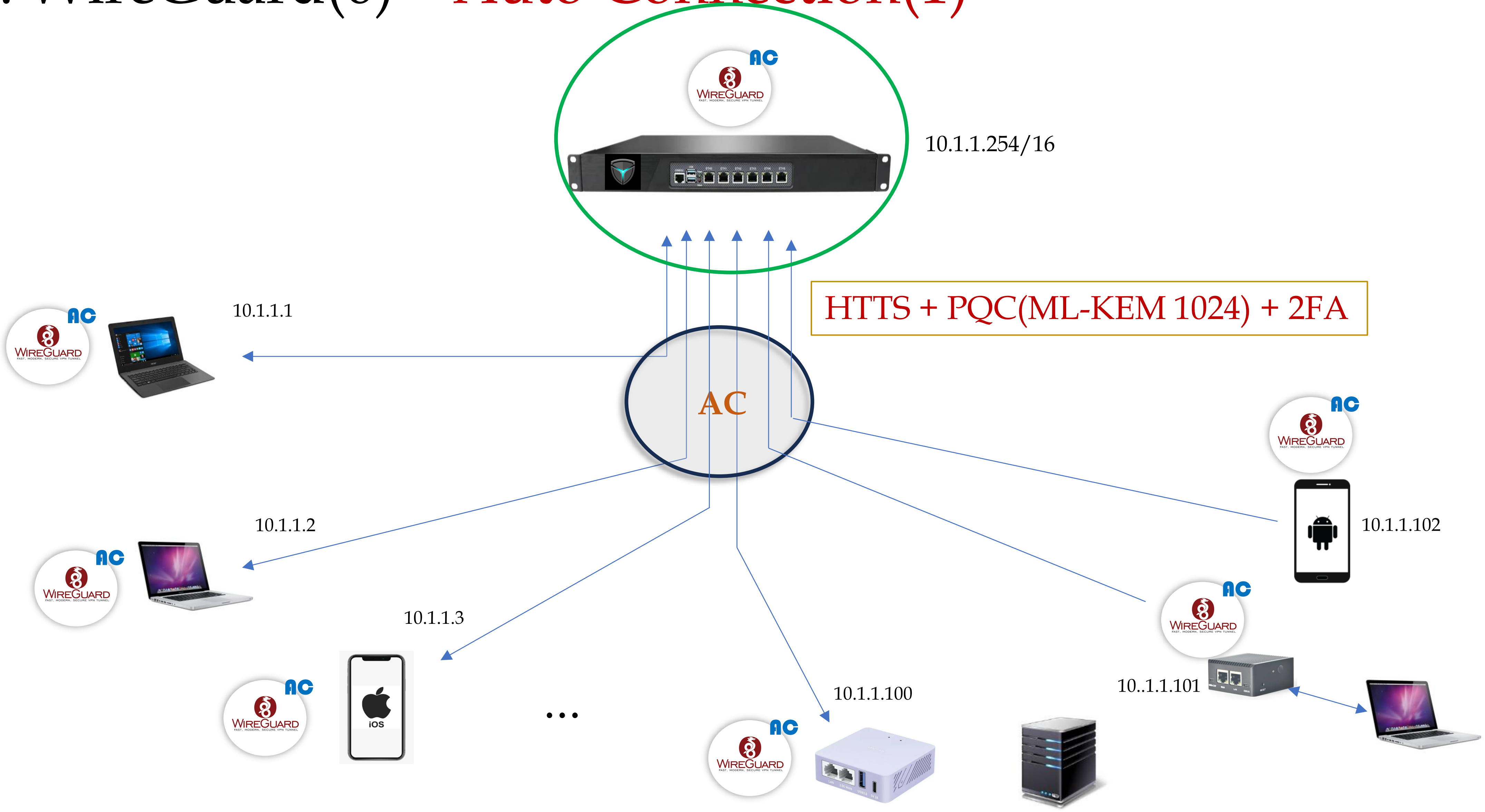




**PQC 알고리즘이 통합된 WireGuard Protocol**

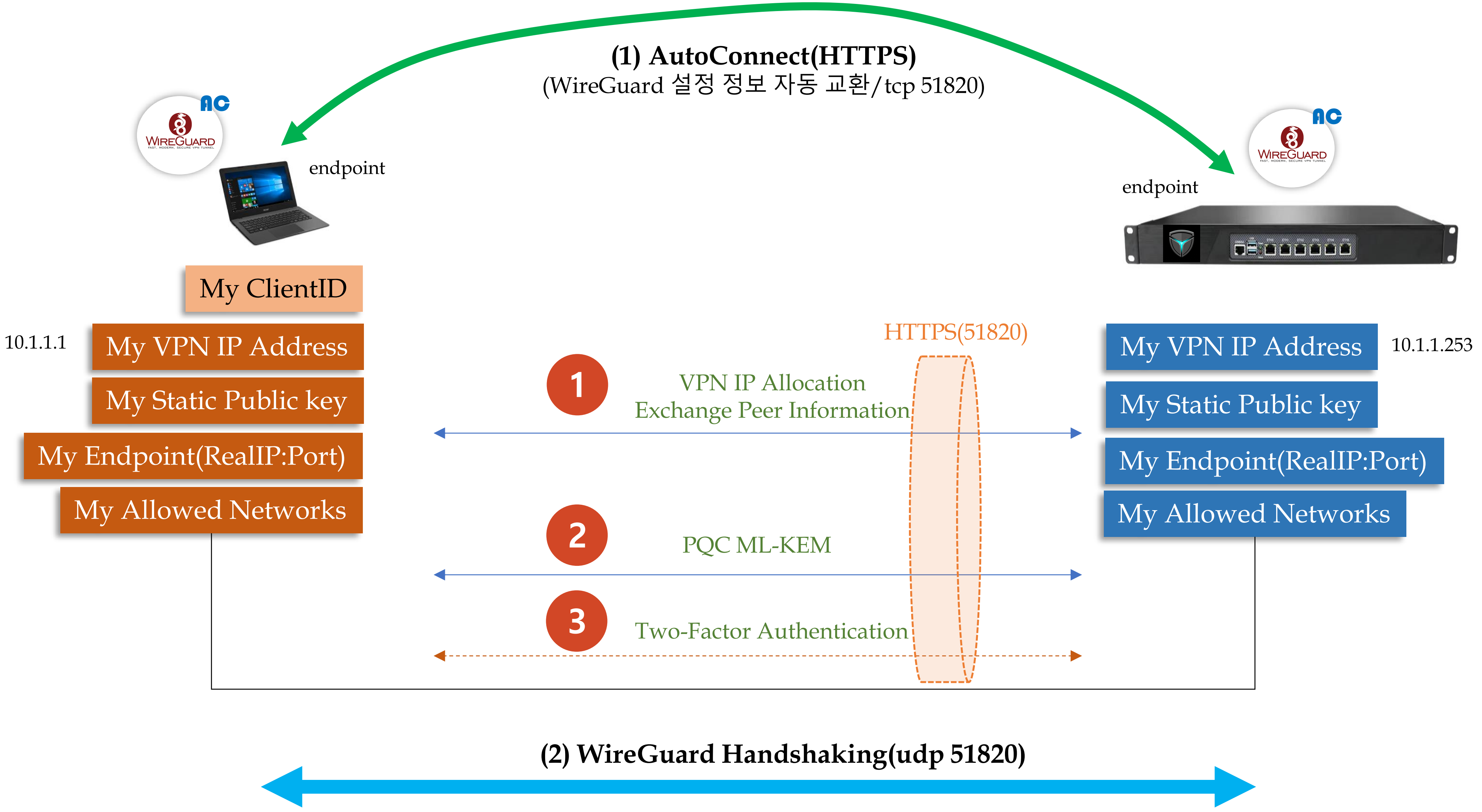
# 2. WireGuard(5) – PQC(2)



# 2. WireGuard(6) – Auto Connection(1)



# 2. WireGuard(6) – Auto Connection(2)



# 3. ZTNA

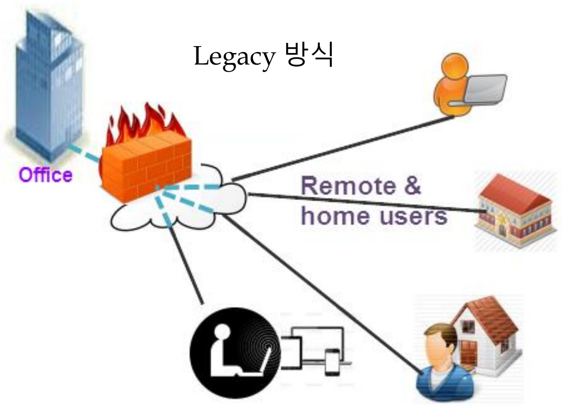
**Zero Trust Network Access**
**based on eBPF**

ZERO Trust

# 3. ZTNA(1) – Concept(1)

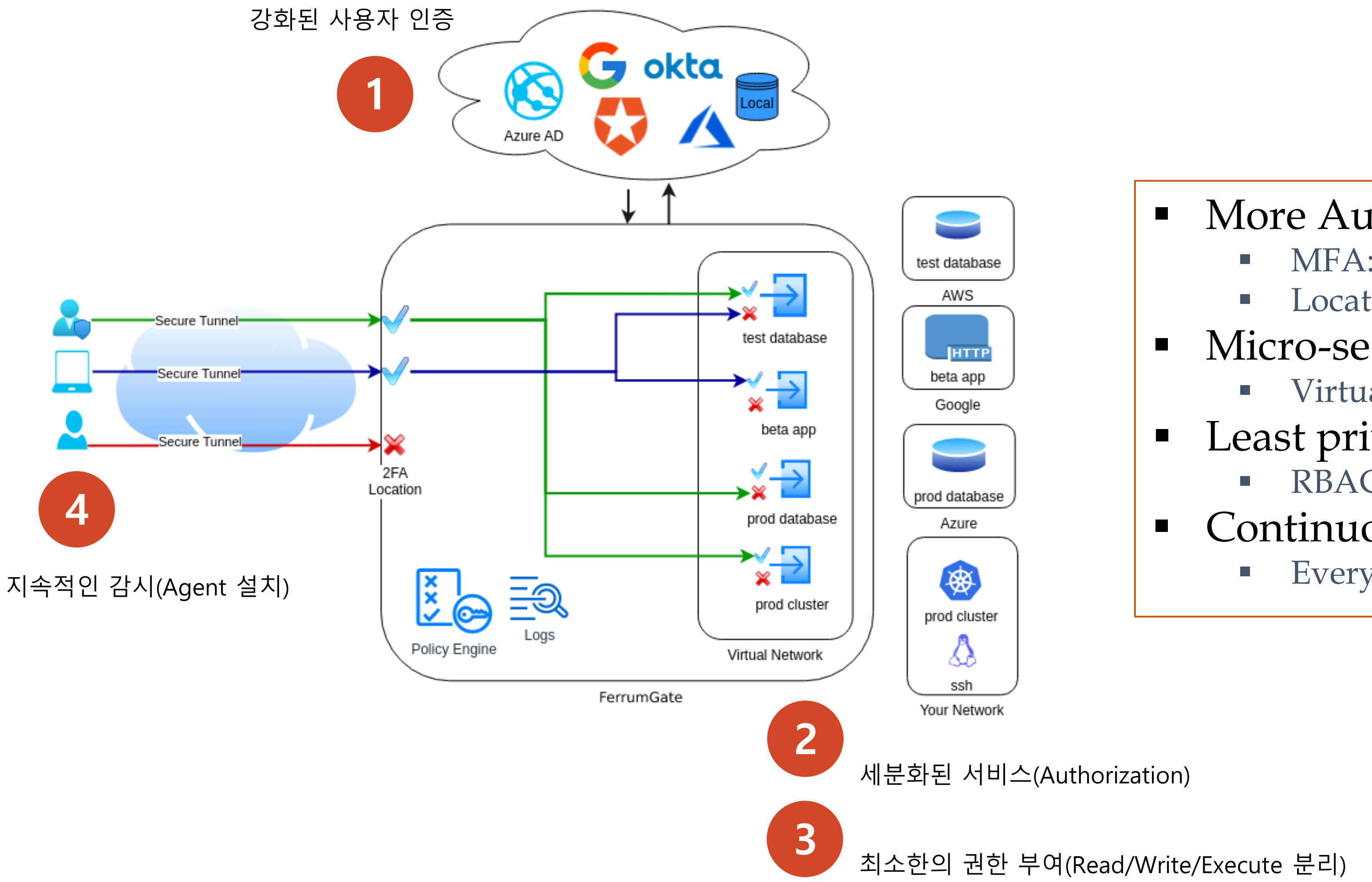
- **Legacy 방식(Firewall/VPN/IPS/UTM)**
  - Internal network(Trusted Area)과 External network(Untrusted Area) 구분
  - Internal network <=> Perimeter Security 제품 ⇔ External network : 안전한 내부망과 불안전한 외부망 경계(Perimeter)에 보안 제품 설치
  - Static Configuration, IP/Port 기반의 통제

**VPN is Dead**




- **ZTNA or SDP(Software Defined Perimeter) 방식**
  - 환경의 변화: **Cloud, Mobile, BYOD**
    - *이제는 중요한 것들이 Perimeter 바깥에 있는 세상이 되었다.*
  - **Zero Trust**: "Internal, External network 모두 안전하지 않다"에서 출발
  - S/W 방식의 **중앙 Controller**에 의한 접근 통제
    - *실제 network에 접근하기 전에, 인증(Authentication) & 접근 통제(Authorization)을 수행하는 개념*
    - *내부 Resource는 Appliance(Gateway)에 의해 모두 Hidden 상태, 인증을 통과한 후에 외부에 Open되는 구조.* **Dark Network(외부에 보이지 않는 network) 개념 출현**
  - Dynamic Control, IP 기반이 아니라, 사용자 기반의 통제

# 3. ZTNA(1) – Concept(2)

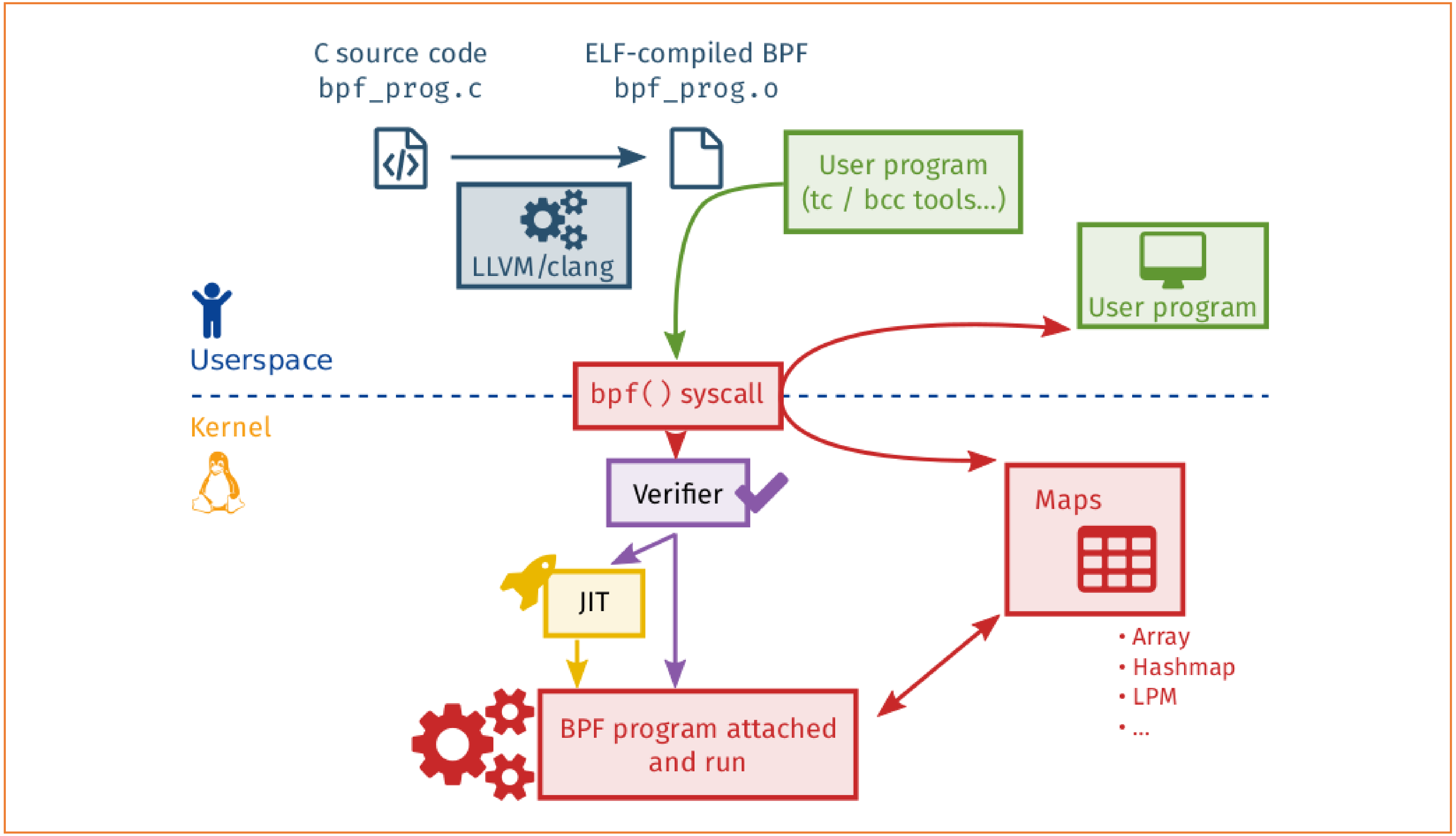



- More Authentication Options
  - MFA: SAML, OAUTH2, LDAP, FIDO2
  - Location Check, Time Check, Device Posture Check
- Micro-segmentation
  - Virtual Application Networks
- Least privilege access
  - RBAC(Role Based Access Control)
- Continuous monitoring
  - Every packet, every flow, every connection

[출처] https://ferrumgate.com/

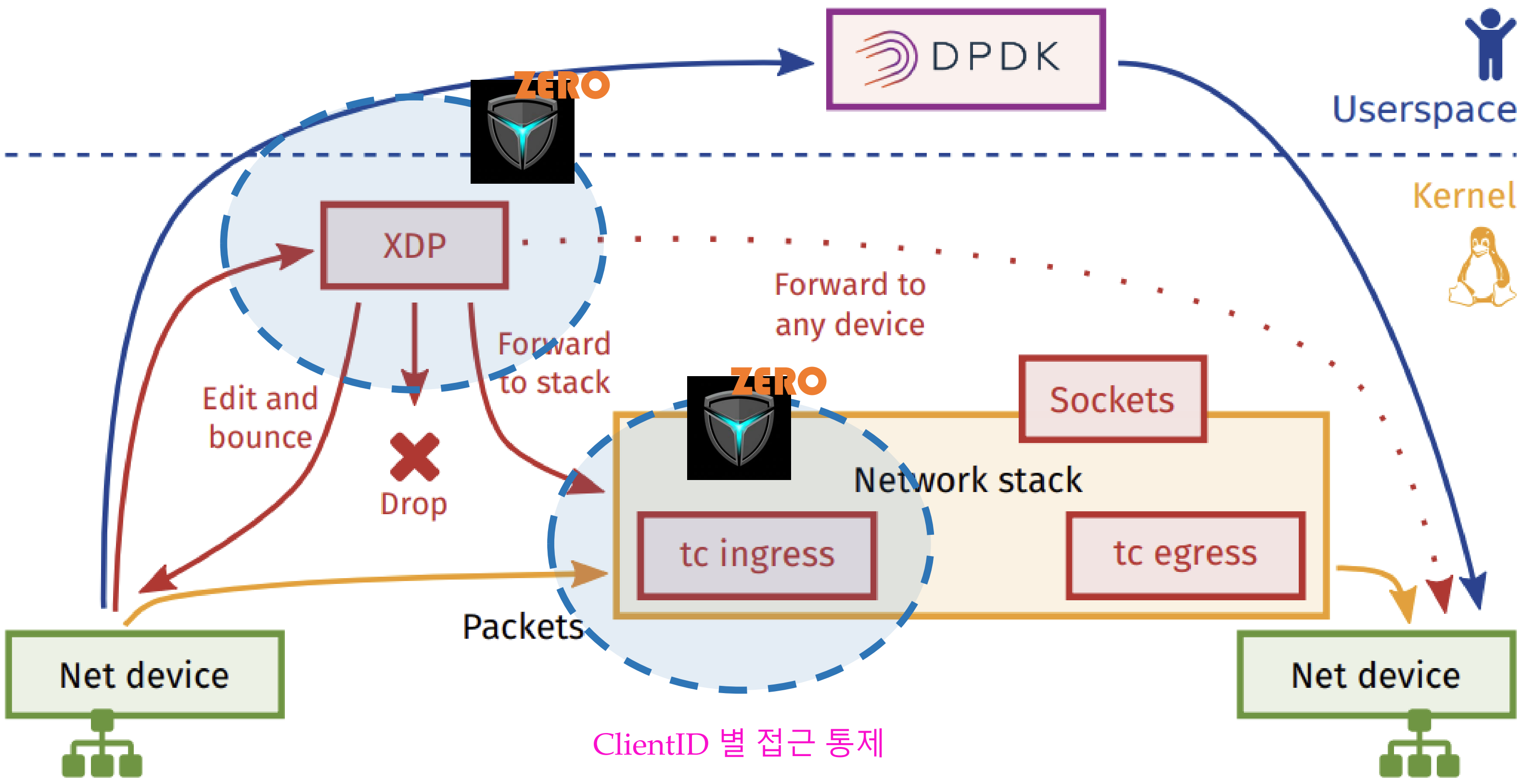
# 3. ZTNA(2) – Two-Factor Authentication

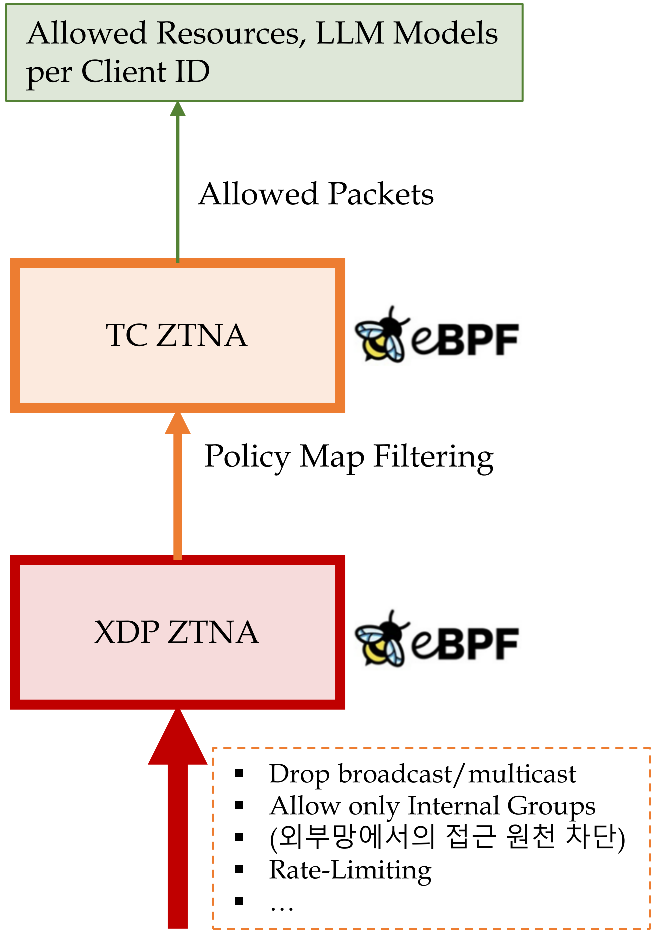
# 3. ZTNA(3) – eBPF ZTNA Enforcement(1)




[출처] https://media.frnog.org/FRnOG_28/FRnOG_28-3.pdf

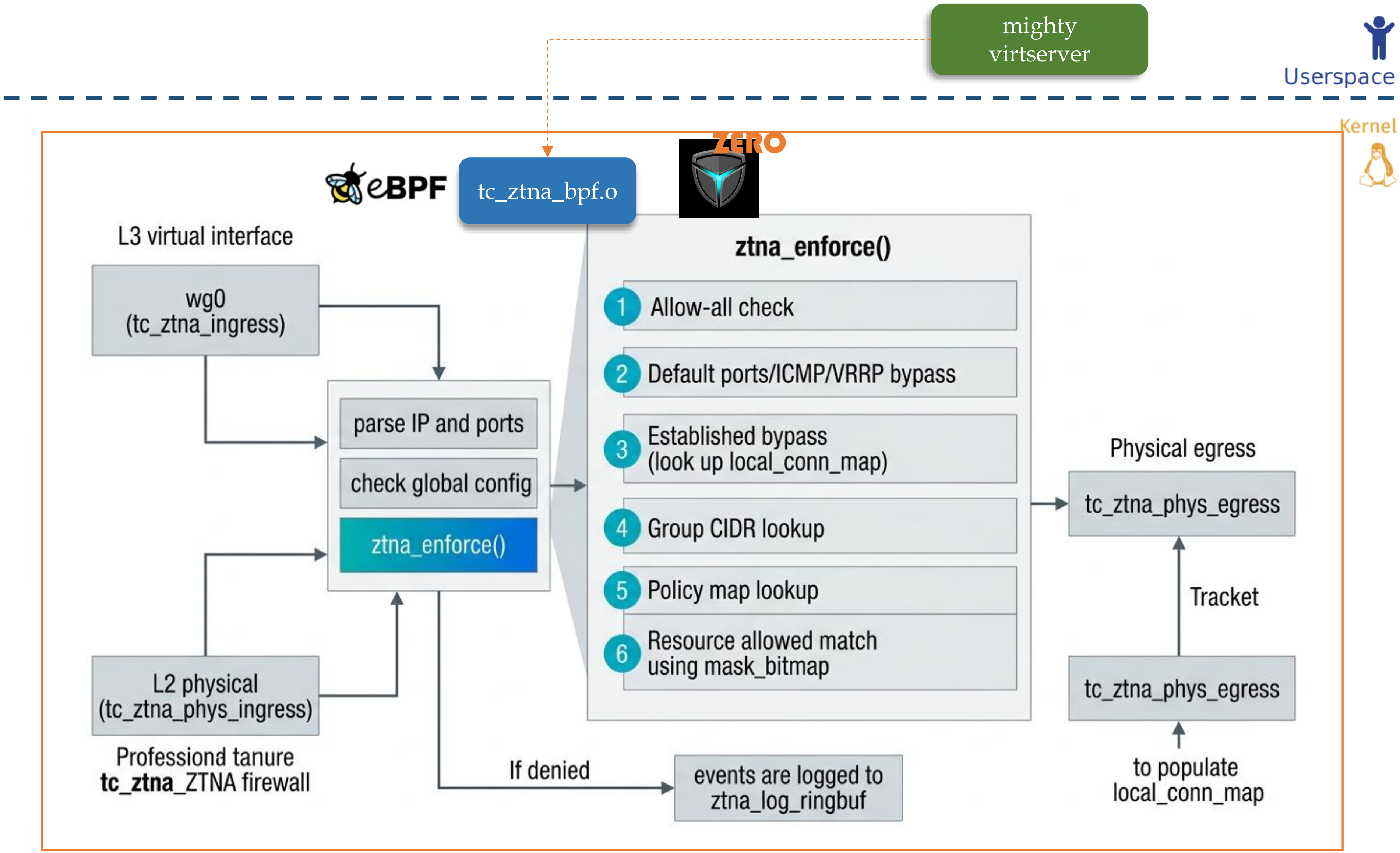
# 3. ZTNA(3) – eBPF ZTNA Enforcement(2)



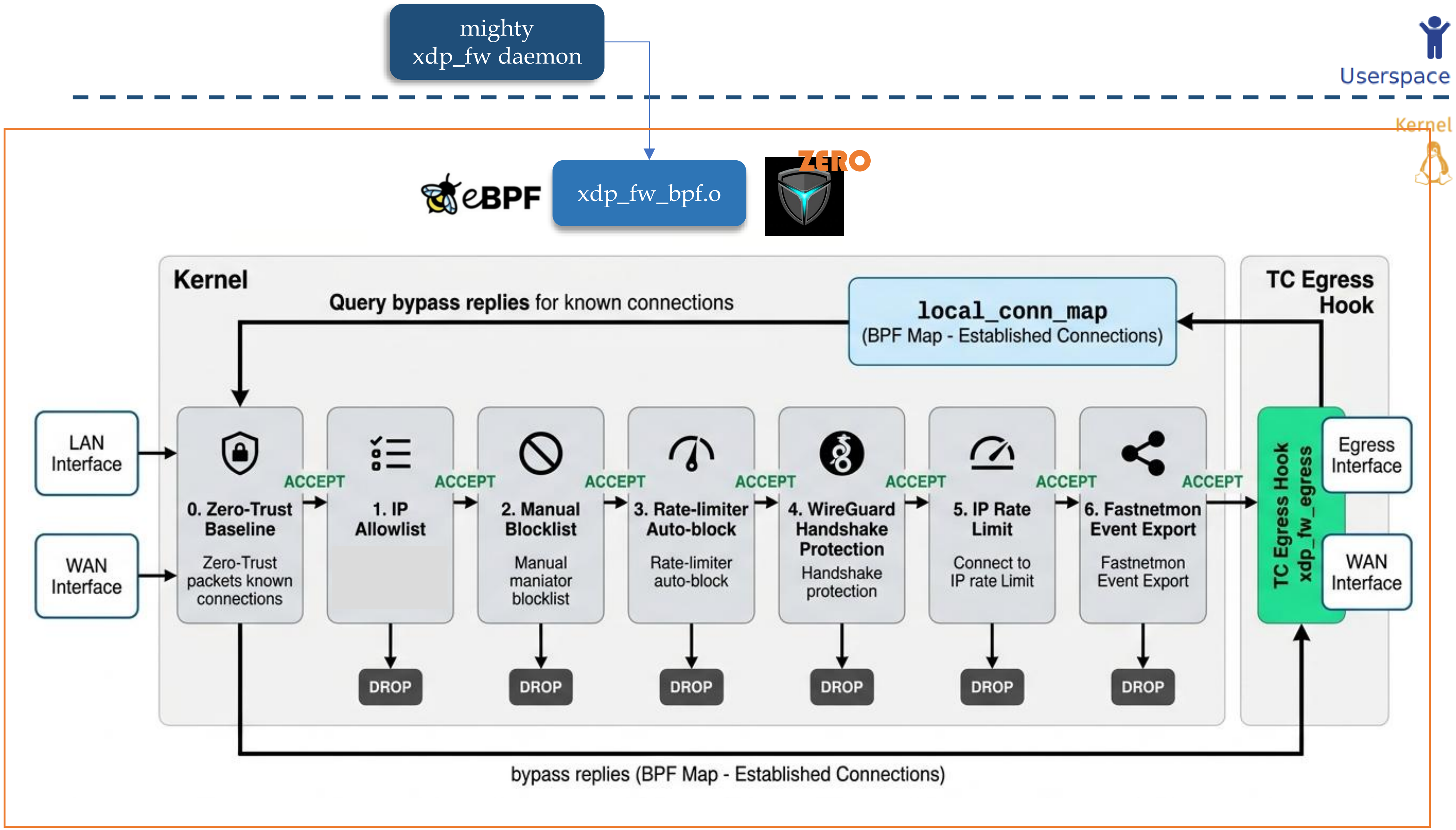




[출처] https://media.frnog.org/FRnOG_28/FRnOG_28-3.pdf

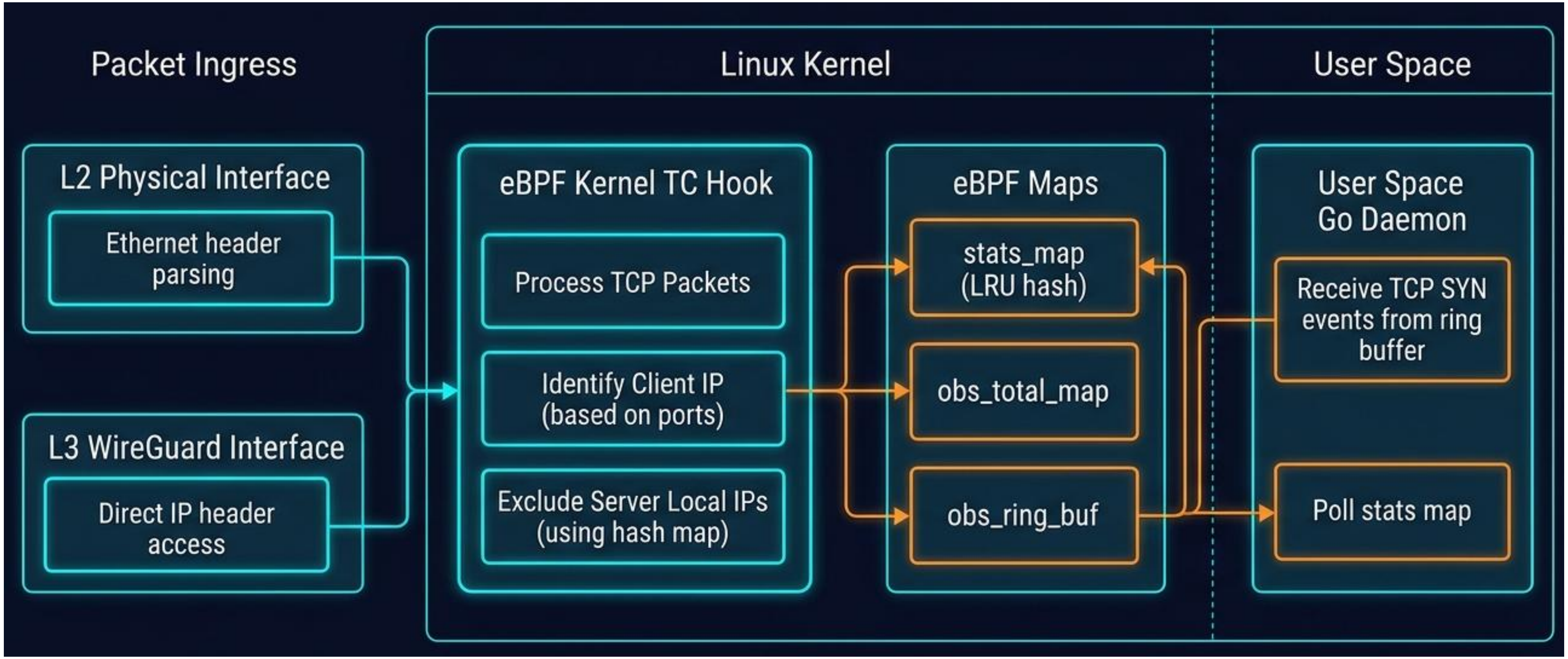
# 3. ZTNA(3) – eBPF ZTNA Enforcement(3)



# 3. ZTNA(3) – eBPF ZTNA Enforcement(4)



# 3. ZTNA(4) – Client ID별 Session Monitoring



# 3. ZTNA(5) – Client ID별 Model 제한


The image above was captured from agentgateway.dev.

# 3. ZTNA(6) – Client ID별 Service 제한

# 4. AI Application Proxy

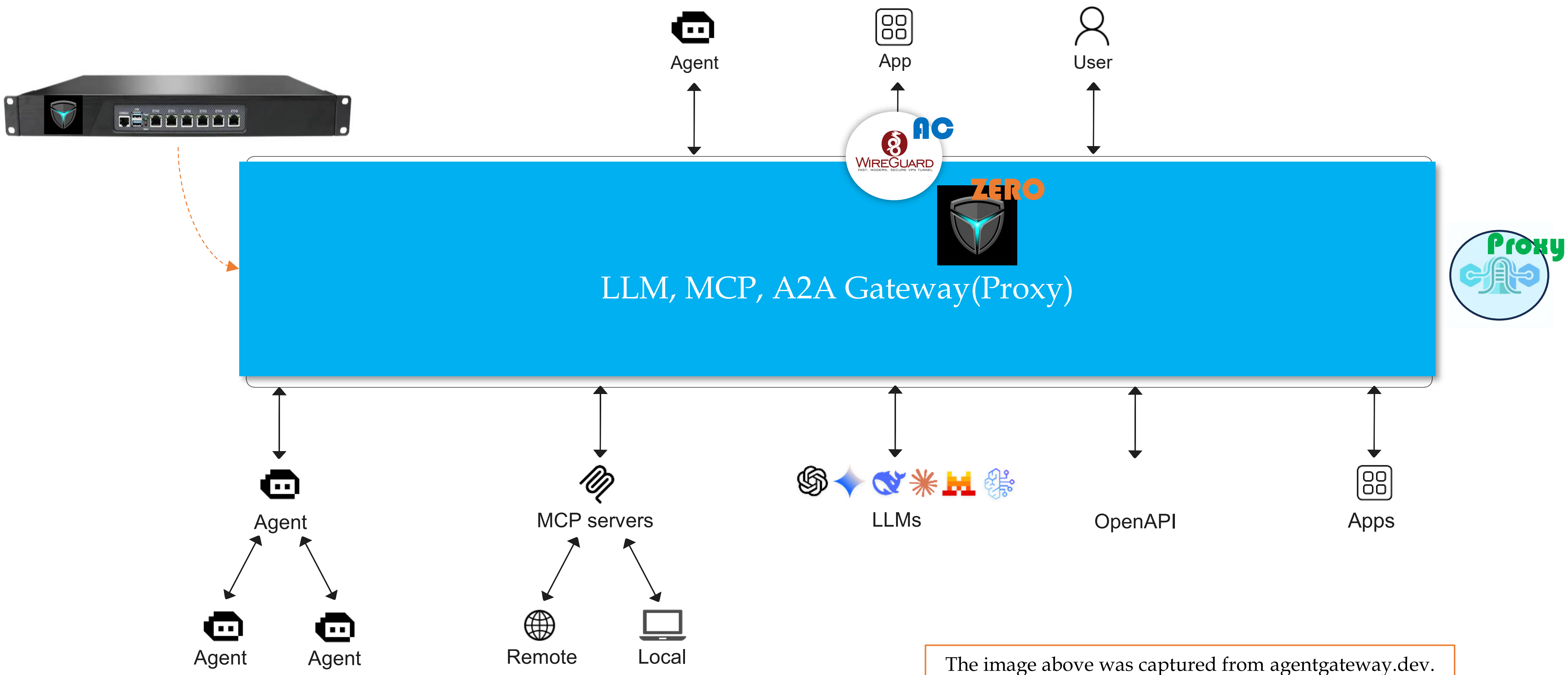
## AI Gateway – LLM/MCP/A2A

Proxy

Powered by agentgateway

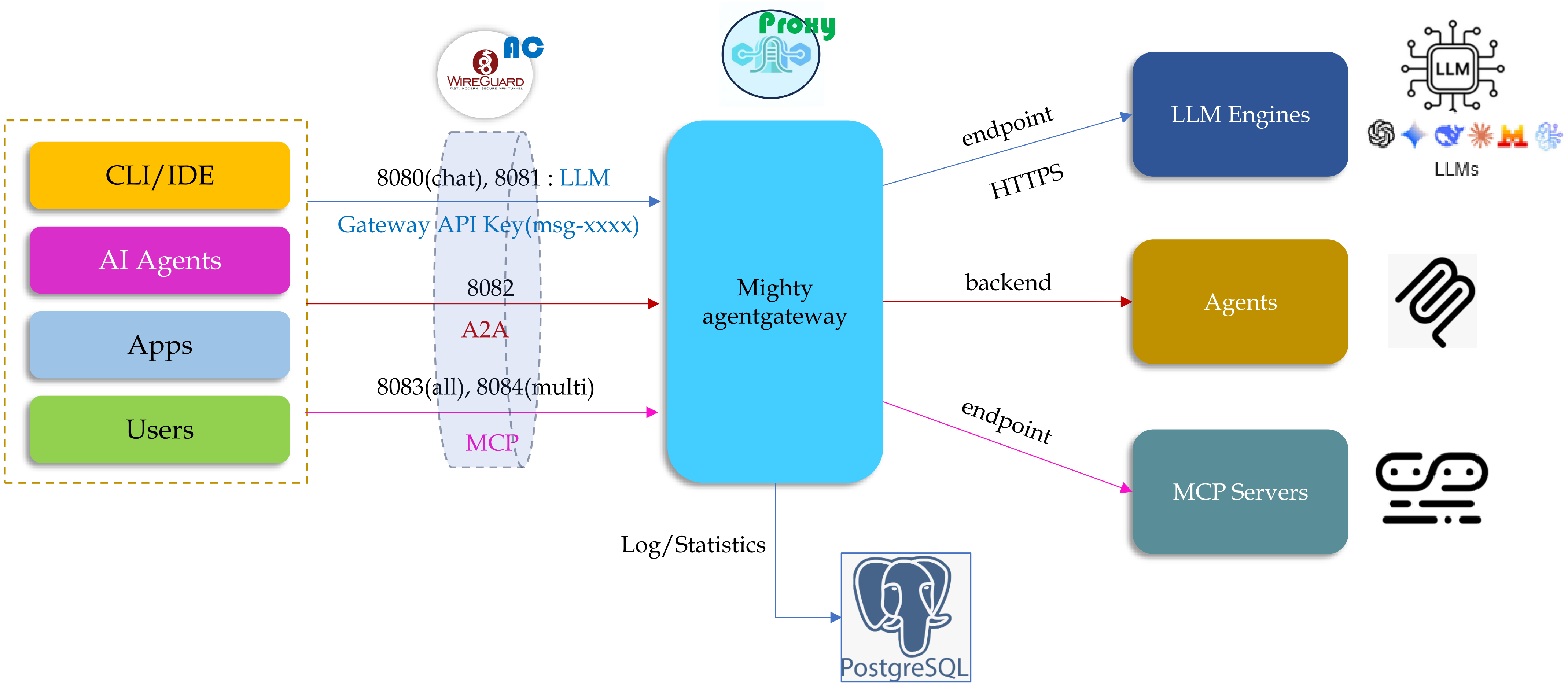
The logo image above was captured from agentgateway.dev.

# 4. AI Application Proxy(1) – Overview(1)

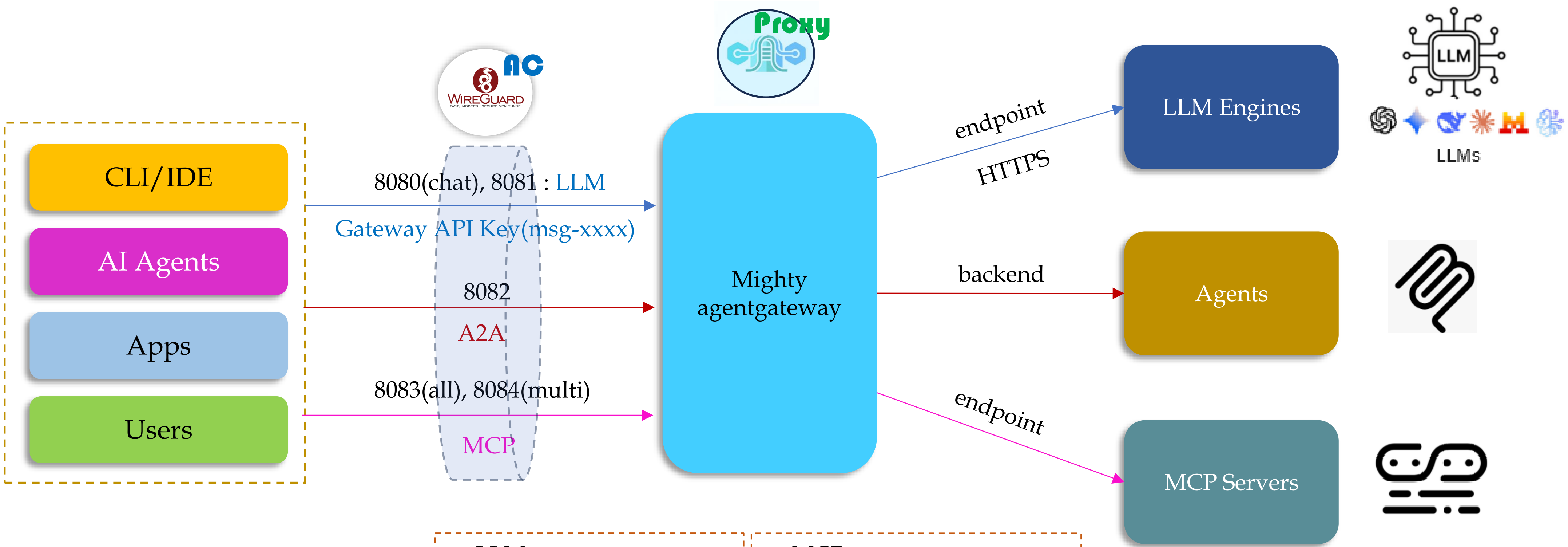



The image above was captured from agentgateway.dev.

# 4. AI Application Proxy(1) – Overview(2)



# 4. AI Application Proxy(1) – Overview(3)



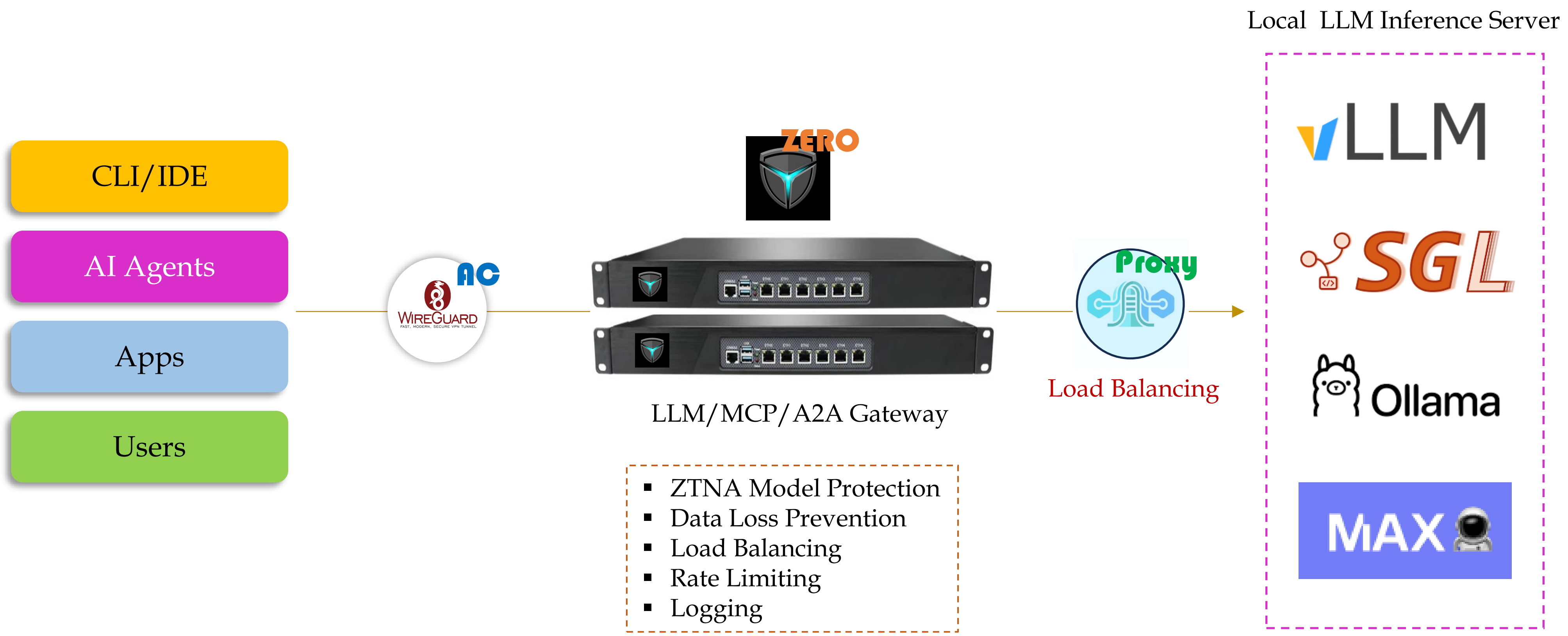

- LLM
  - ✓ ZTNA Model Protection
  - ✓ Data Loss Prevention
  - ✓ Load Balancing
  - ✓ Rate Limiting
  - ✓ Logging

- MCP
  - ✓ Authentication
  - ✓ Access Control(CEL rules)
  - ✓ Rate Limiting
  - ✓ Logging

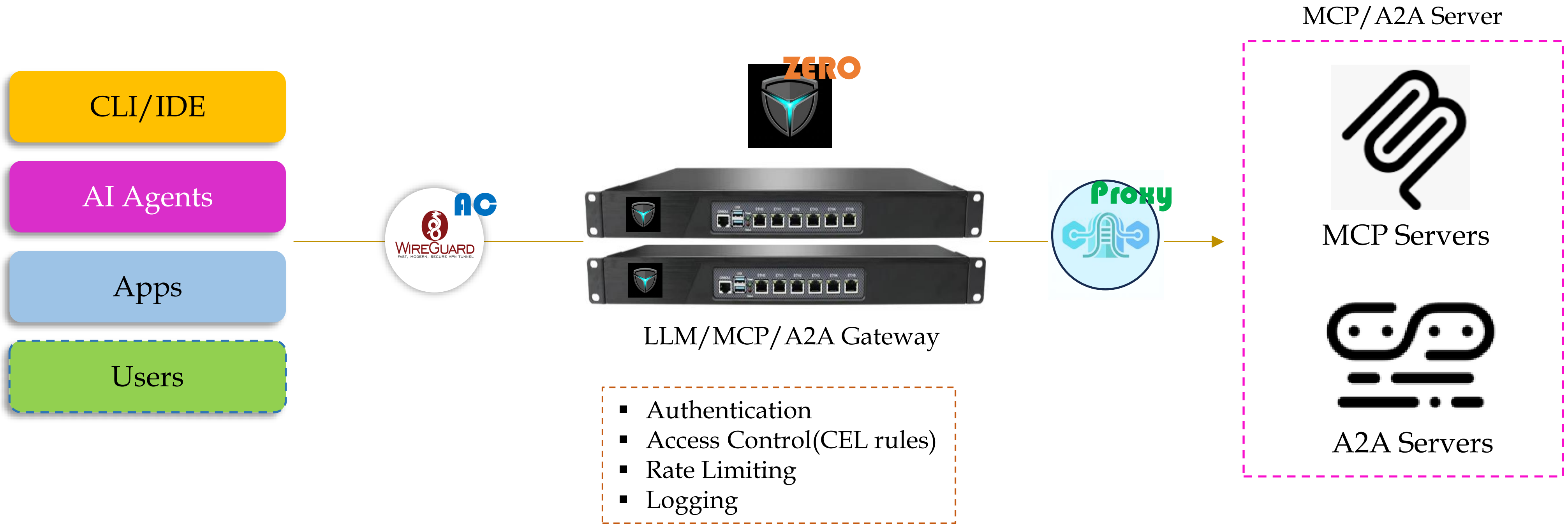
# 4. AI Application Proxy(2) – LLM Gateway(1)


The image above was captured from agentgateway.dev.

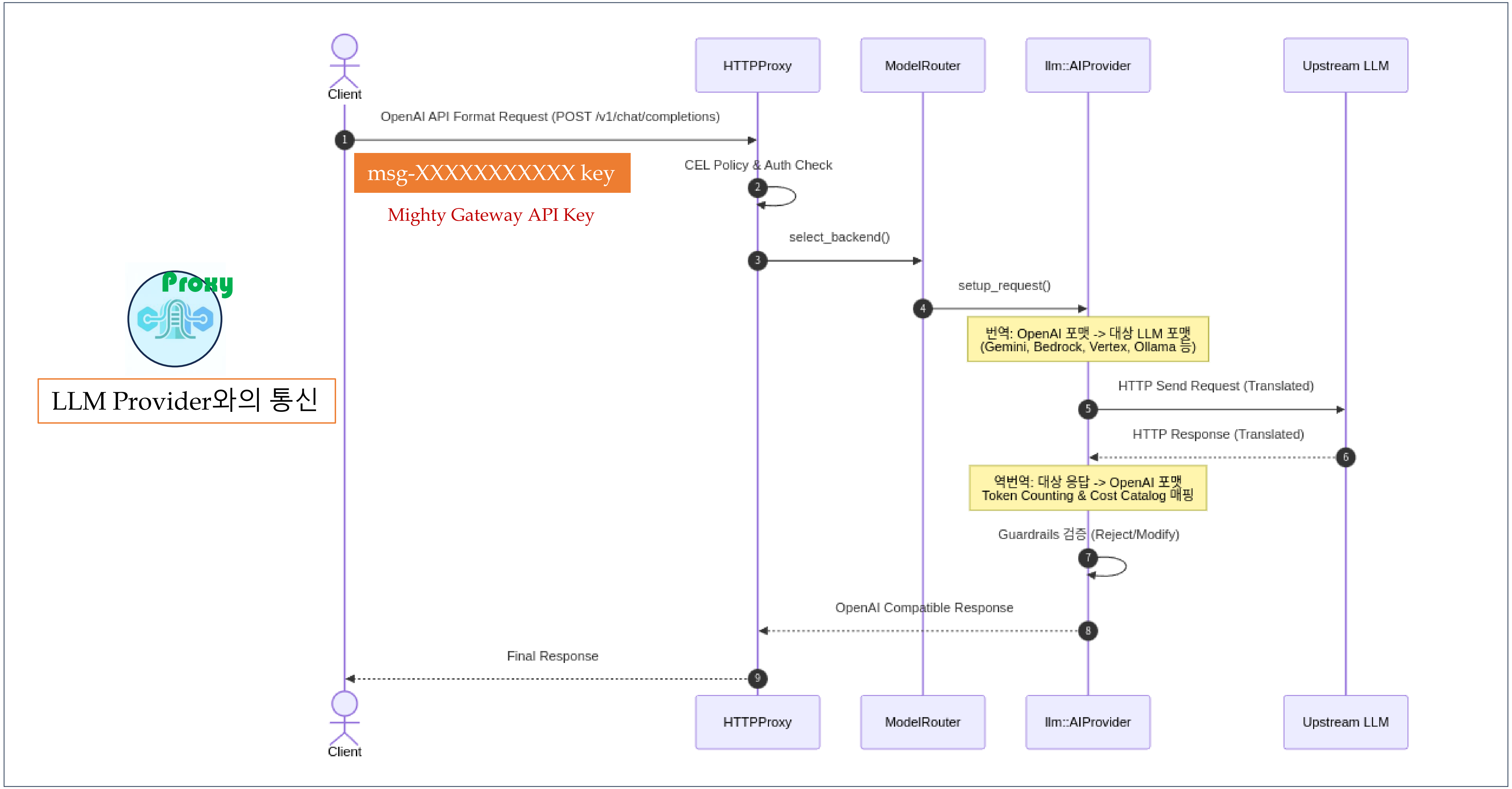
# 4. AI Application Proxy(2) - LLM Gateway(2)



# 4. AI Application Proxy(3) – MCP/A2A Gateway



# 4. AI Application Proxy(4) – Architecture



# 5. Mighty AI Security Platform

# 5. Mighty AI Security Platform(1)


Mighty AI Security Gateway


Mighty AI Mini Gateway

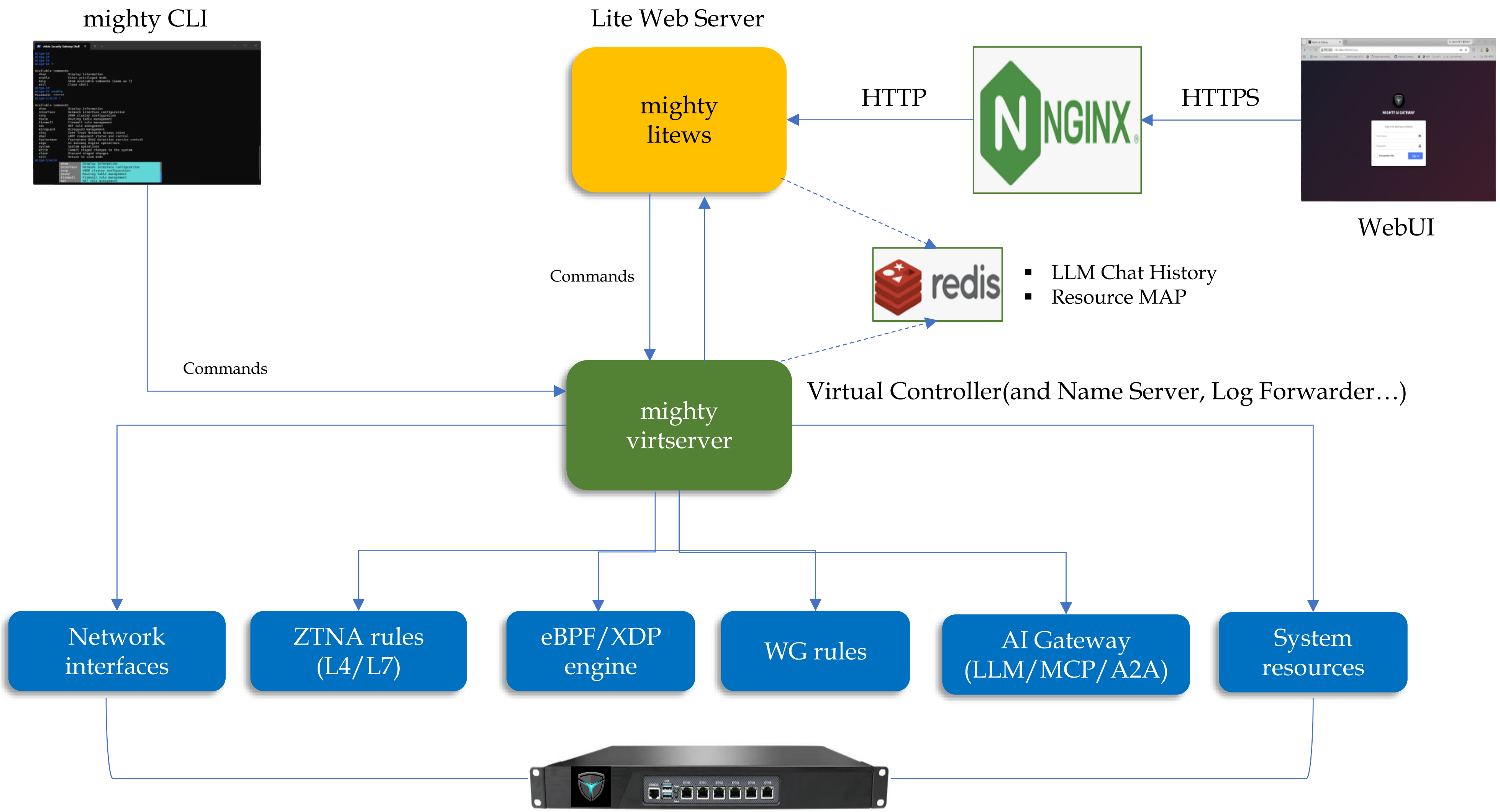

Mighty AI Windows Agent


Mighty AI Ubuntu Agent

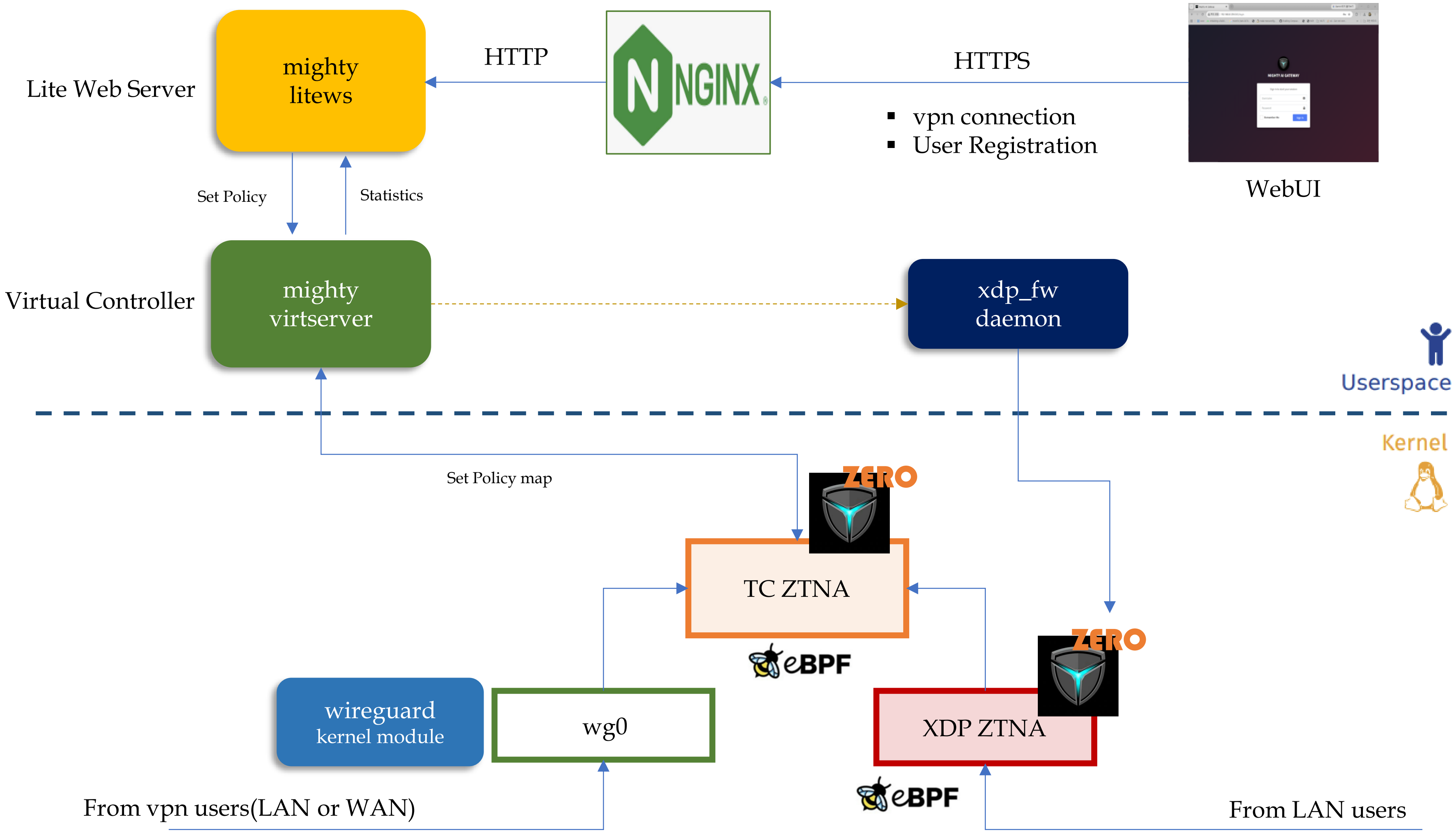

Mighty AI macOS/iOS Agent


Mighty AI Android Agent

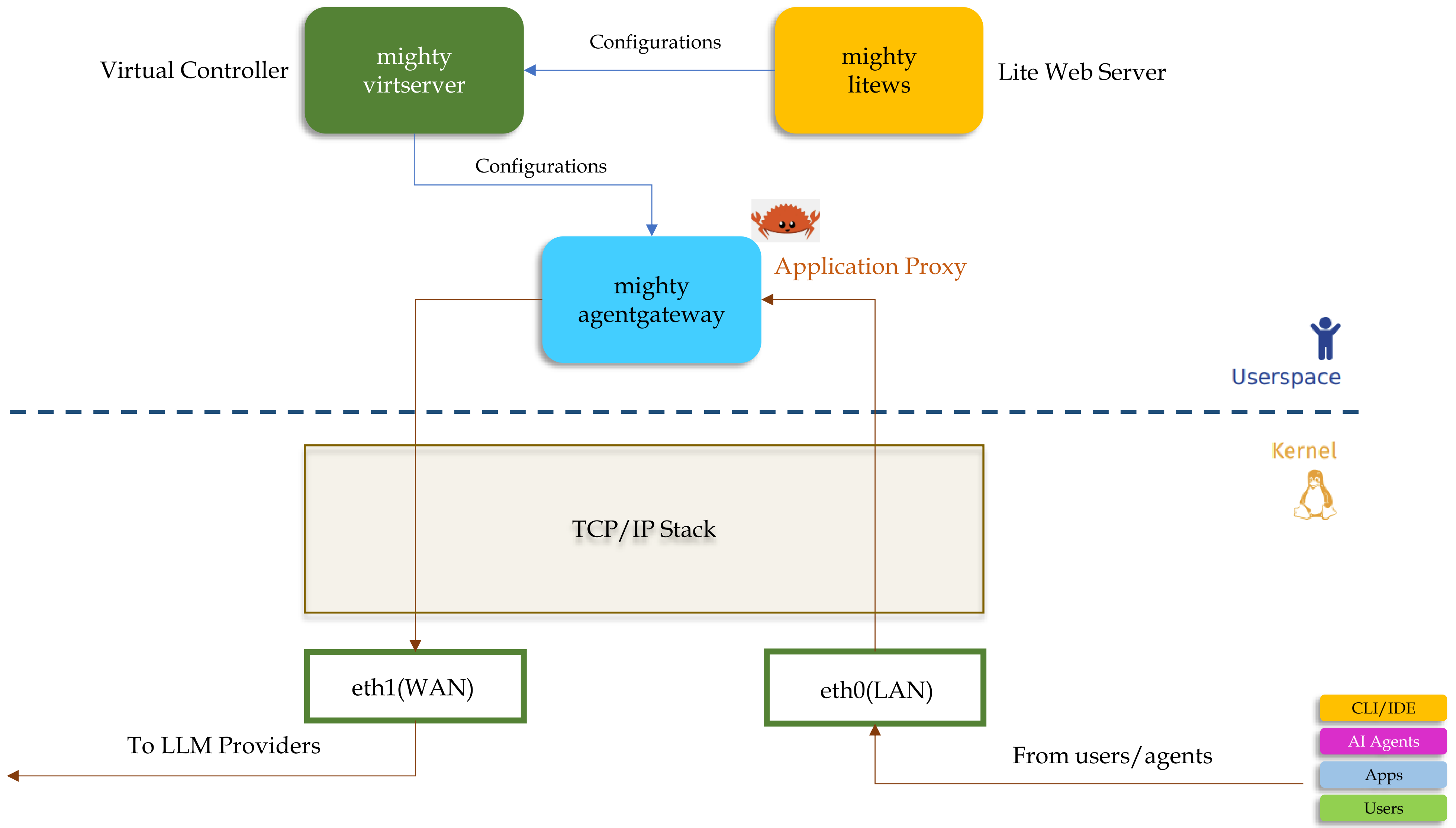
# 5. Mighty AI Security Platform(2) – Mighty Engine(1)



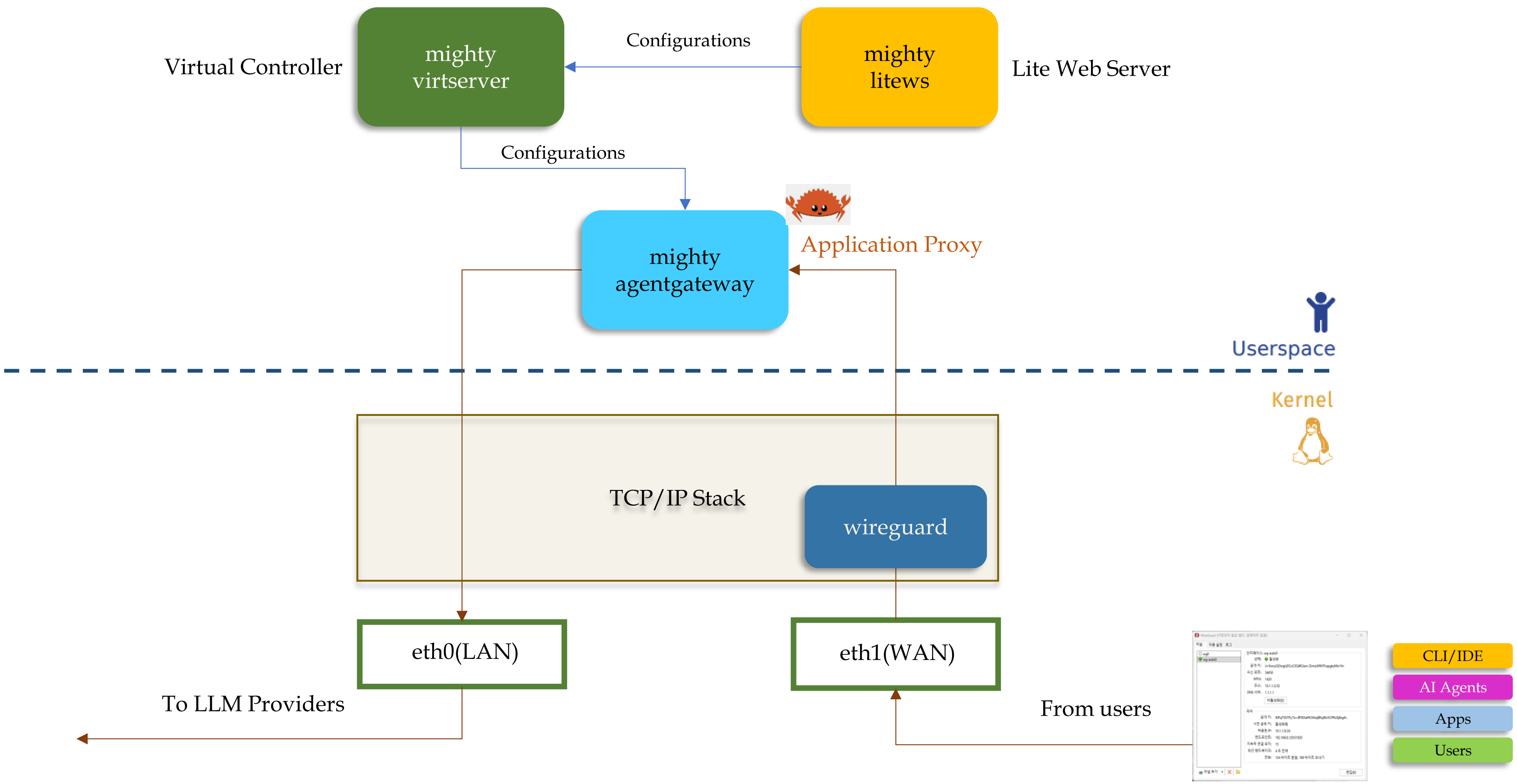
# 5. Mighty AI Security Platform(2) – Mighty Engine(2)



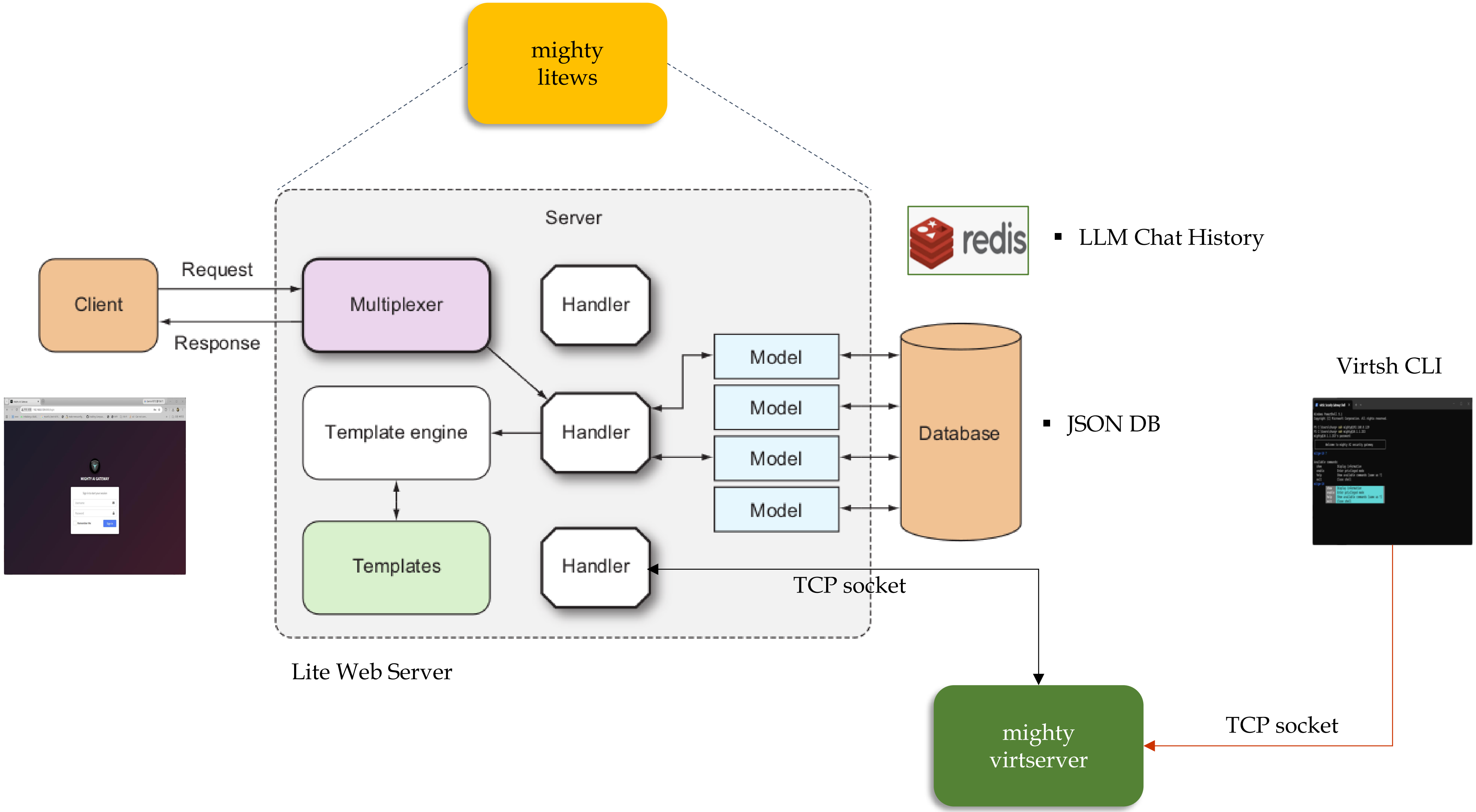
# 5. Mighty AI Security Platform(2) – Mighty Engine(3)



# 5. Mighty AI Security Platform(2) – Mighty Engine(4)

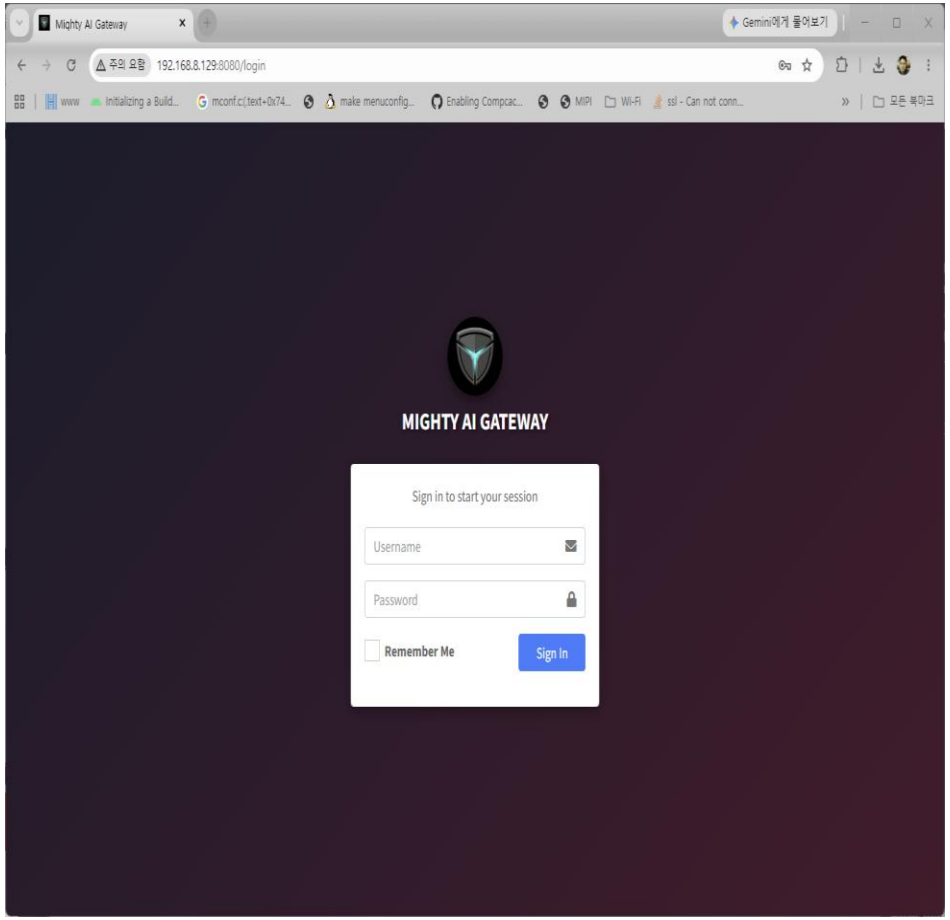


# 5. Mighty AI Security Platform(2) – Mighty Engine(5)

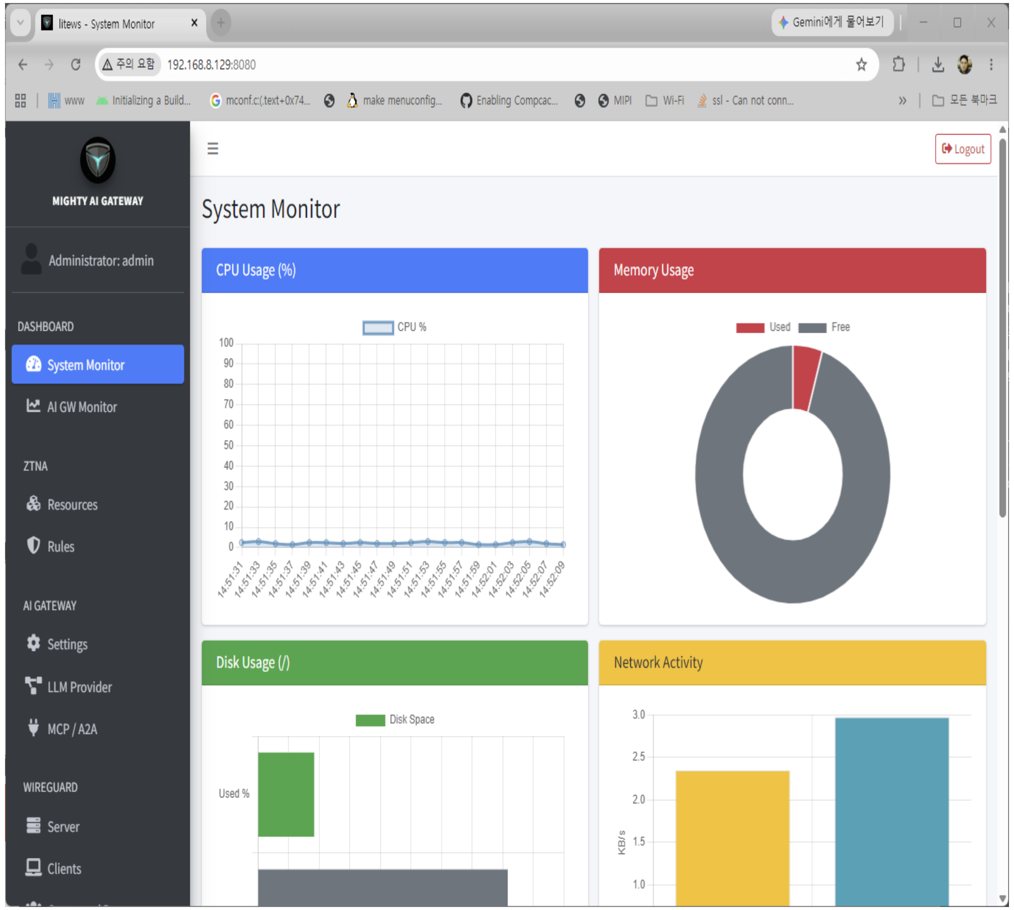


# 5. Mighty AI Security Platform(3) – WebUI(1)

- WebUI Login Page

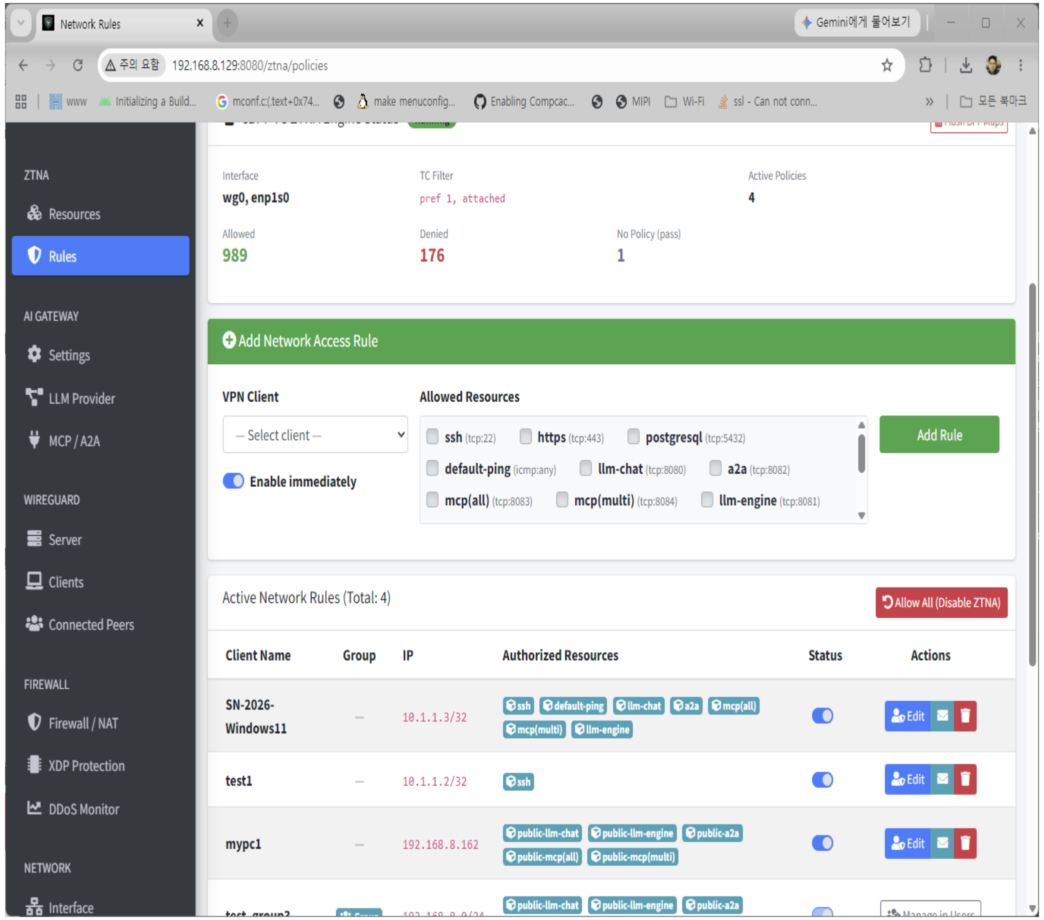



- Dashboard Page

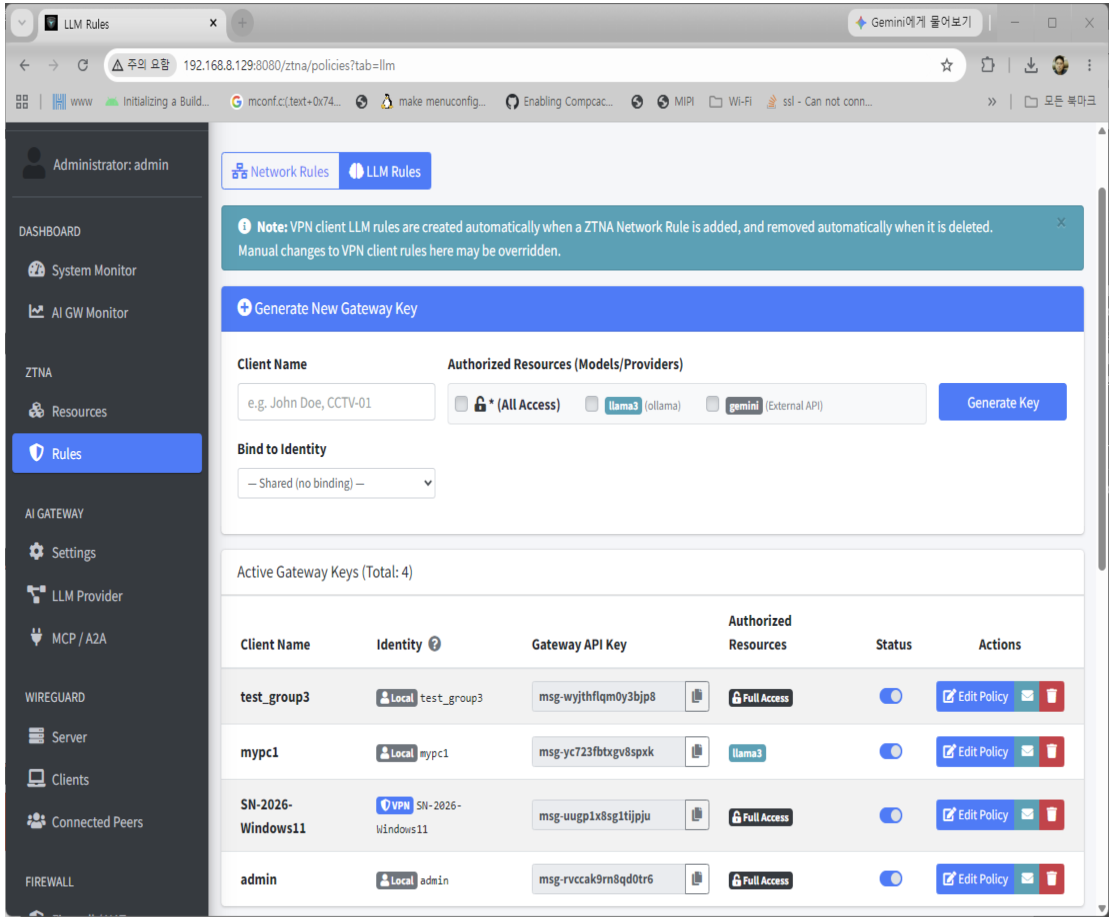


# 5. Mighty AI Security Platform(3) – WebUI(2)

- ZTNA Network Rules

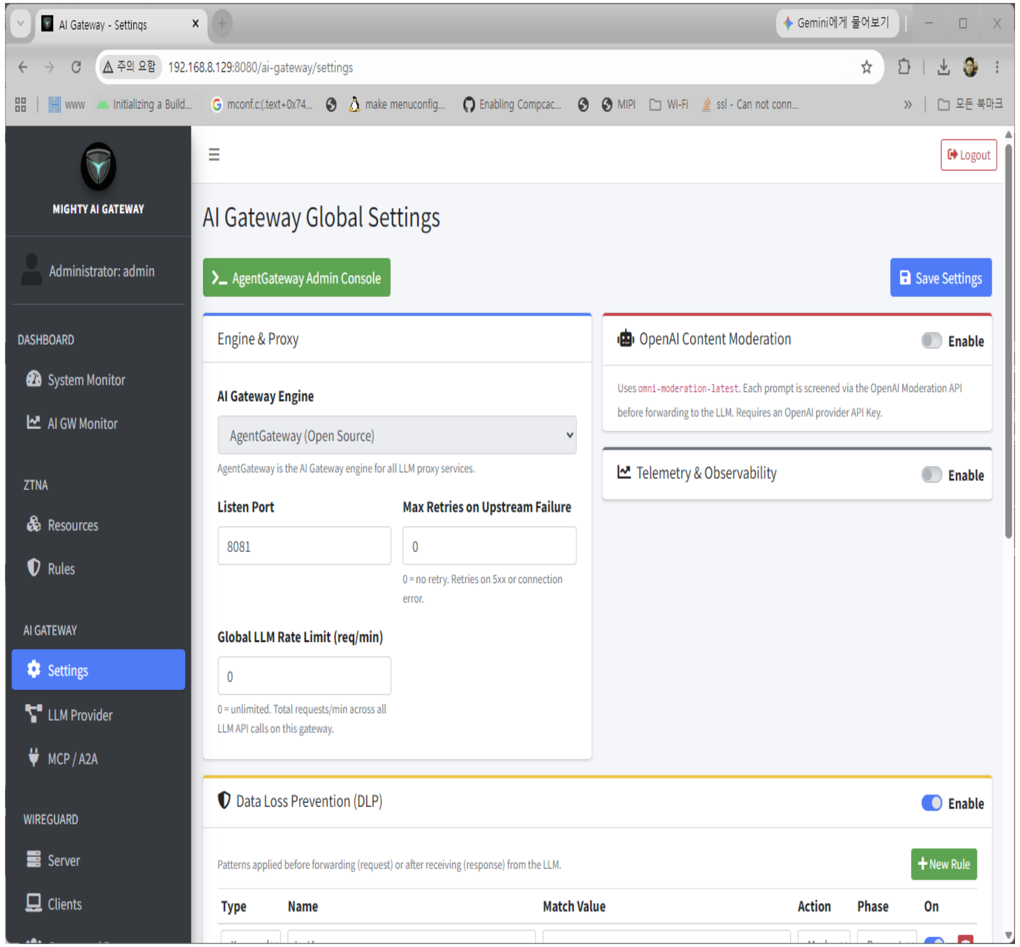



- ZTNA LLM Rules

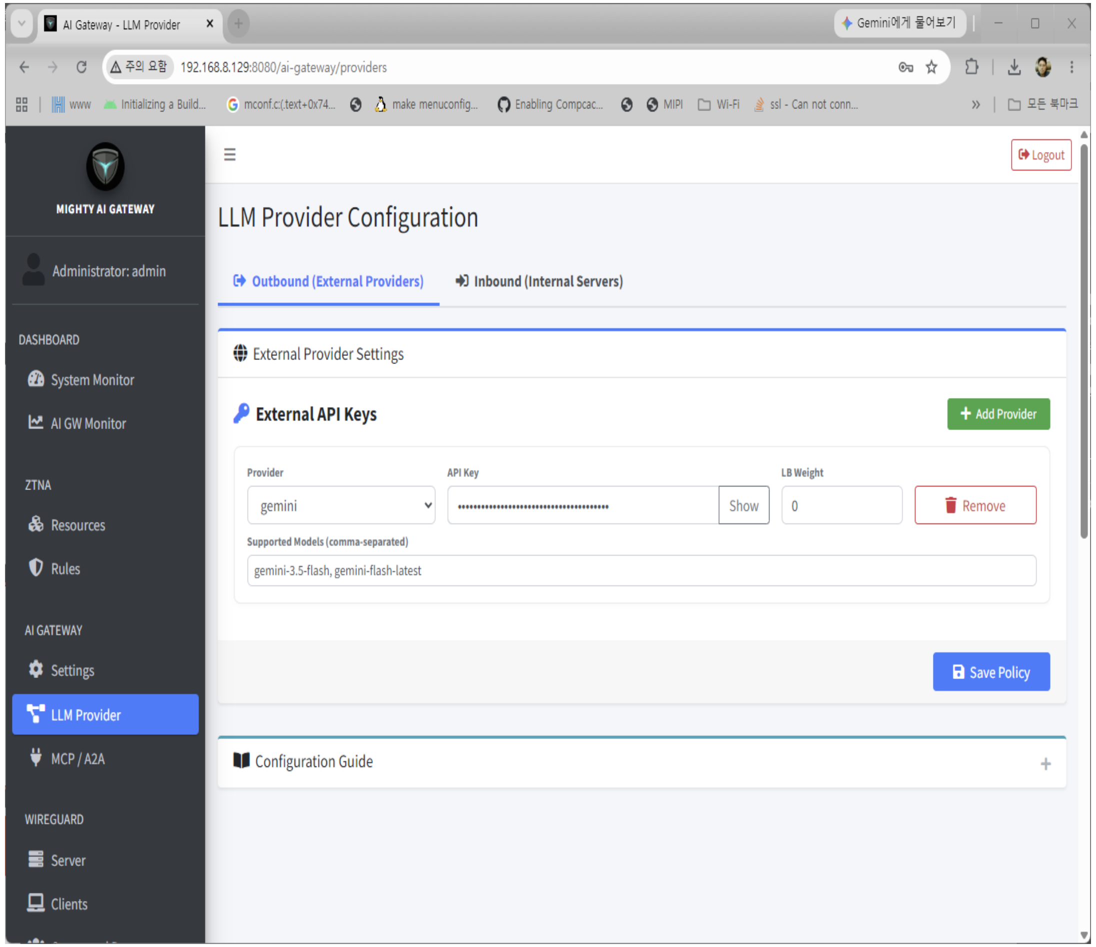


# 5. Mighty AI Security Platform(3) – WebUI(3)

- AI Gateway Settings

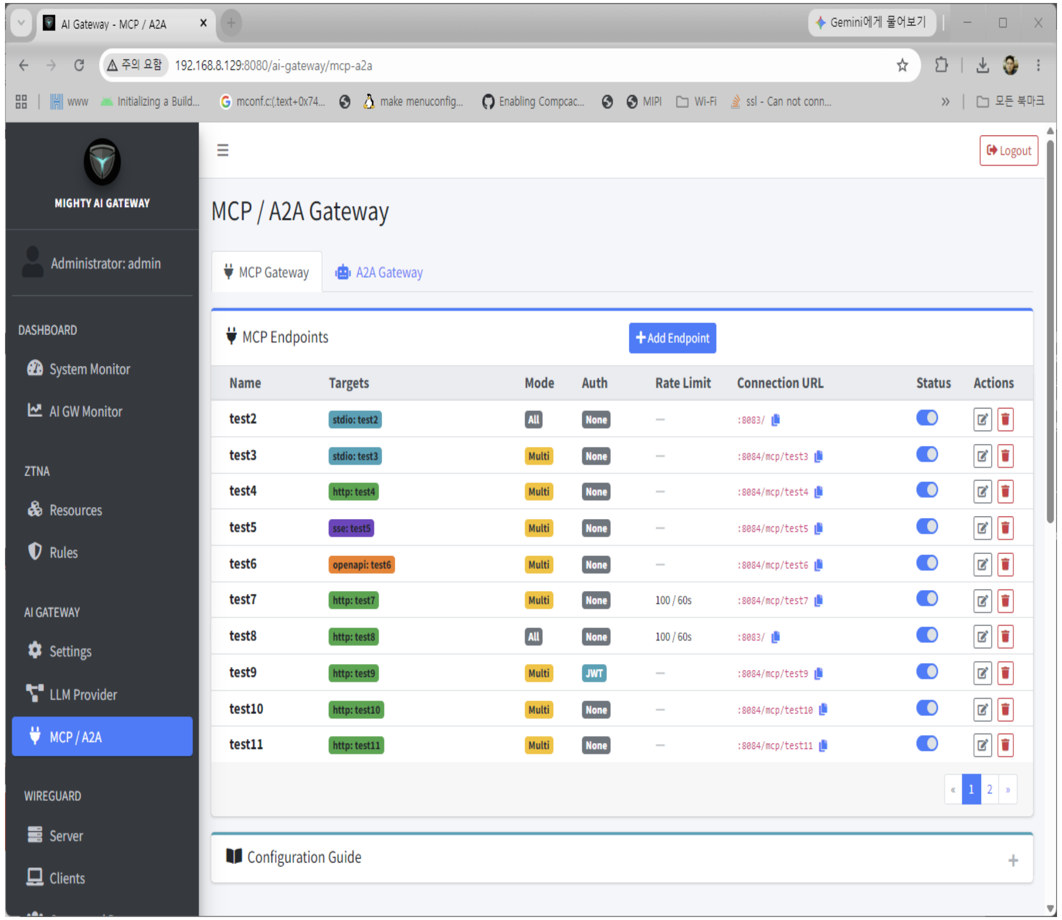



- LLM Provider Configuration

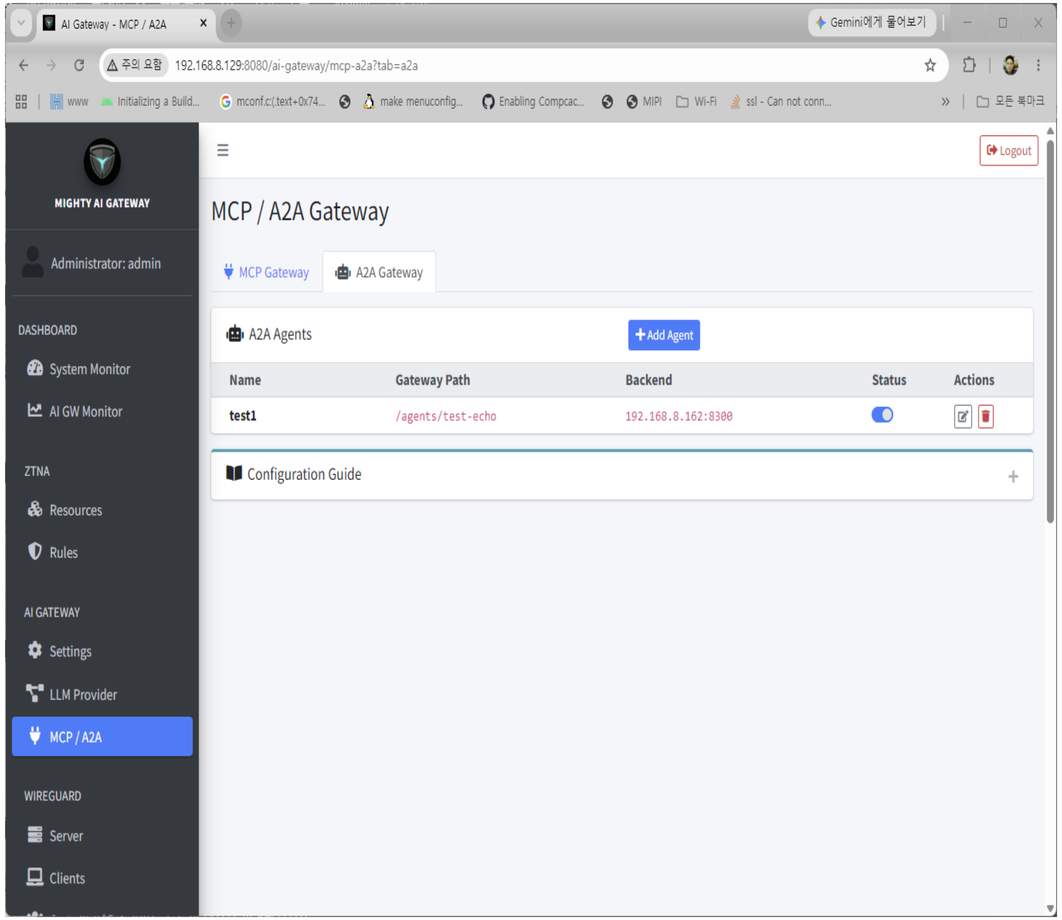


# 5. Mighty AI Security Platform(3) – WebUI(4)

- MCP Gateway Configuration

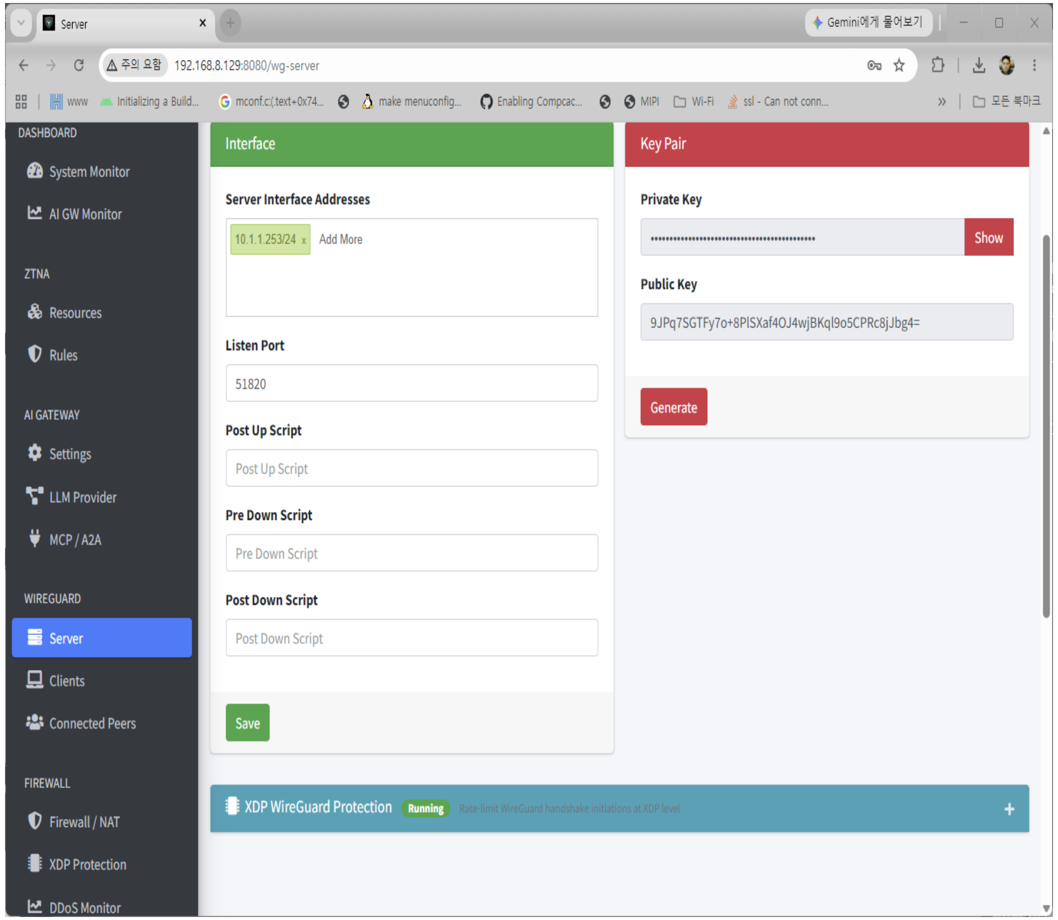



- A2A Gateway Configuration

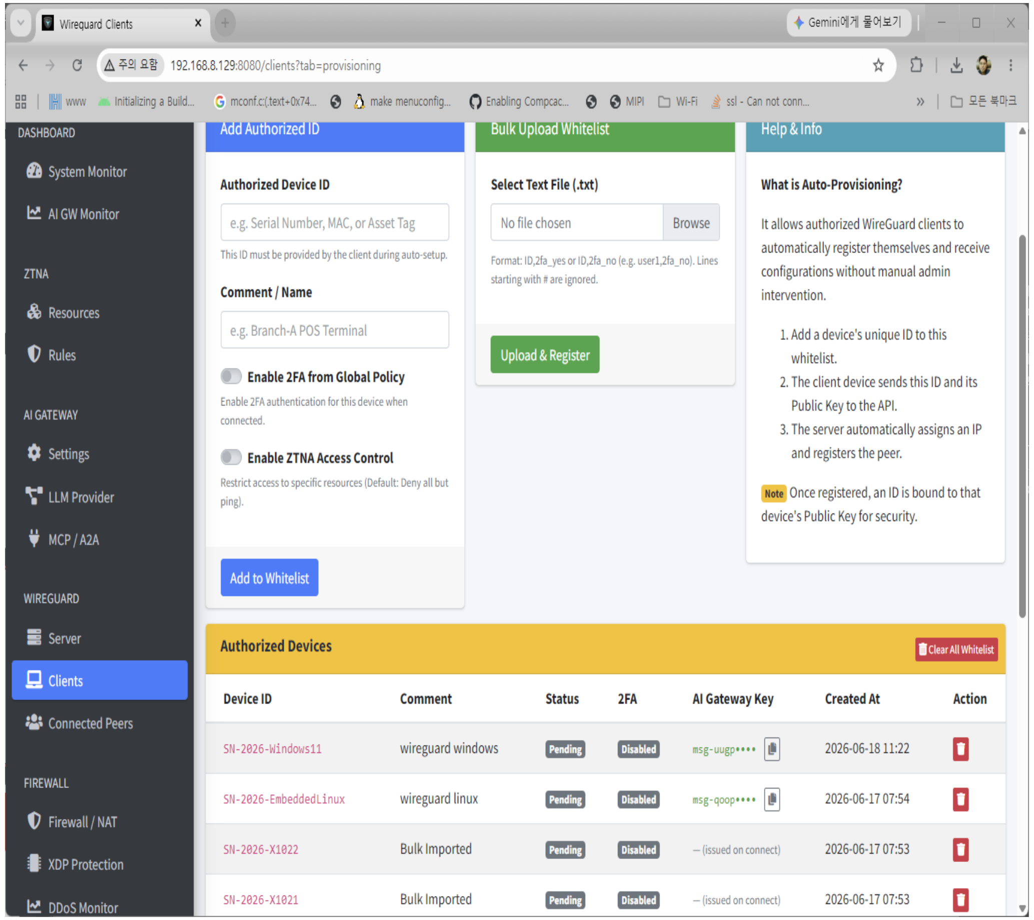


# 5. Mighty AI Security Platform(3) – WebUI(5)

- WireGuard Server Configuration

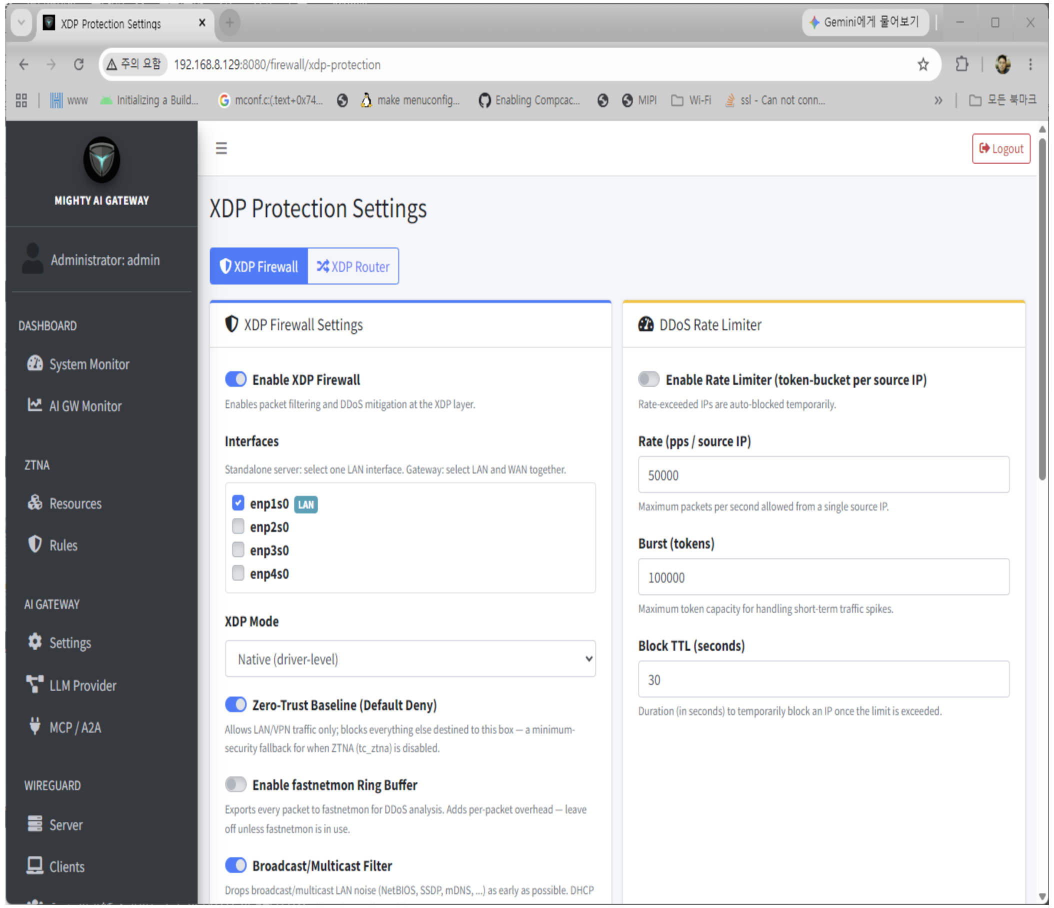



- WireGuard Client Configuration

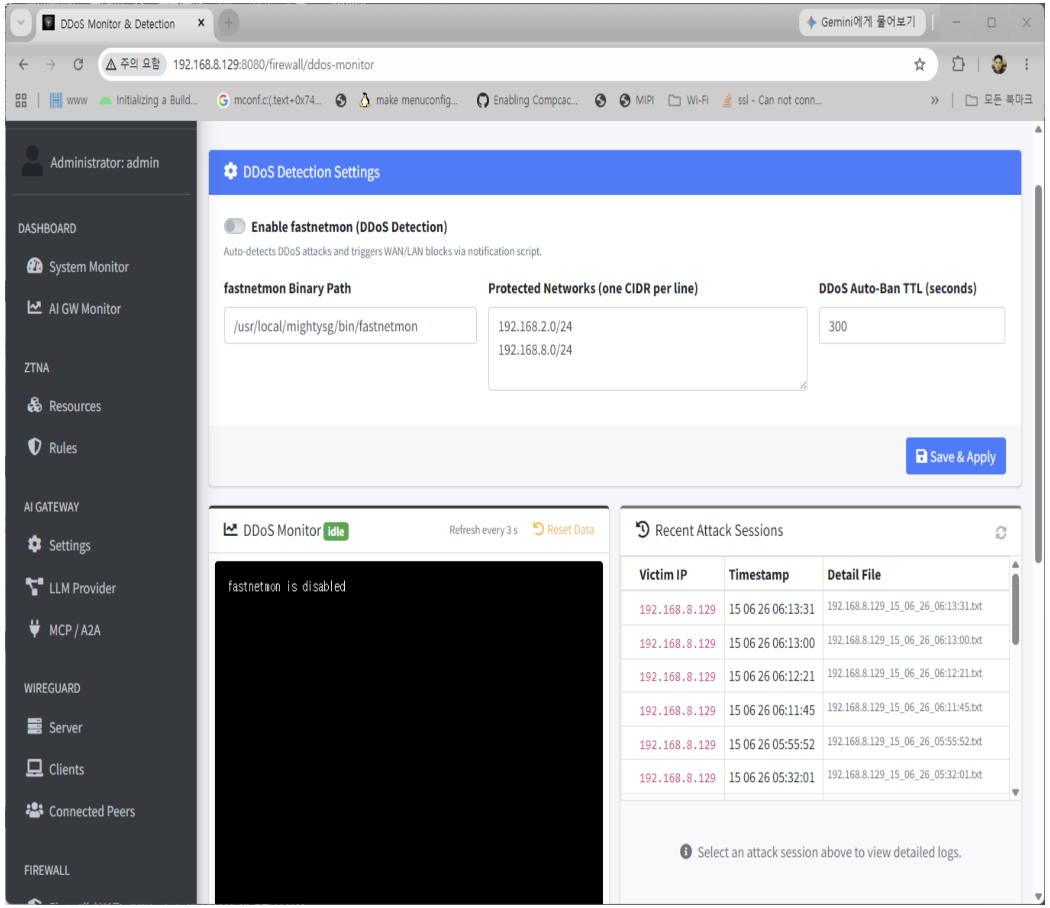


# 5. Mighty AI Security Platform(3) – WebUI(6)

- XDP Protection Configuration

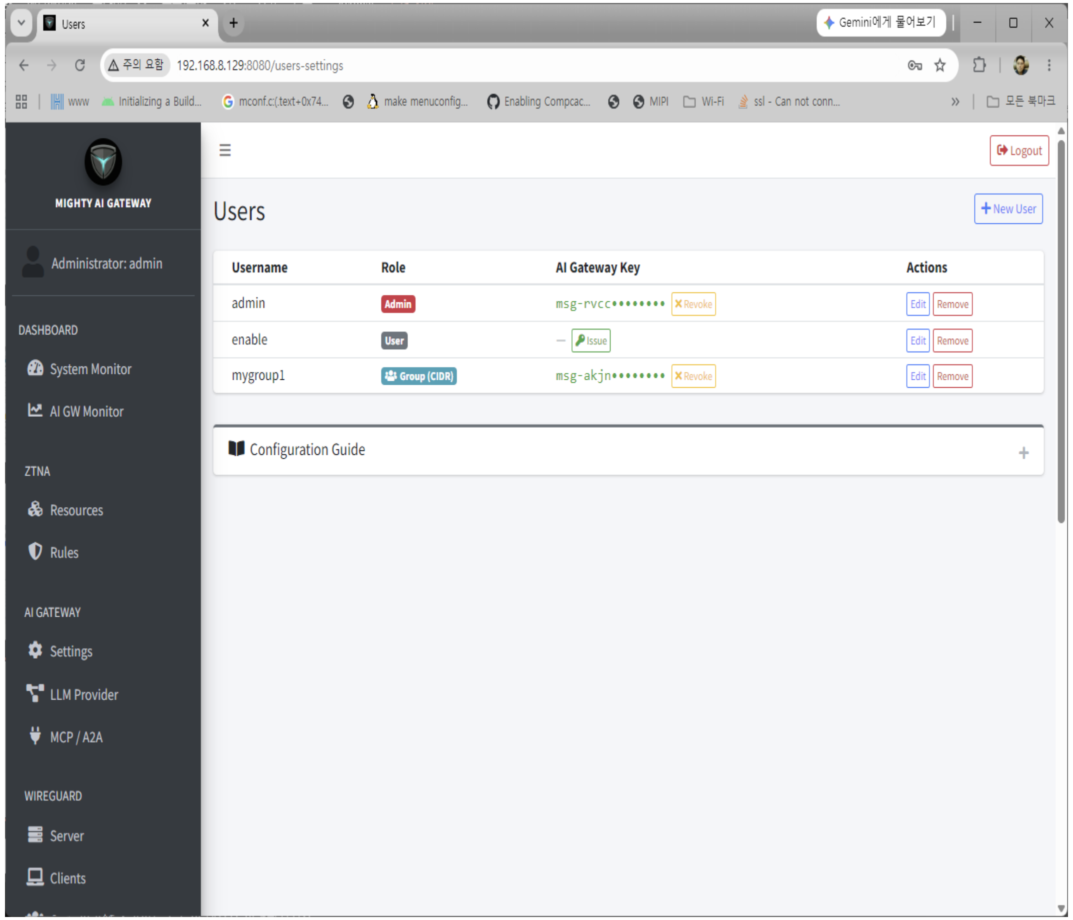



- DDoS Attack Monitor

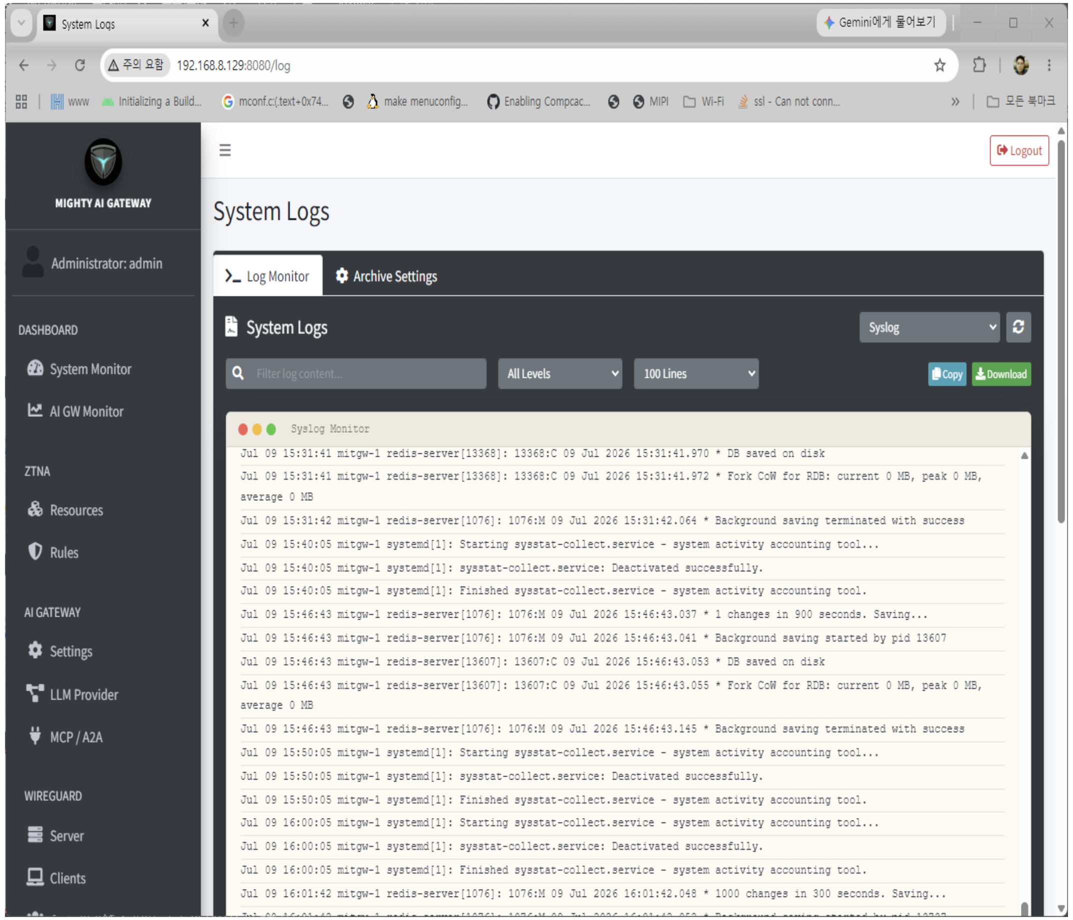


# 5. Mighty AI Security Platform(3) – WebUI(7)

- User/Groups Page

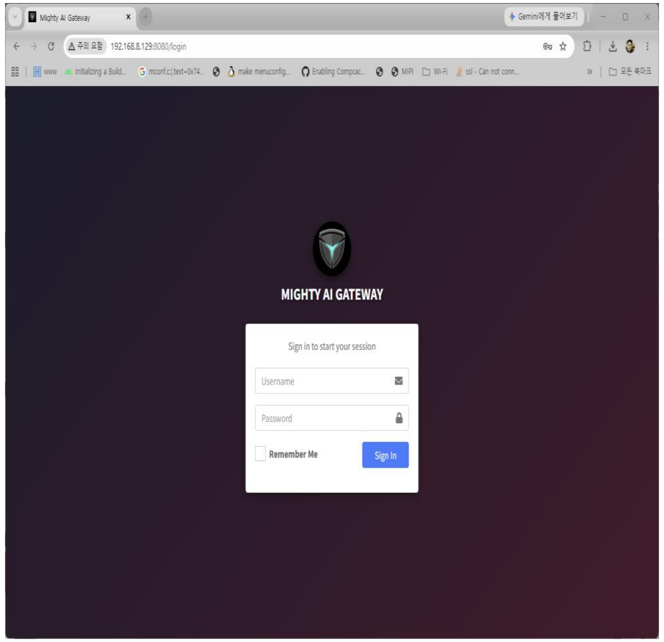



- System Logs Page

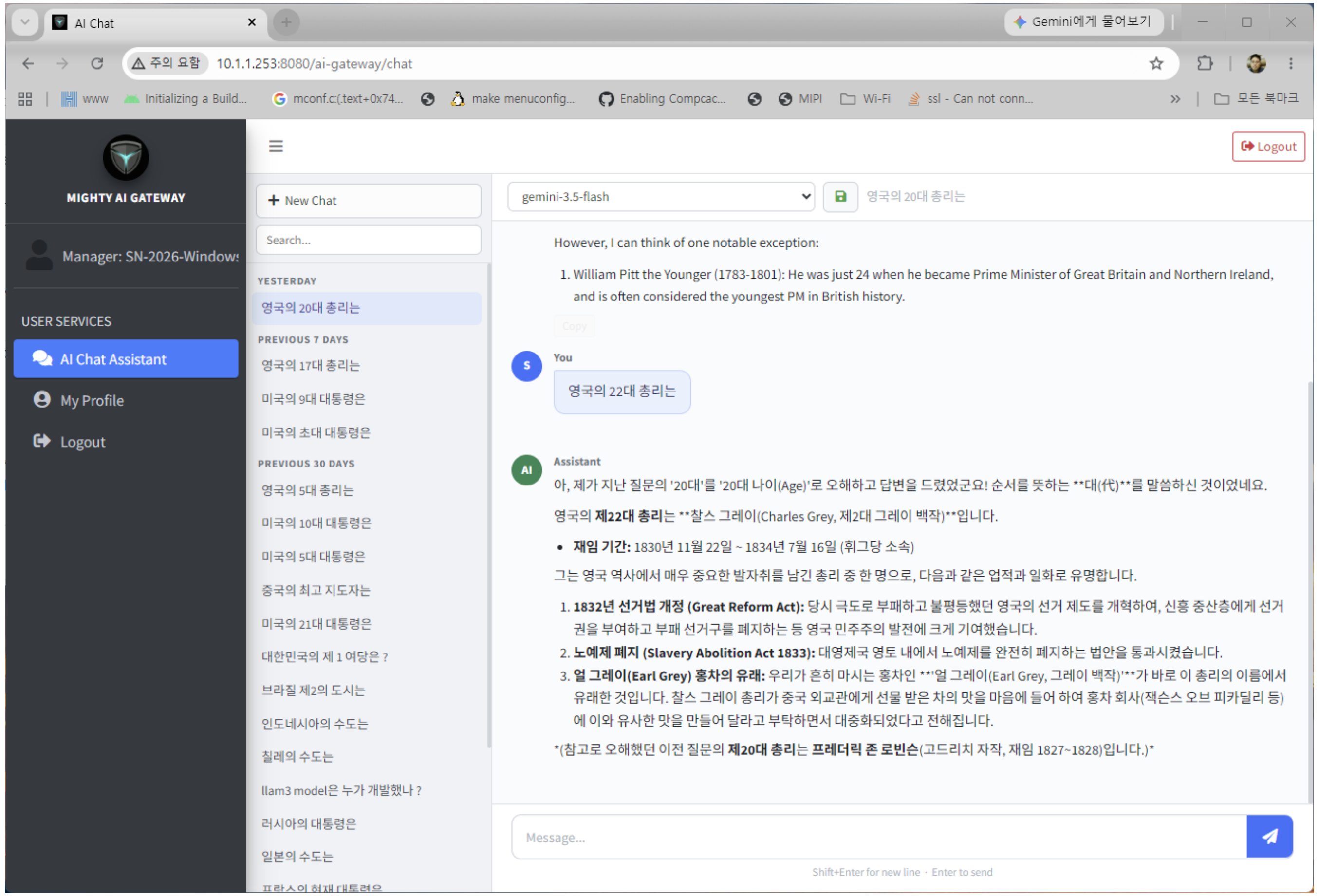


# 5. Mighty AI Security Platform(3) – WebUI(8)

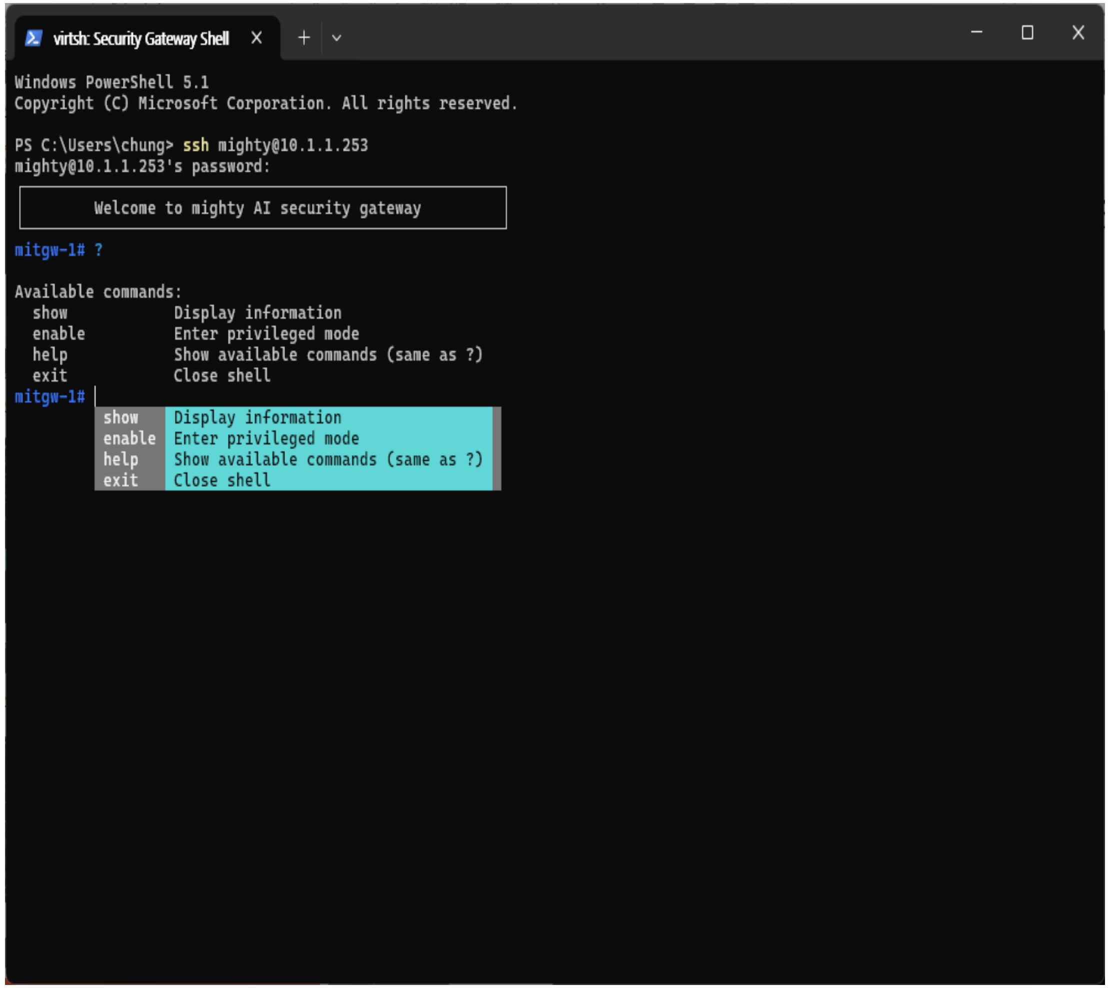
- User(VPN or No VPN) Login Page

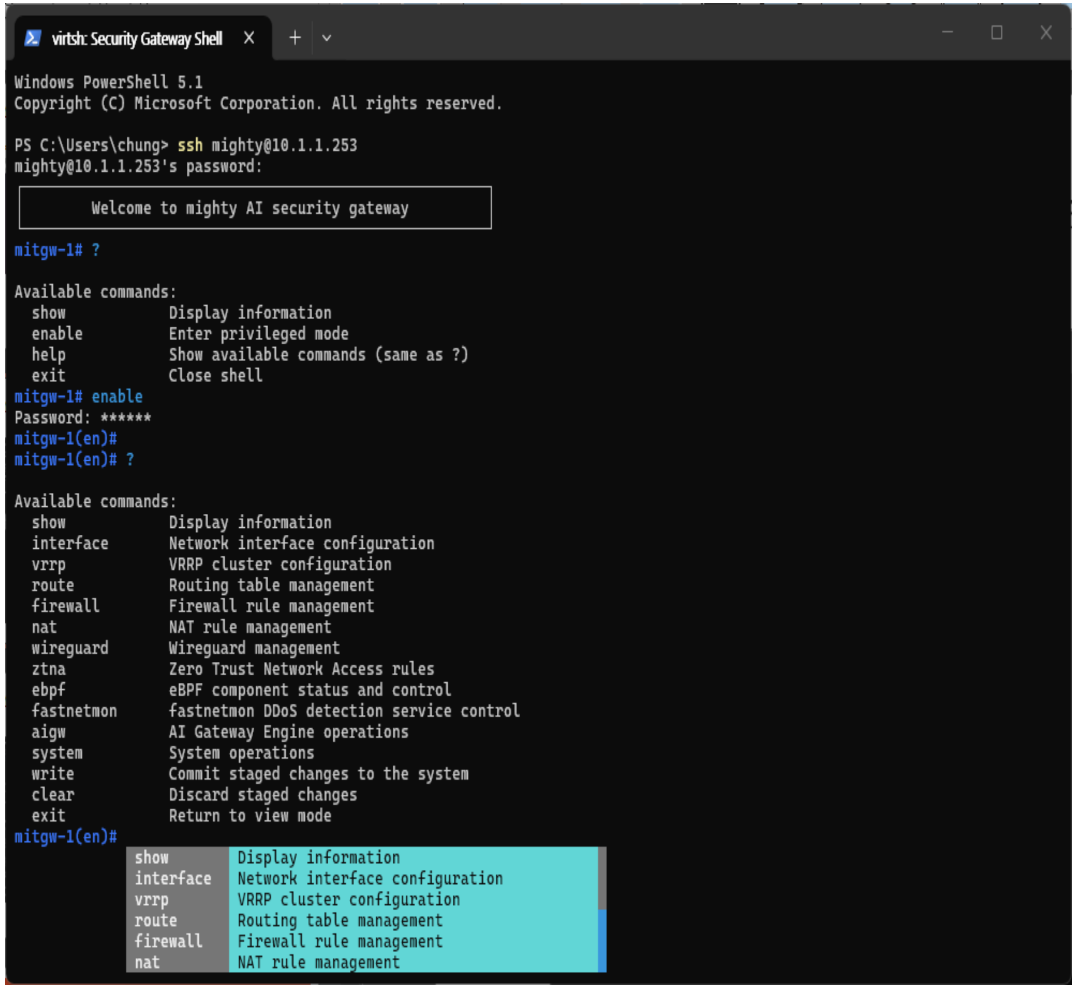



- LLM Chat User Page

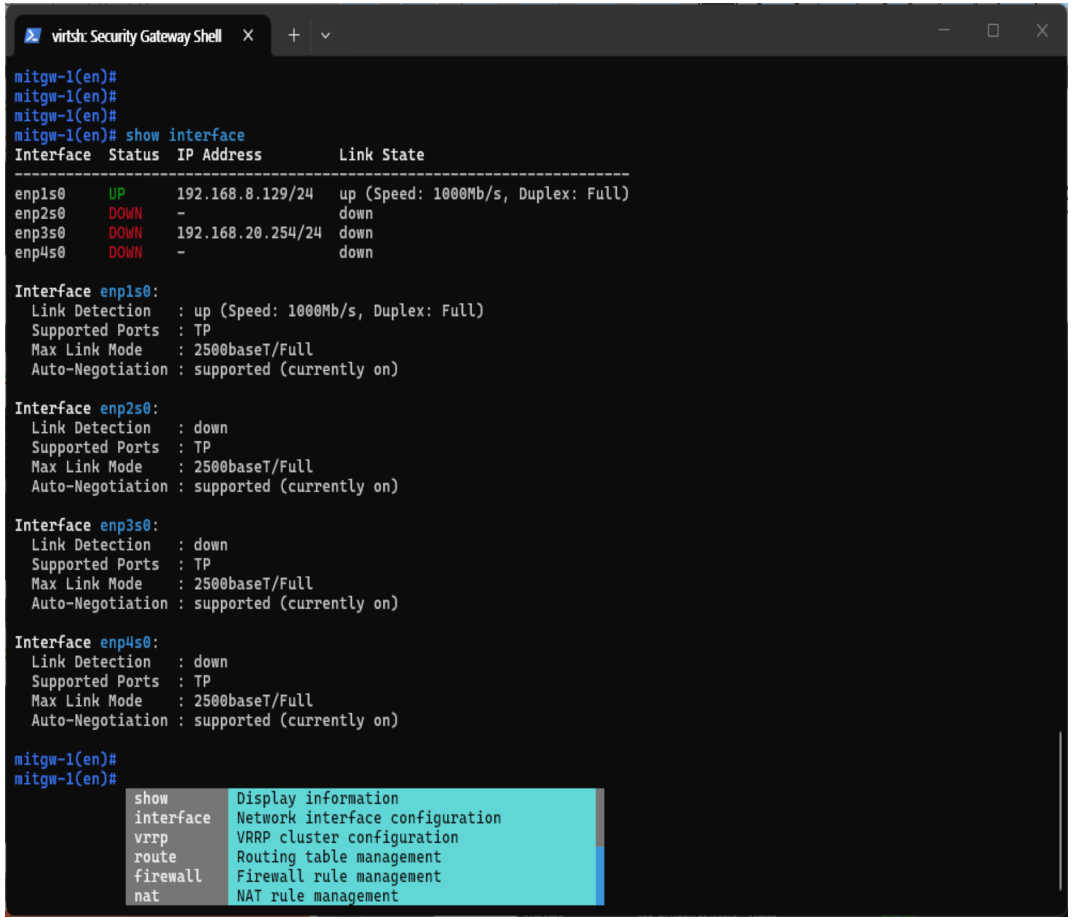


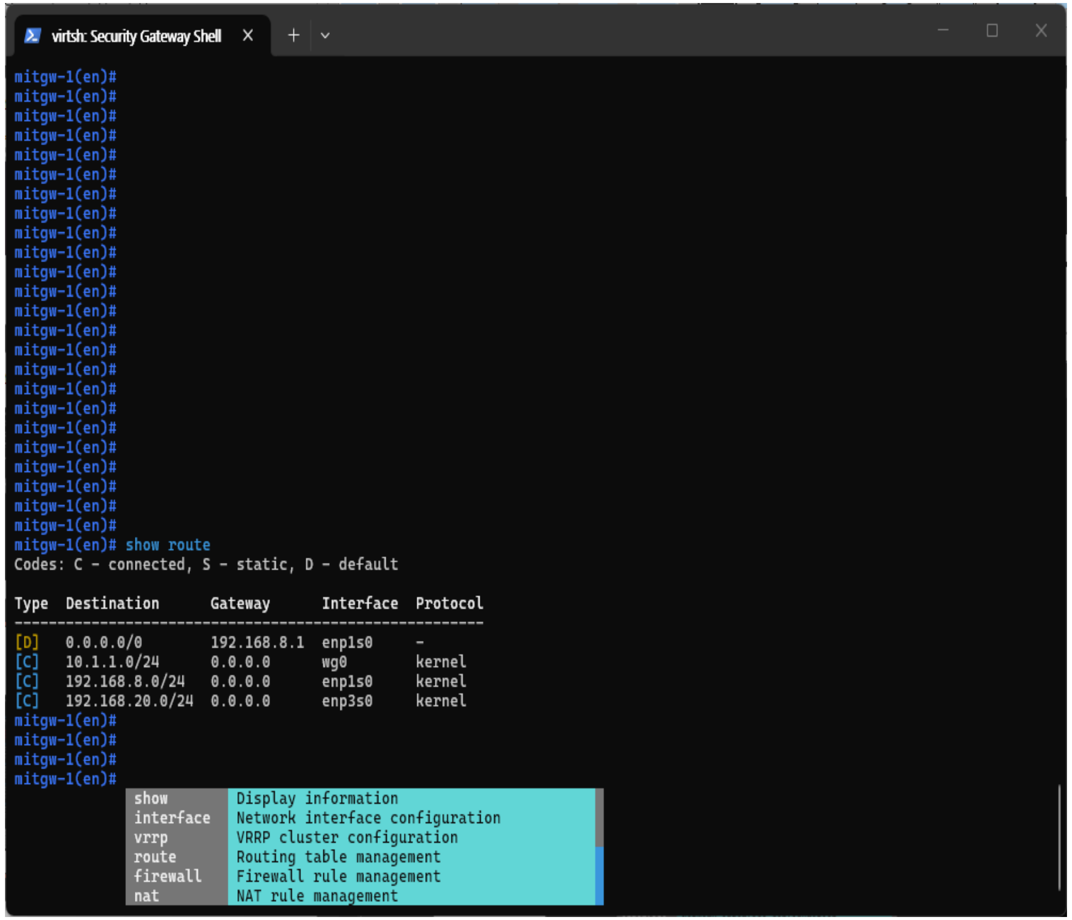
# 5. Mighty AI Security Platform(4) – CLI(1)



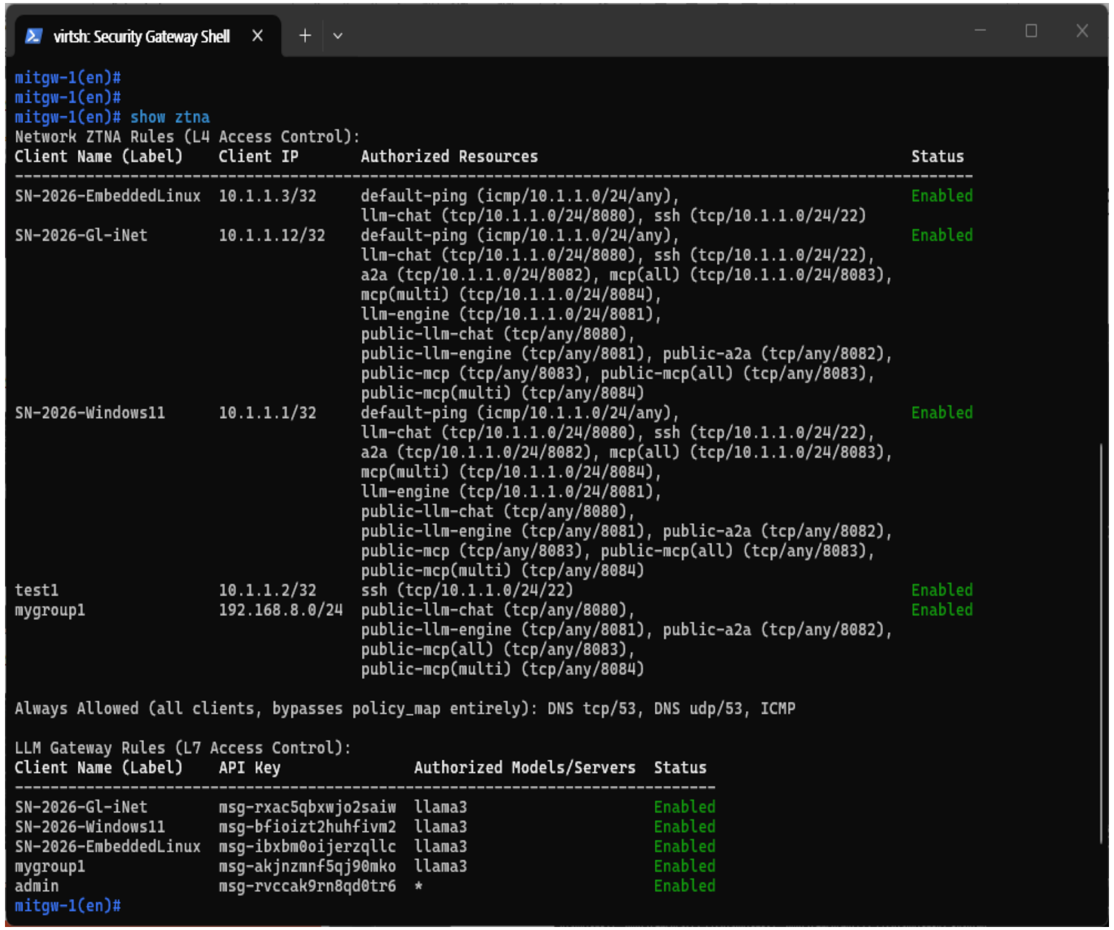



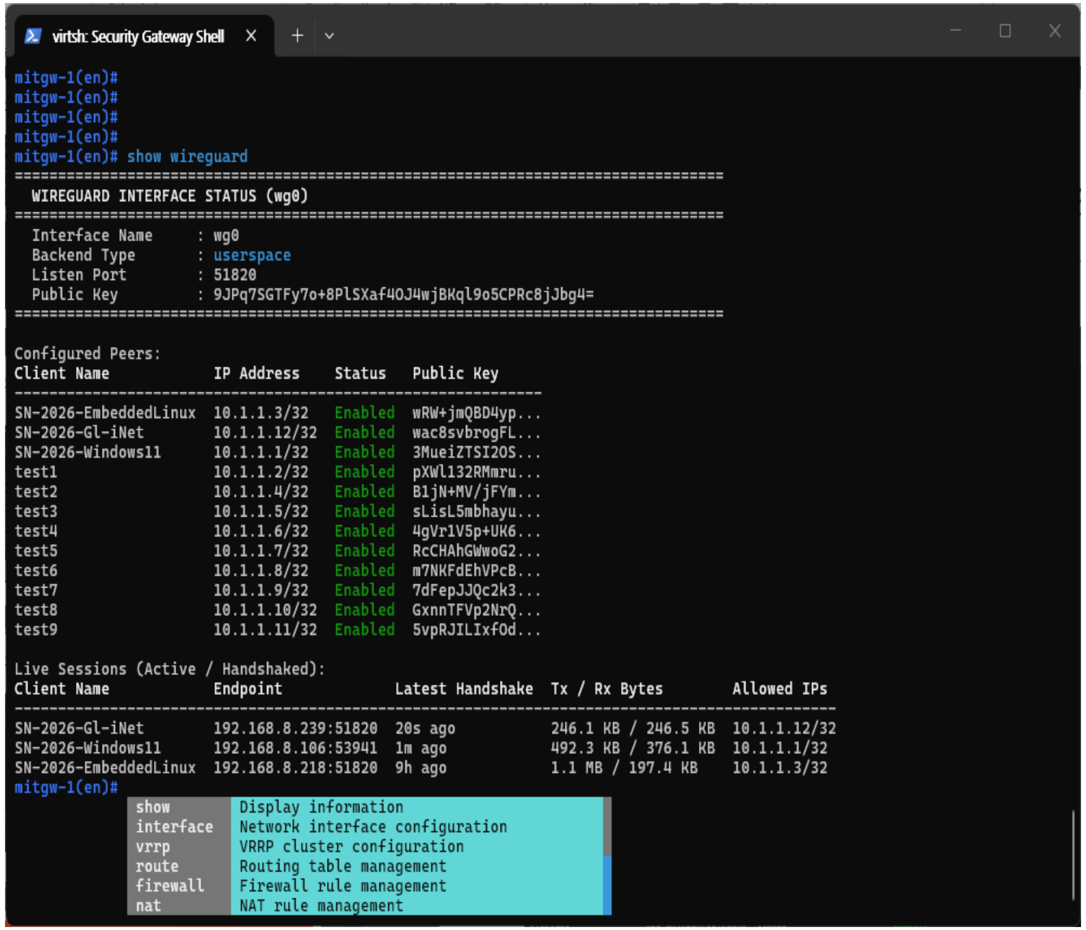
# 5. Mighty AI Security Platform(4) – CLI(2)



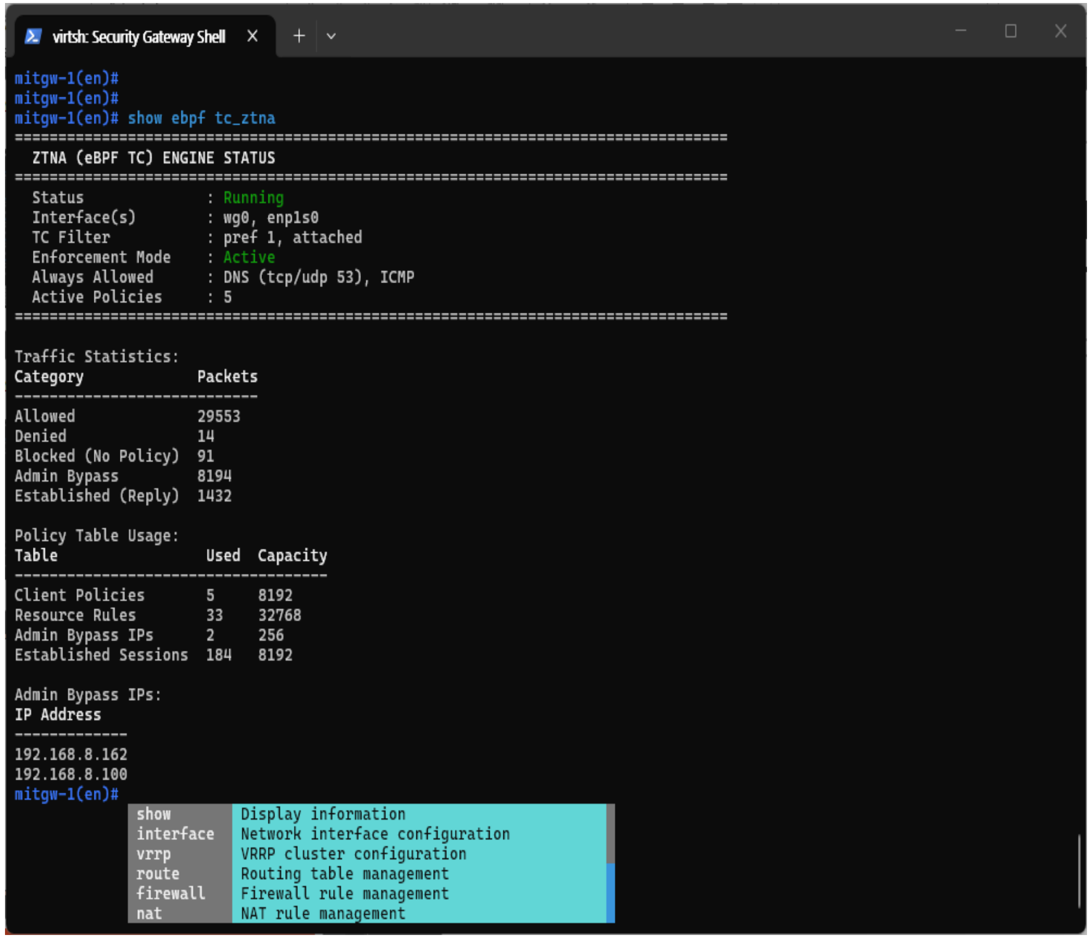



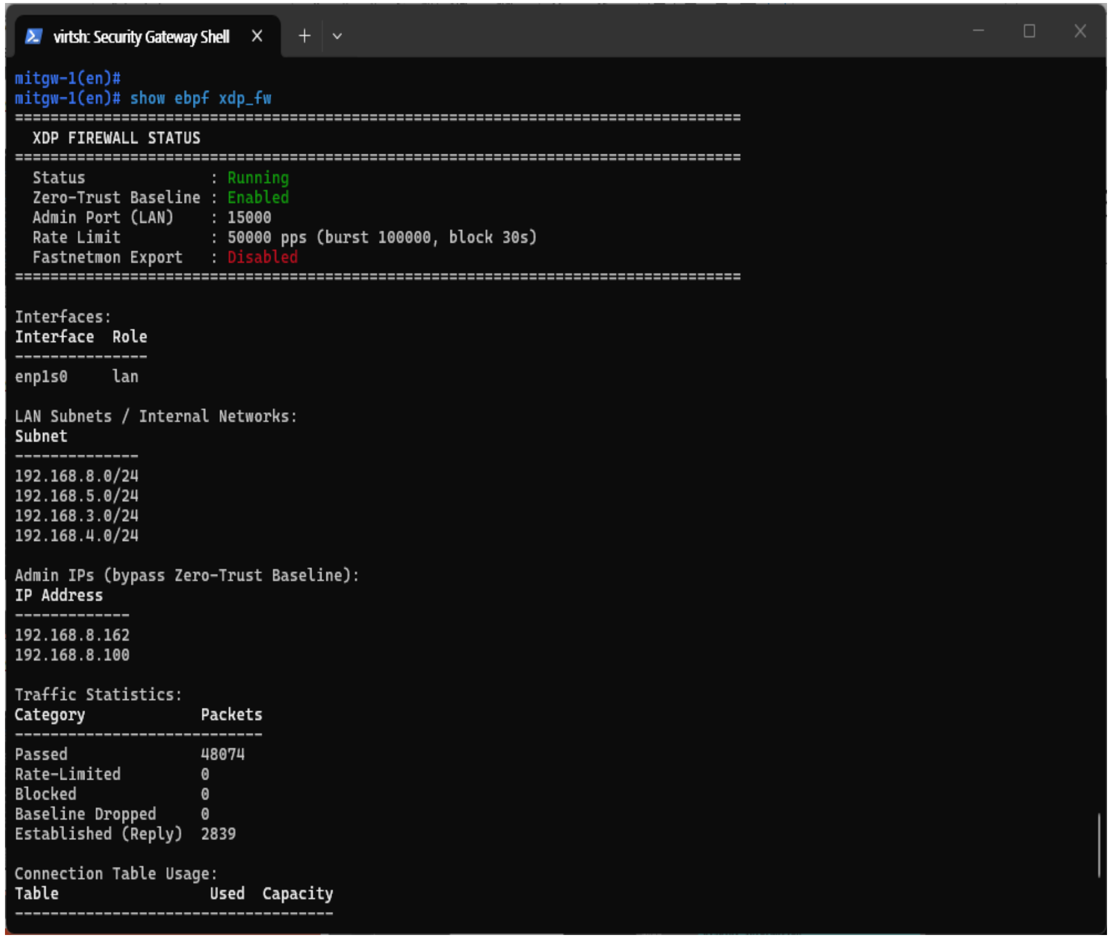
# 5. Mighty AI Security Platform(4) – CLI(3)



# 5. Mighty AI Security Platform(4) – CLI(4)



# 5. Mighty AI Security Platform(4) – CLI(5)

virtsh: Security Gateway Shell

```
  exit             Return to view mode
mitgw-1(en)#
mitgw-1(en)# show aigw
==================================================================================
  AI GATEWAY GLOBAL STATUS
==================================================================================
  Engine Type          : agentgateway
  LLM Port             : 8081
  Aux Port (Mgmt/A2A)  : 8082
  MCP Port             : 8083
  MCP Port (multi)     : 8084
  Default Provider     : openai
  DLP Filter           : true
  Moderation Filter    : false
  Max Retries          : 0
  Rate Limit (RPM)     : unlimited
==================================================================================

LLM Provider Settings:
Provider  API Key (Masked)  Weight  Models
----------------------------------------------------
gemini    AIza...agu8       -       gemini-3.5-flash,
                                    gemini-flash-latest

Internal LLM Servers:
Server Name  Engine Type  Weight  Server URL
---------------------------------------------------------
llama3       ollama       -       http://192.168.8.187:11434

MCP Gateway Endpoints:
Name    Mode   Targets        Auth  Access Control (CEL)  Rate Limit  Failure Mode  Enabled
-------------------------------------------------------------------------------------------
test2   all    stdio:test2    None  2 rule(s)             -           fail_closed   true
test3   multi  stdio:test3    None  None (open)           -           fail_closed   true
test4   multi  http:test4     None  None (open)           -           fail_closed   true
test5   multi  sse:test5      None  None (open)           -           fail_closed   true
test6   multi  openapi:test6  None  None (open)           -           fail_closed   true
test7   multi  http:test7     None  None (open)           100/60s     fail_closed   true
test8   all    http:test8     None  None (open)           100/60s     fail_closed   true
test9   multi  http:test9     JWT   None (open)           -           fail_closed   true
test10  multi  http:test10    None  None (open)           -           fail_closed   true
test11  multi  http:test11    None  None (open)           -           fail_closed   true
test12  multi  http:test12    None  None (open)           100/60s     fail_closed   true
```

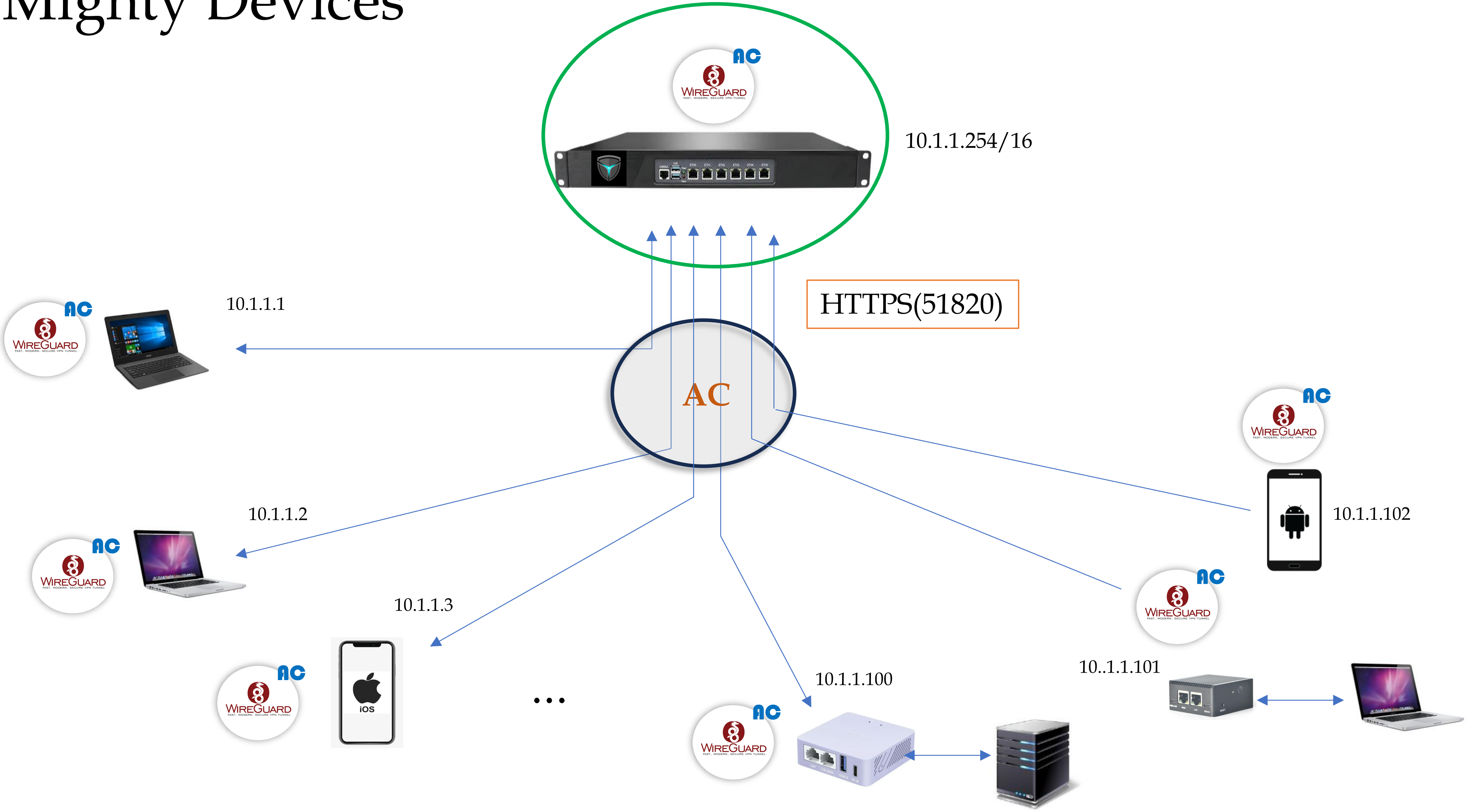
virtsh: Security Gateway Shell

```
A2A Agents:
Name   Path                Backend              Enabled
-----------------------------------------------------
test1  /agents/test-echo  192.168.8.162:8300   true
mitgw-1(en)#
mitgw-1(en)# ?

Available commands:
  show            Display information
  interface       Network interface configuration
  vrrp            VRRP cluster configuration
  route           Routing table management
  firewall        Firewall rule management
  nat             NAT rule management
  wireguard       Wireguard management
  ztna            Zero Trust Network Access rules
  ebpf            eBPF component status and control
  fastnetmon      fastnetmon DDoS detection service control
  aigw            AI Gateway Engine operations
  system          System operations
  write           Commit staged changes to the system
  clear           Discard staged changes
  exit            Return to view mode
mitgw-1(en)# show system

Possible completions:
  hostname        Show system hostname
  dns             Show current DNS servers
  2fa             Show 2FA status and policy
  monit           Show Monit process monitoring status
  upgrade         Show configured upgrade server
  version         Show installed software version and build date
  time            Show time synchronization settings and NTP status
  mightysg        Show mightysg process status
mitgw-1(en)#
mitgw-1(en)#
mitgw-1(en)#
             show       Display information
             interface  Network interface configuration
             vrrp       VRRP cluster configuration
             route      Routing table management
             firewall   Firewall rule management
             nat        NAT rule management
```

# 6. Mighty Devices

**Mighty Devices**

# 6. Mighty Devices



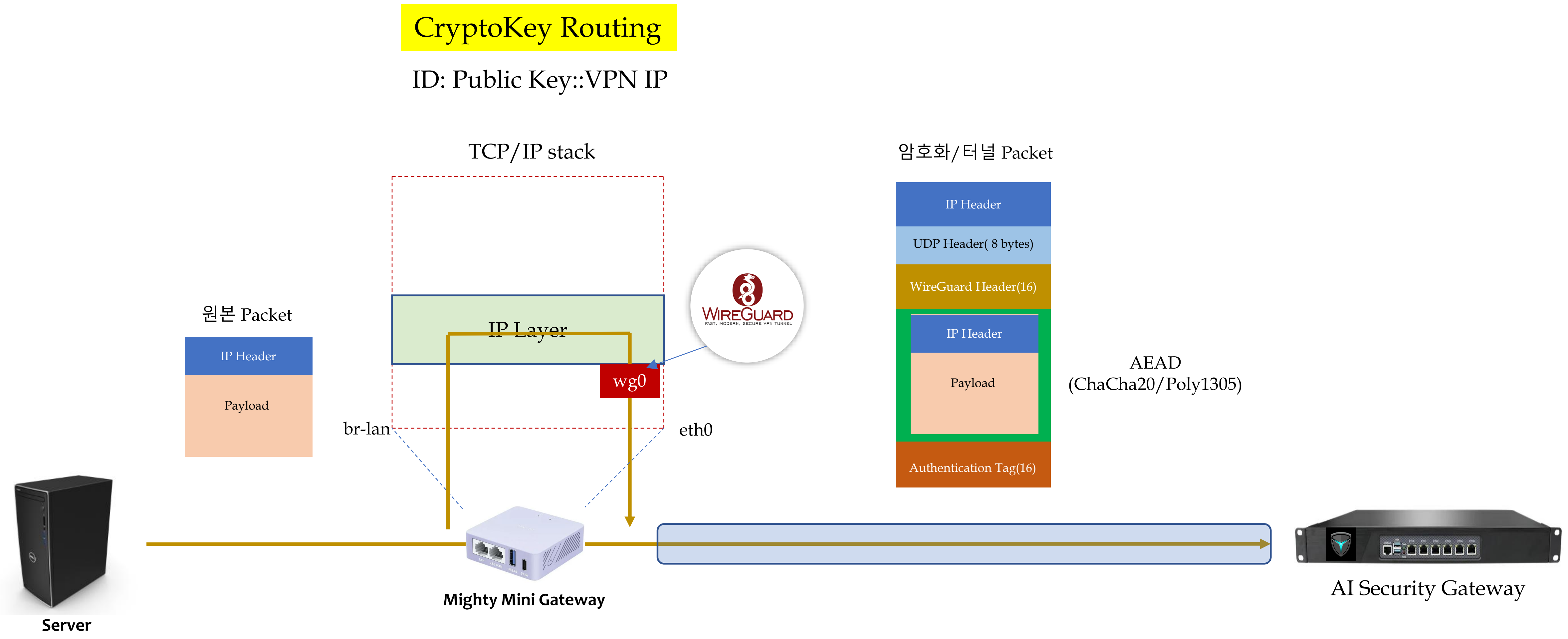
# 6.1 Mighty Mini Gateway

## Mighty Mini Security Gateway

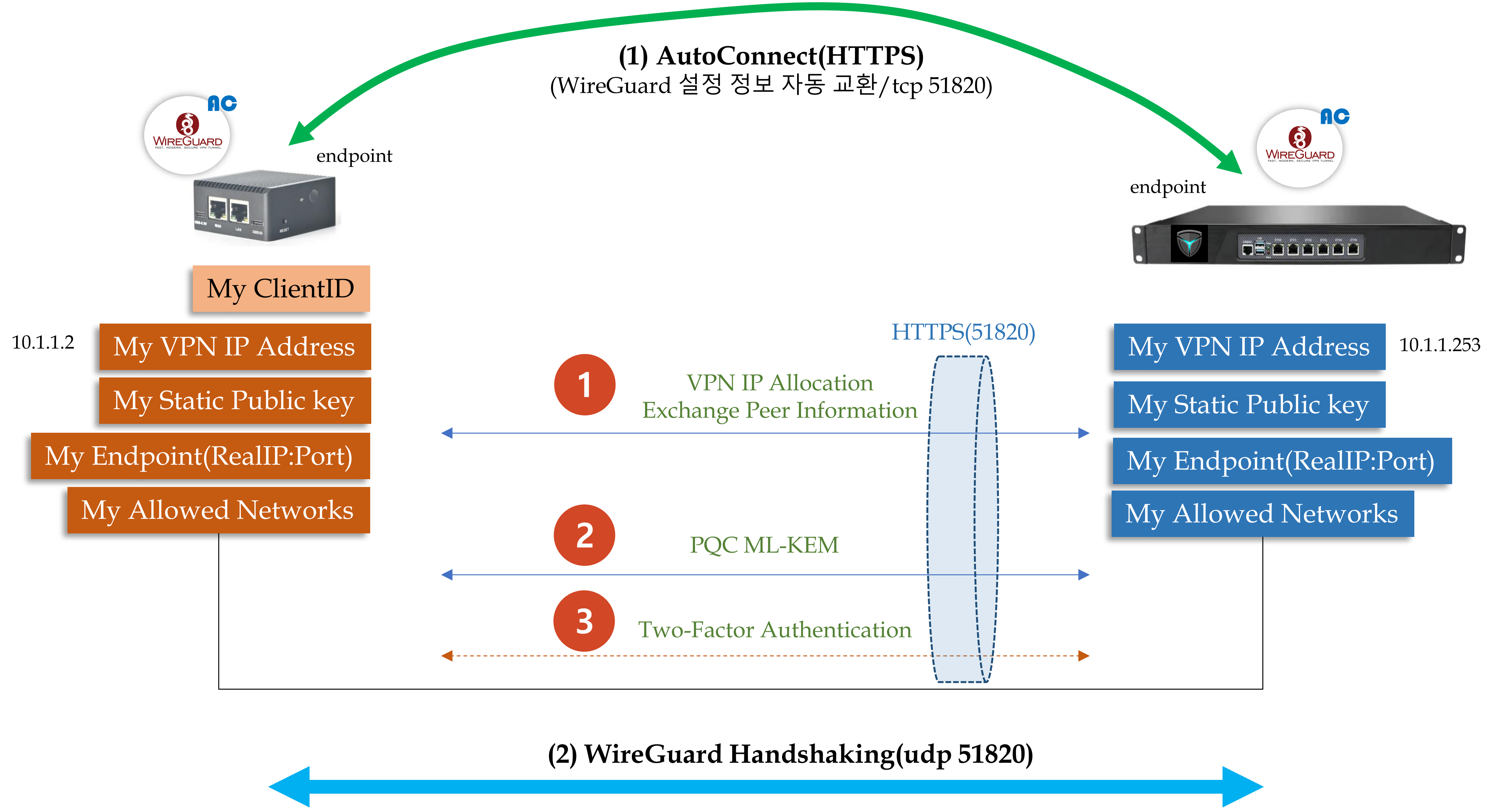
GL-iNet Series
https://www.gl-inet.com/

NanoPi Series
https://www.friendlyelec.com/

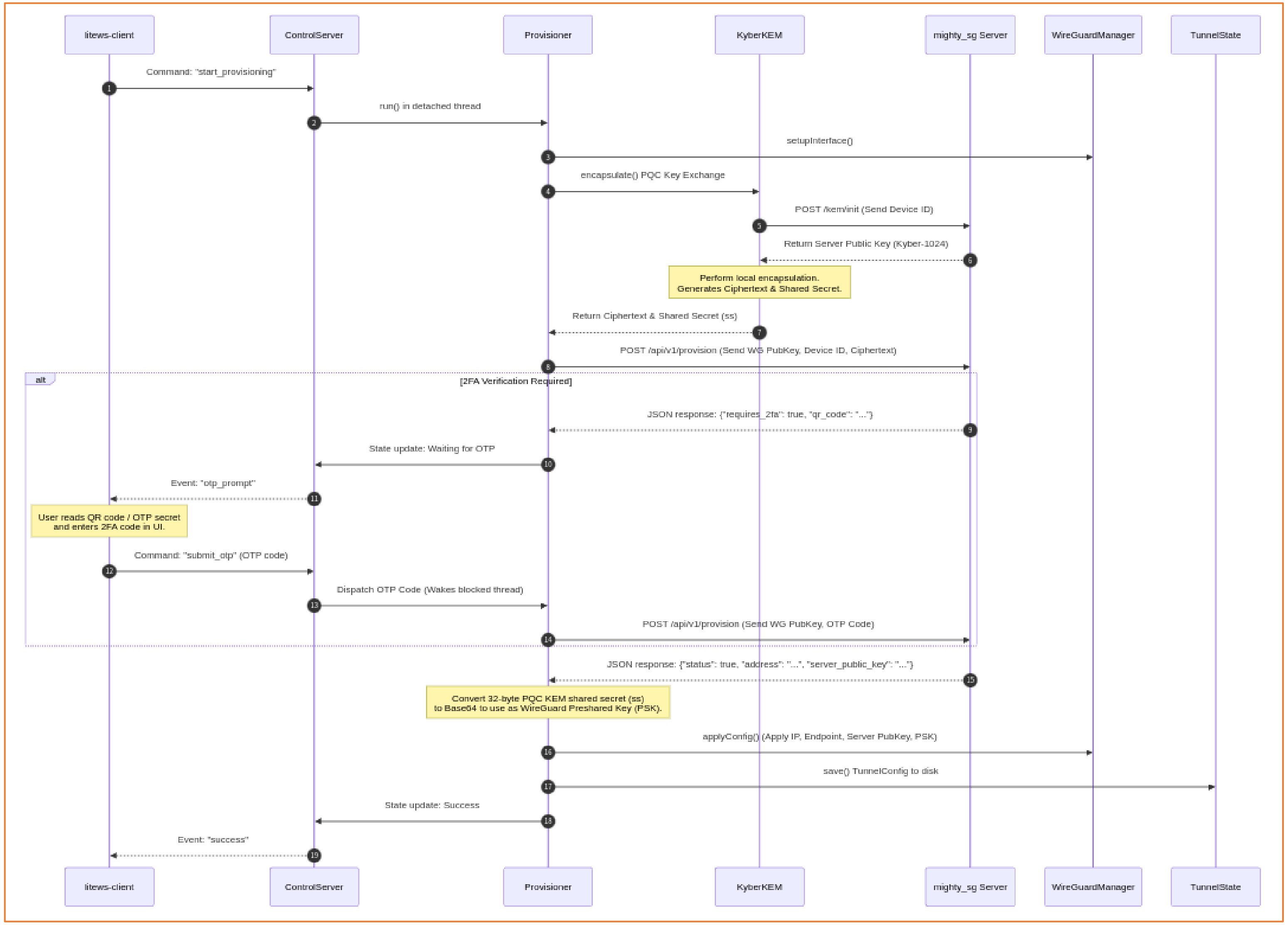
# 6.1 Mighty Mini Gateway(1) – Overview



# 6.1 Mighty Mini Gateway(2) – Auto Connection

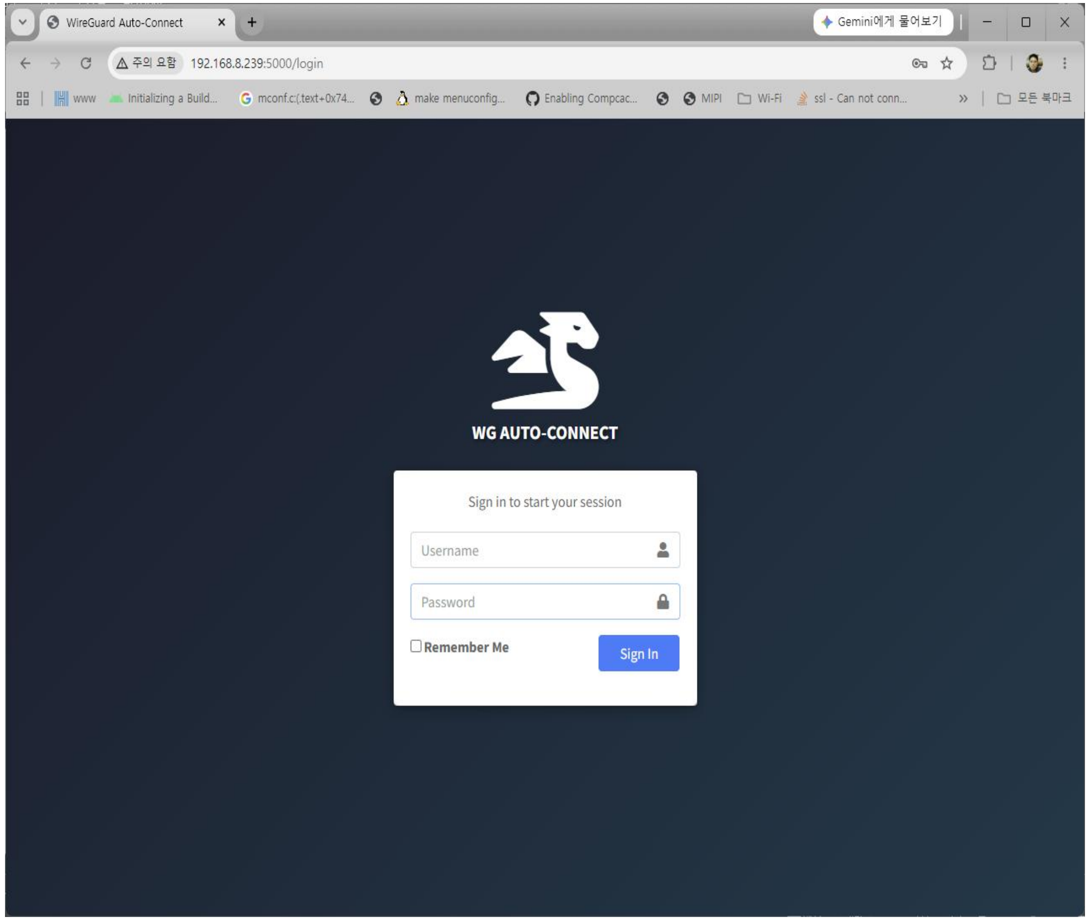


# 6.1 Mighty Mini Gateway(3) – Architecture

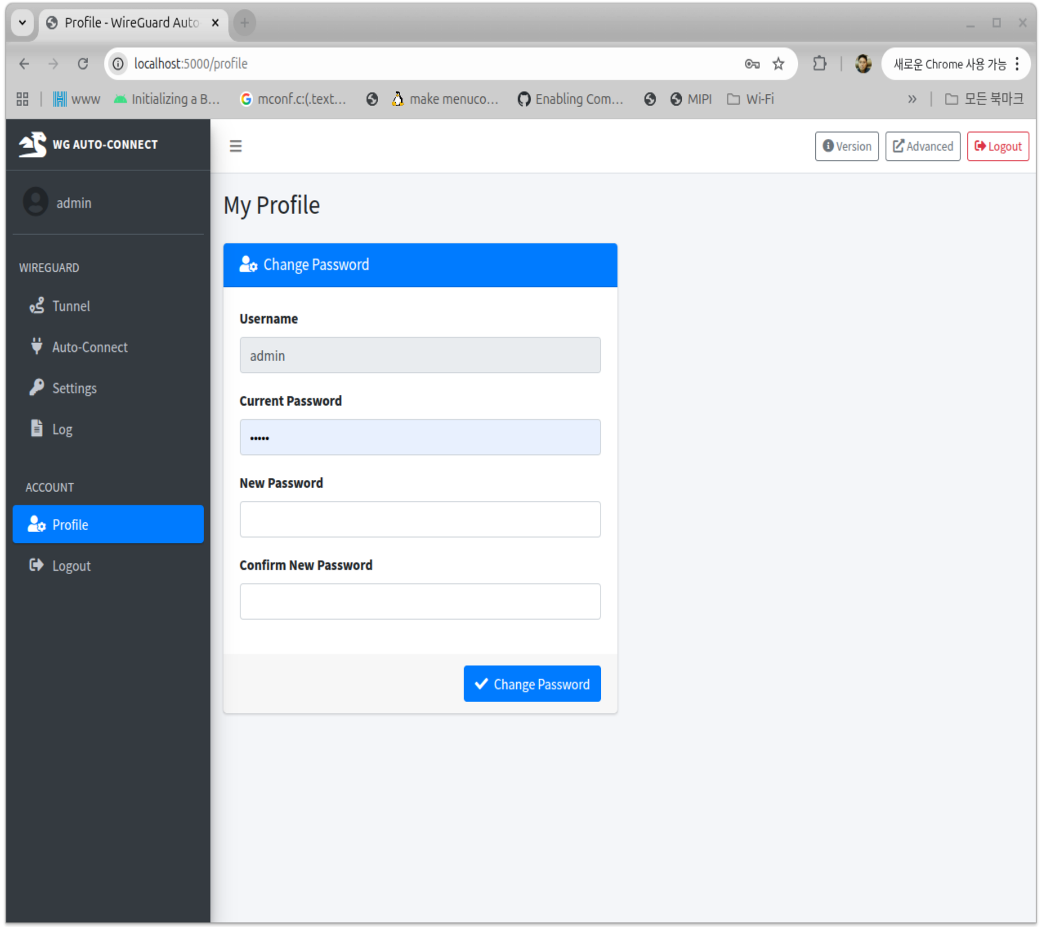


# 6.1 Mighty Mini Gateway(4) – WebUI(1)

- Login Page

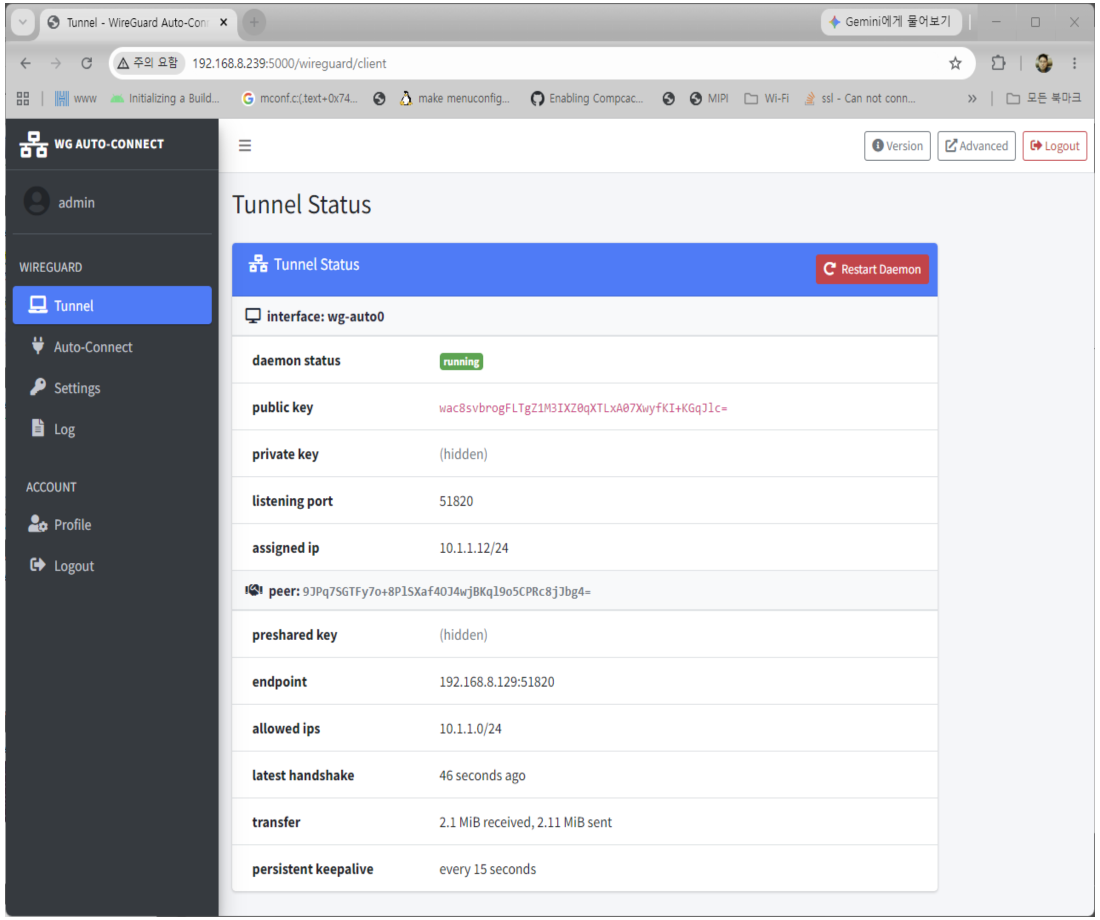



- Admin User Page

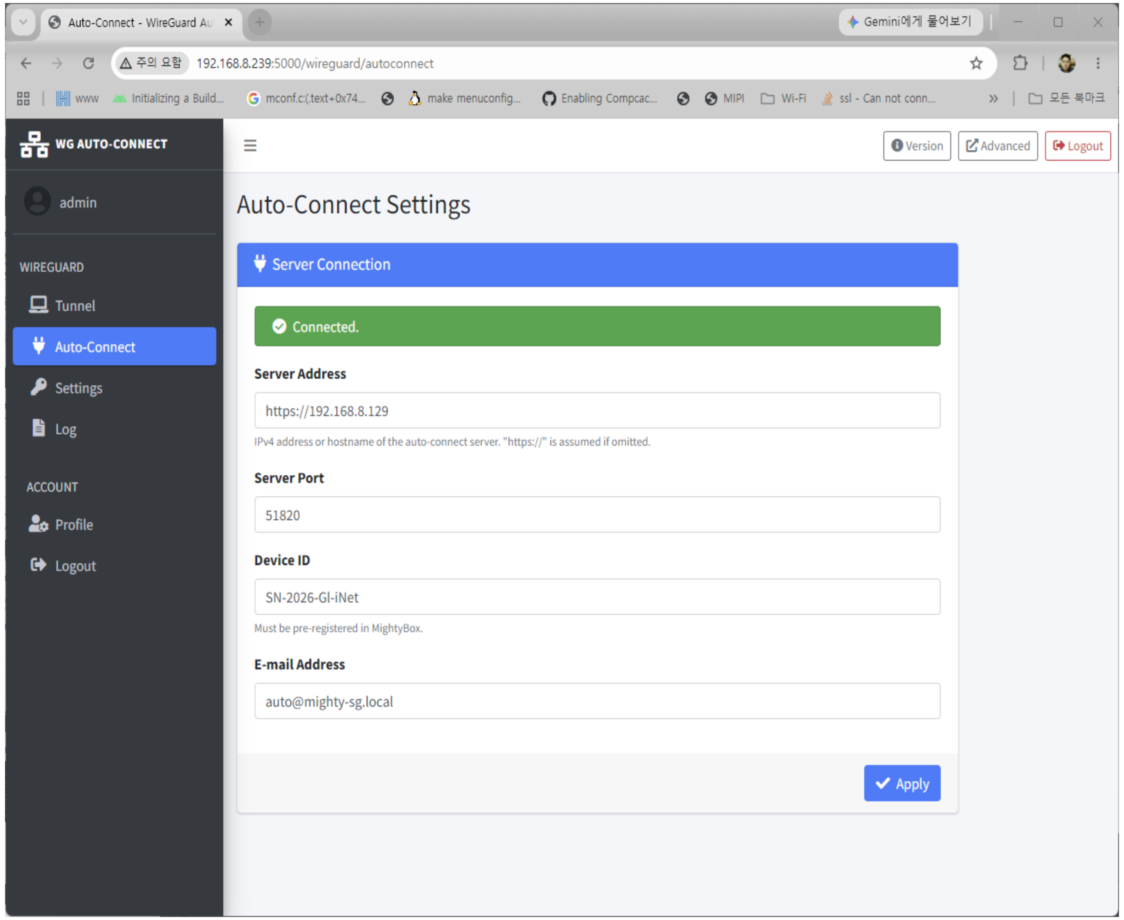


# 6.1 Mighty Mini Gateway(4) – WebUI(2)

- Tunnel Status Page

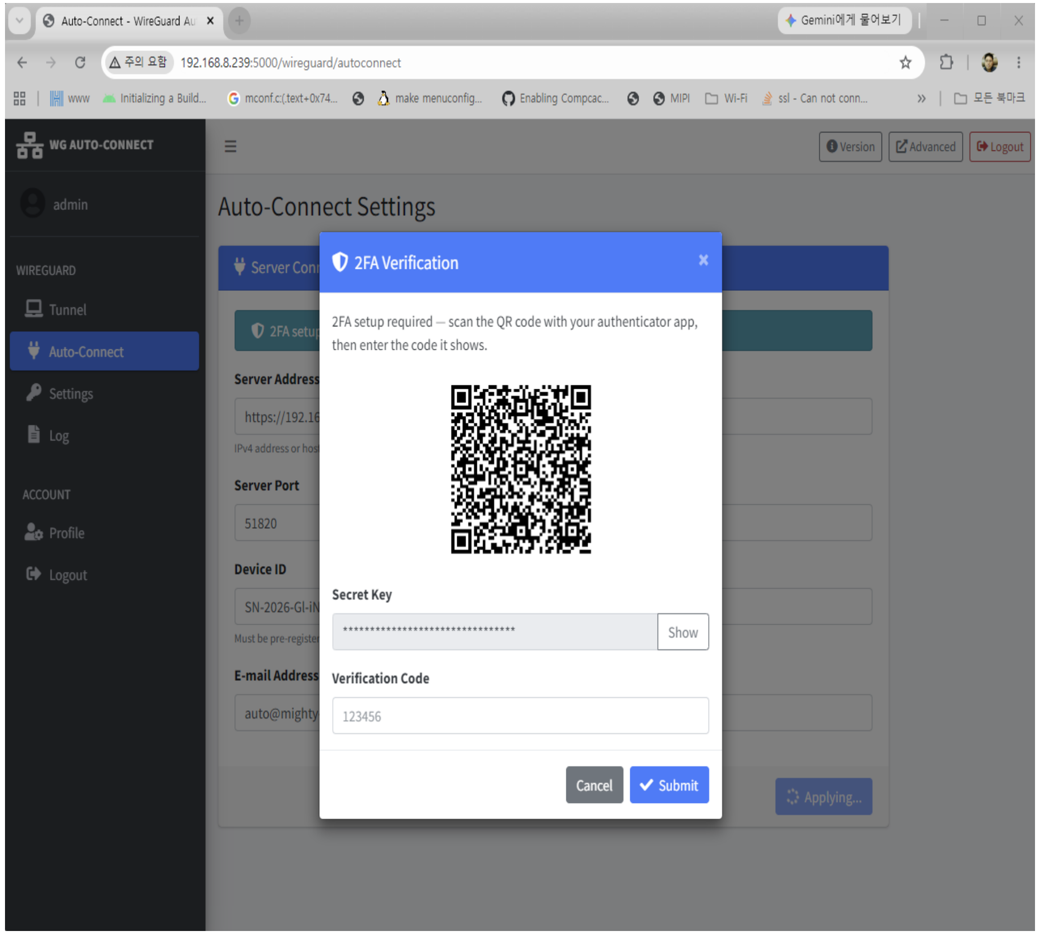



- Auto Connection Page

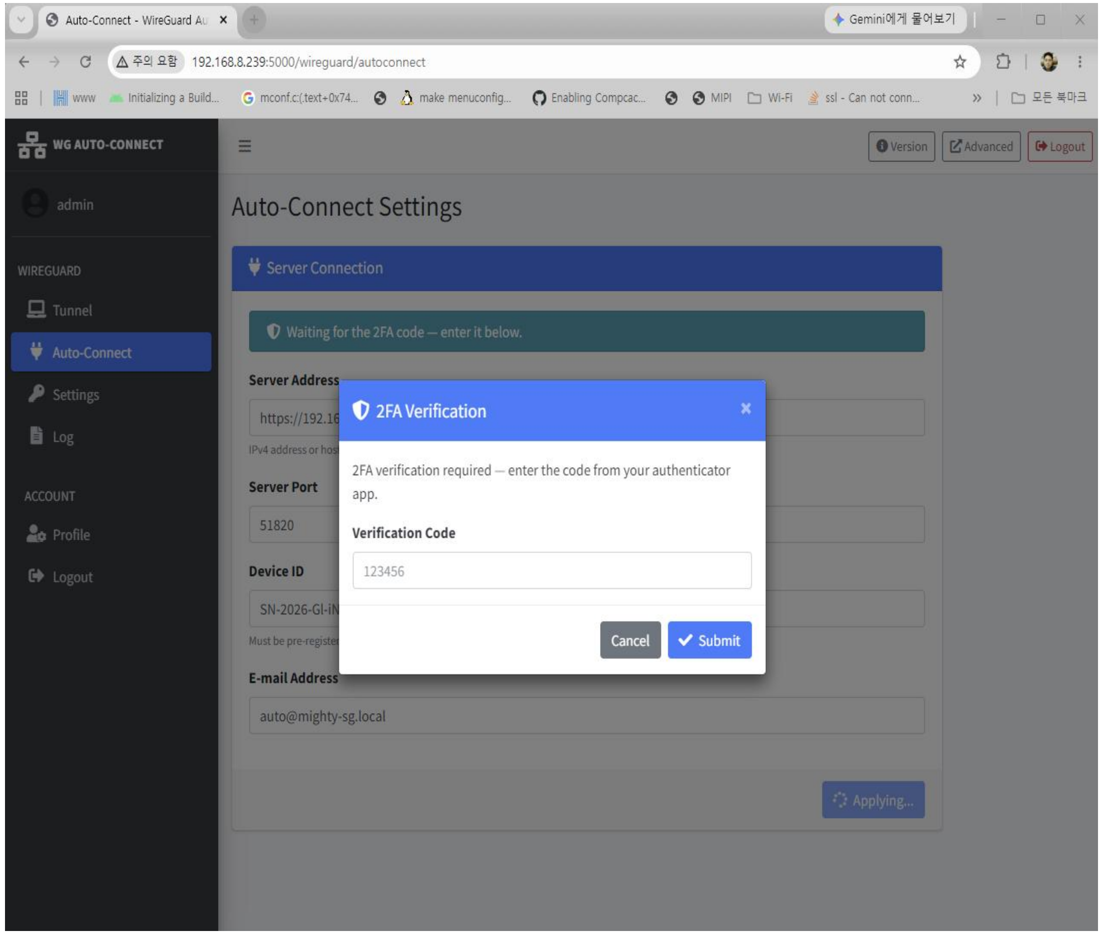


# 6.1 Mighty Mini Gateway(4) – WebUI(3)

- Auto Connection 2FA Step1 Page

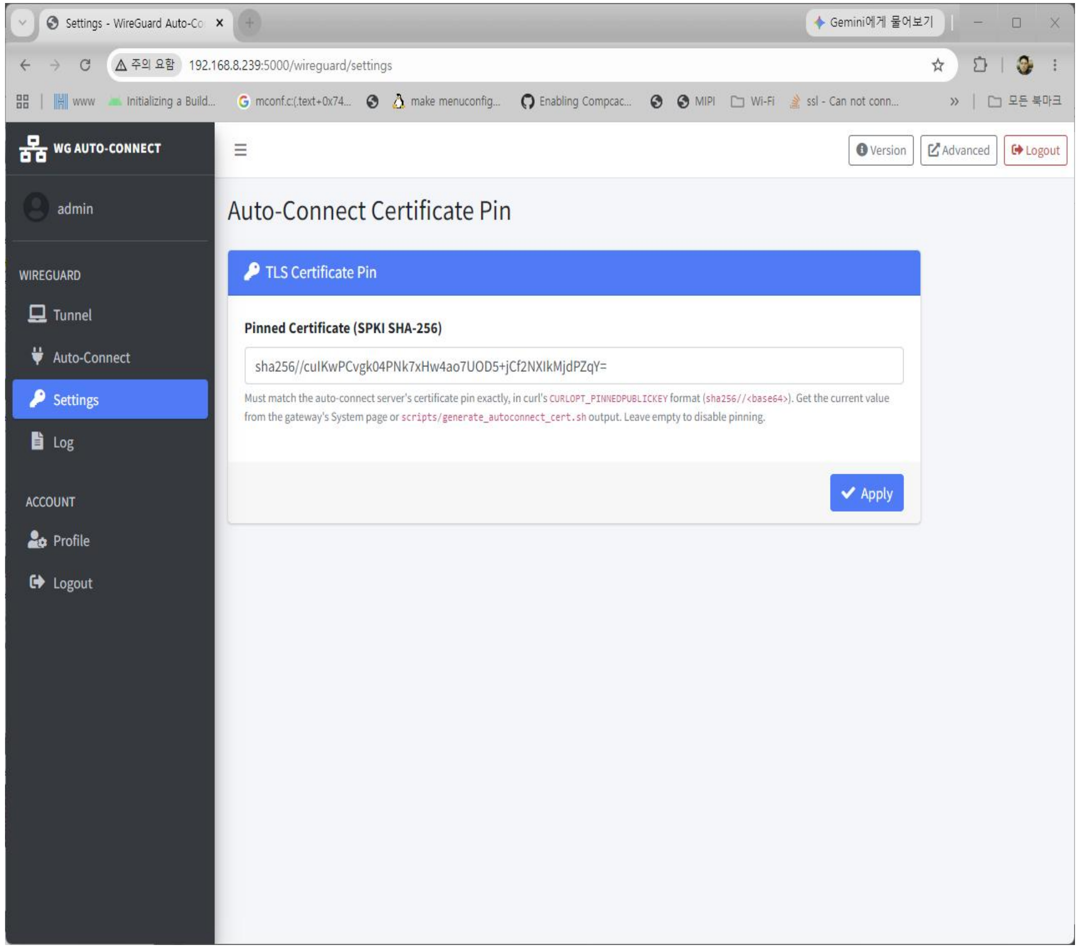



- Auto Connection 2FA Step2 Page

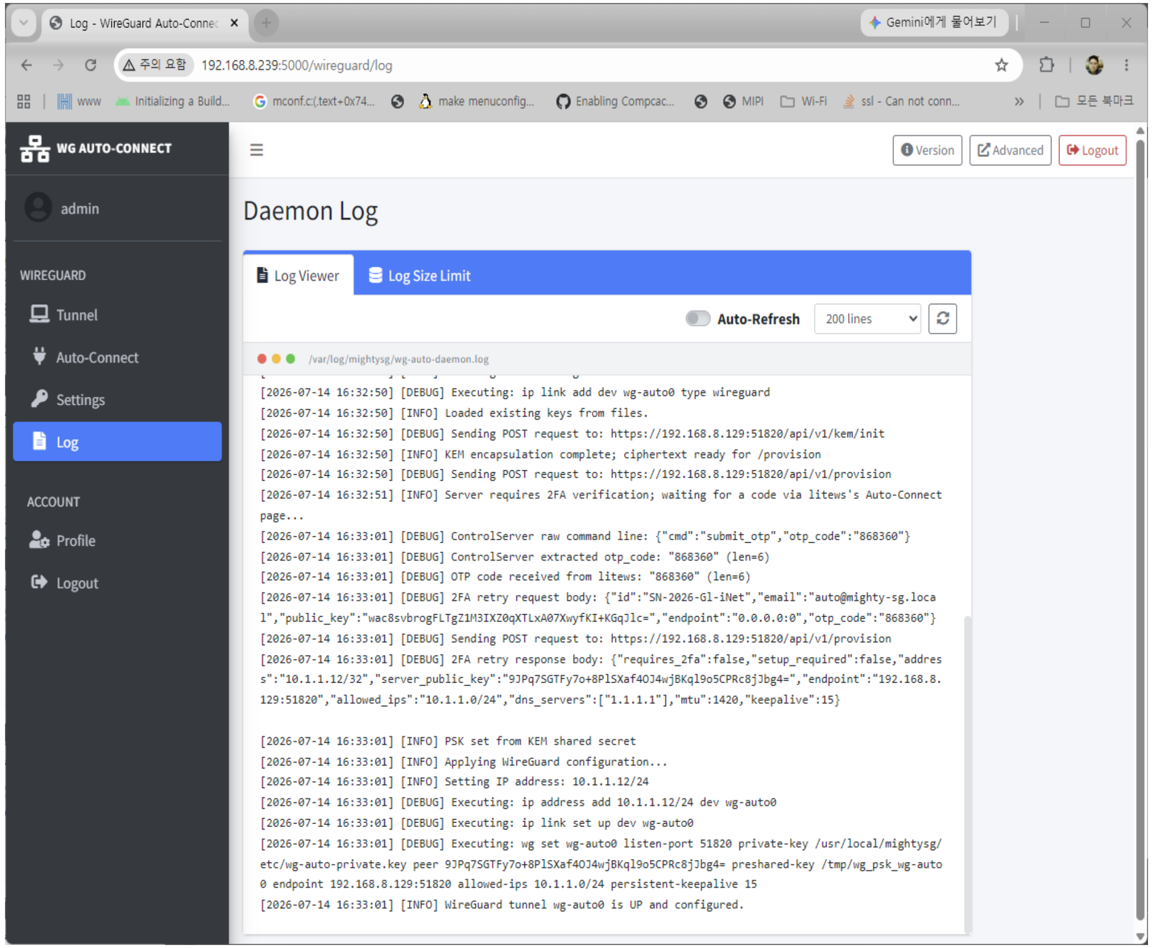


# 6.1 Mighty Mini Gateway(4) – WebUI(4)

- Auto Connection X.509 Certificate Pin Page



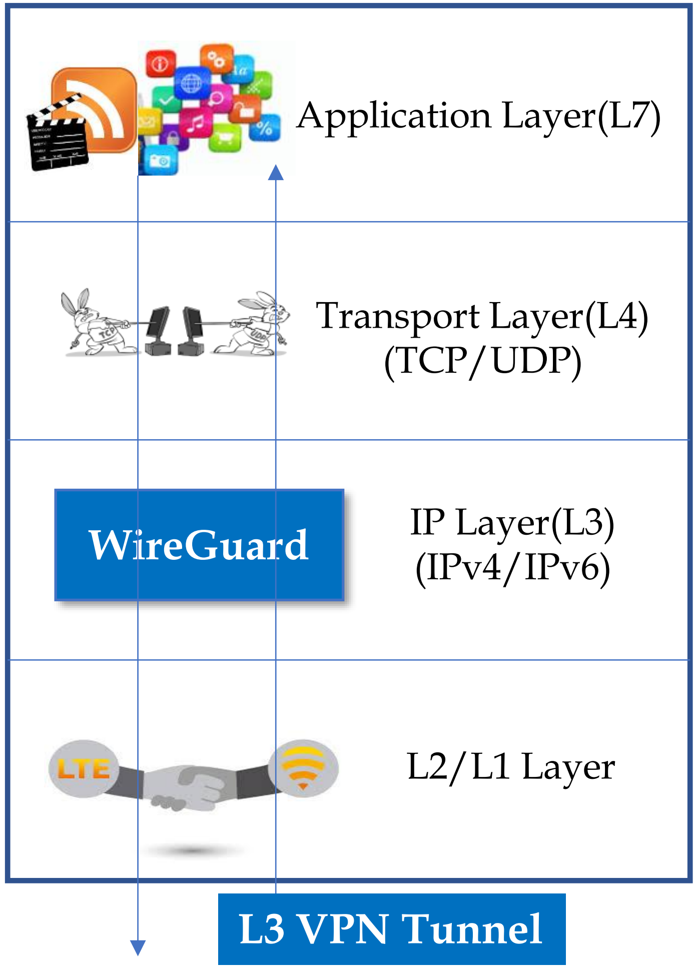

- Log Page



# 6.2 Mighty Ubuntu Linux Agent

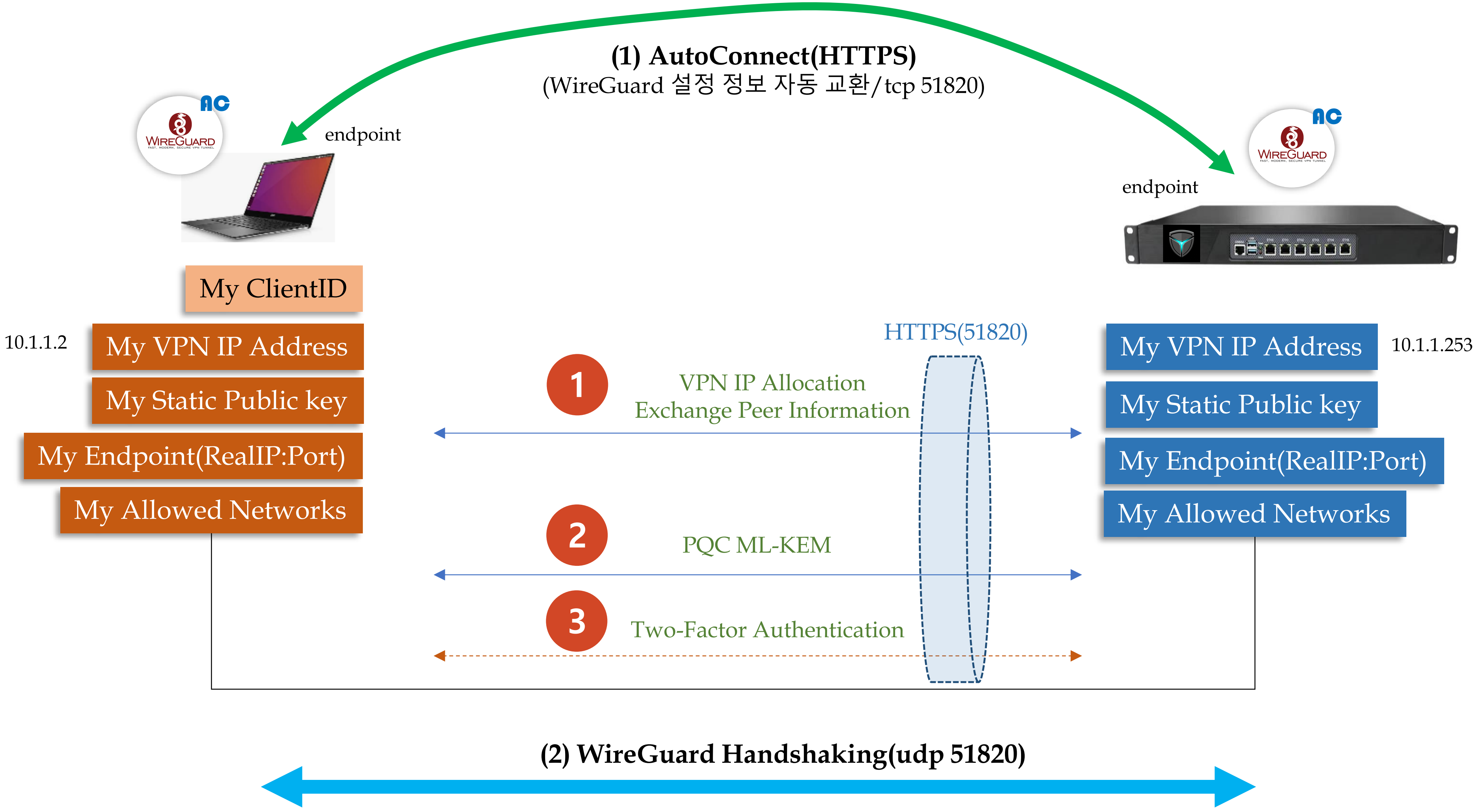
# 6.2 Mighty Linux Agent(1) – WireGuard

**Cryptokey Routing**




```
chyi@earth-new:/mnt/hdd/workspace/mighty_sg_prj/07162026$ sudo wg show
interface: wg-auto0
  public key: NYJM7SxhjeQXjjipIsEiNxZY3w6NcPJl/Mfjfye4mzY=
  private key: (hidden)
  listening port: 51820

peer: 9JPq7SGTFy7o+8PlSXaf4OJ4wjBKql9o5CPRc8jJbg4=
  preshared key: (hidden)
  endpoint: 192.168.8.129:51820
  allowed ips: 10.1.1.0/24
  latest handshake: 1 minute, 40 seconds ago
  transfer: 316 B received, 180 B sent
  persistent keepalive: every 15 seconds
chyi@earth-new:/mnt/hdd/workspace/mighty_sg_prj/07162026$ ping 10.1.1.253
PING 10.1.1.253 (10.1.1.253) 56(84) bytes of data.
64 bytes from 10.1.1.253: icmp_seq=1 ttl=64 time=2.70 ms
64 bytes from 10.1.1.253: icmp_seq=2 ttl=64 time=2.94 ms
^C
--- 10.1.1.253 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1002ms
rtt min/avg/max/mdev = 2.698/2.817/2.936/0.119 ms
```

# 6.2 Mighty Linux Agent(2) – Auto Connection

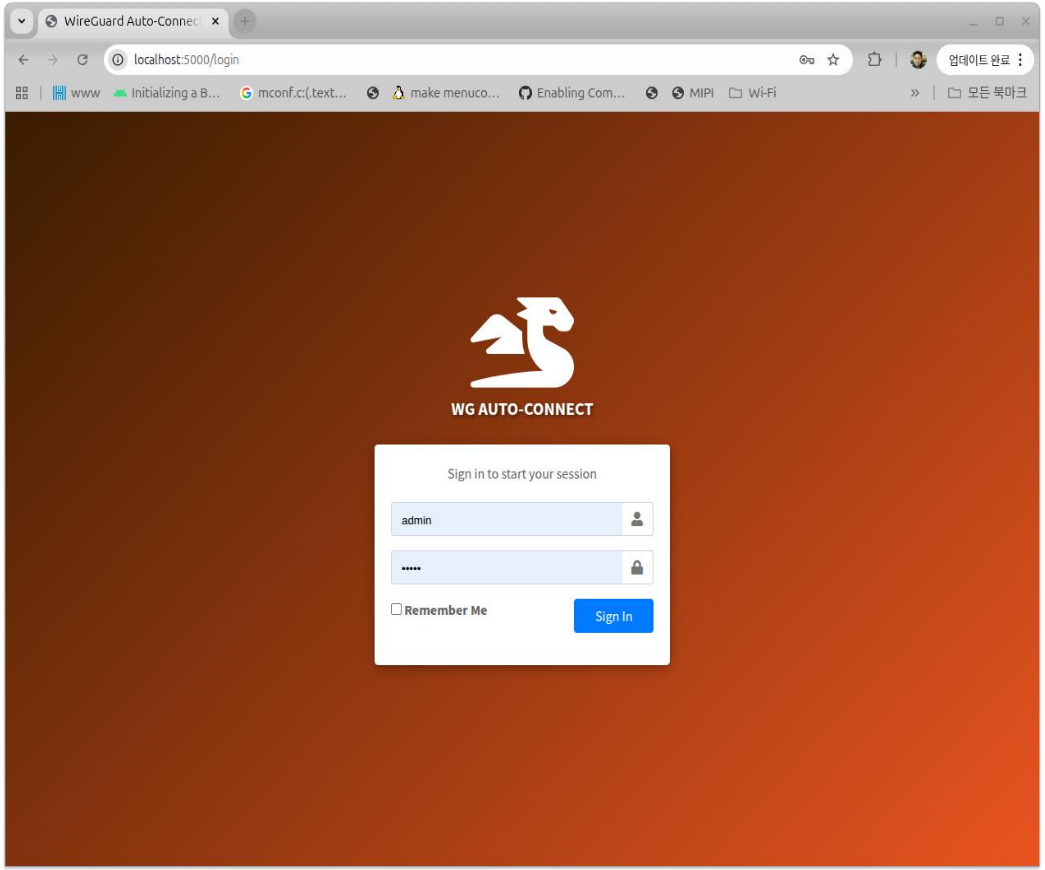


# 6.2 Mighty Linux Agent(3) – Installation

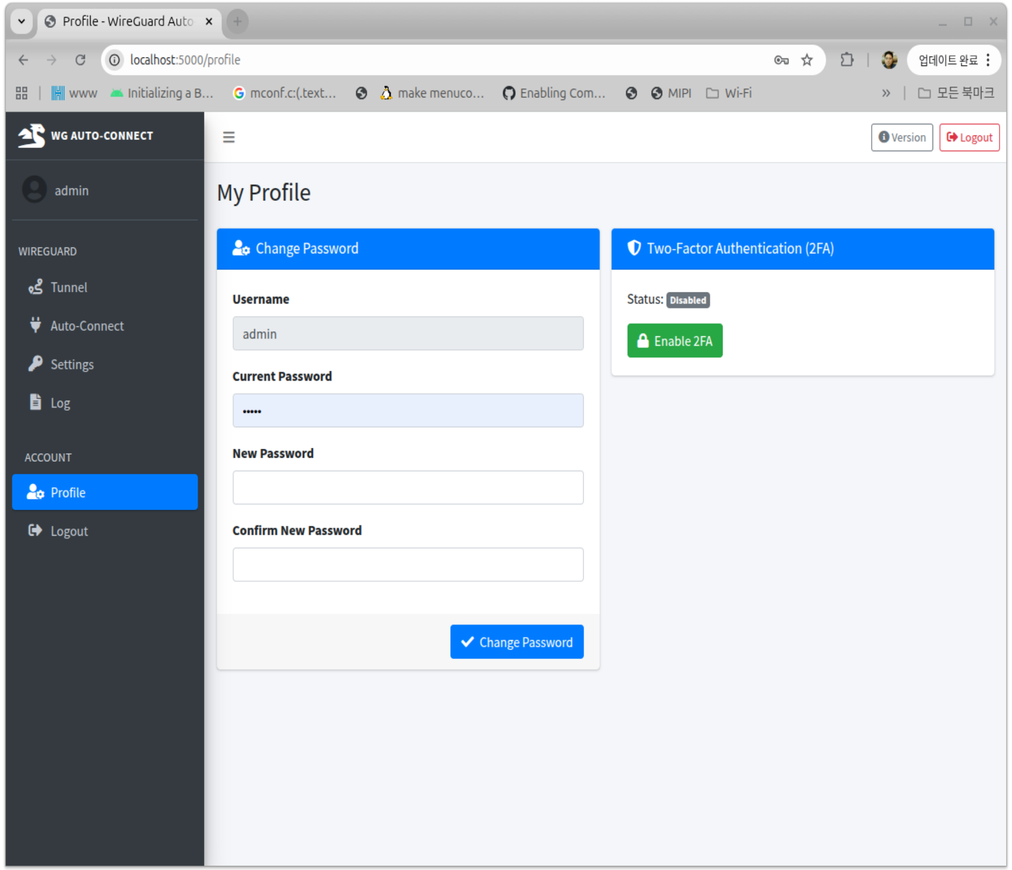
```
chyi@earth-new:/mnt/hdd/workspace/mighty_sg_prj/07162026$ ./Install.sh
Error: this script must be run as root.
chyi@earth-new:/mnt/hdd/workspace/mighty_sg_prj/07162026$ sudo ./Install.sh
[sudo] chyi 암호:
==================================================
  WireGuard Auto-Connect Client — Installer
==================================================
  Log file: /var/log/mightysg/wg_client_install.log

  [1/6] Installing apt packages                        OK
  [2/6] Extracting package files                       OK
  [3/6] Seeding wg-auto-daemon configuration           OK
  [4/6] Configuring firewall (51820 tcp/udp)           OK
  [5/6] Registering systemd services                   OK
  [6/6] Starting services                              OK

  Installation complete! Services are already running.
  Installed at : /usr/local/mightysg
  Services     : wg-auto-daemon.service, litews.service (systemctl status <name>)
  Web UI       : http://<device-ip>:5000  (admin/admin — change it on the Profile page)
  Config       : /usr/local/mightysg/etc/wg-auto-config.json
  Log          : /var/log/mightysg/wg_client_install.log
==================================================
```

# 6.2 Mighty Linux Agent(4) – WebUI(1)

- Login Page

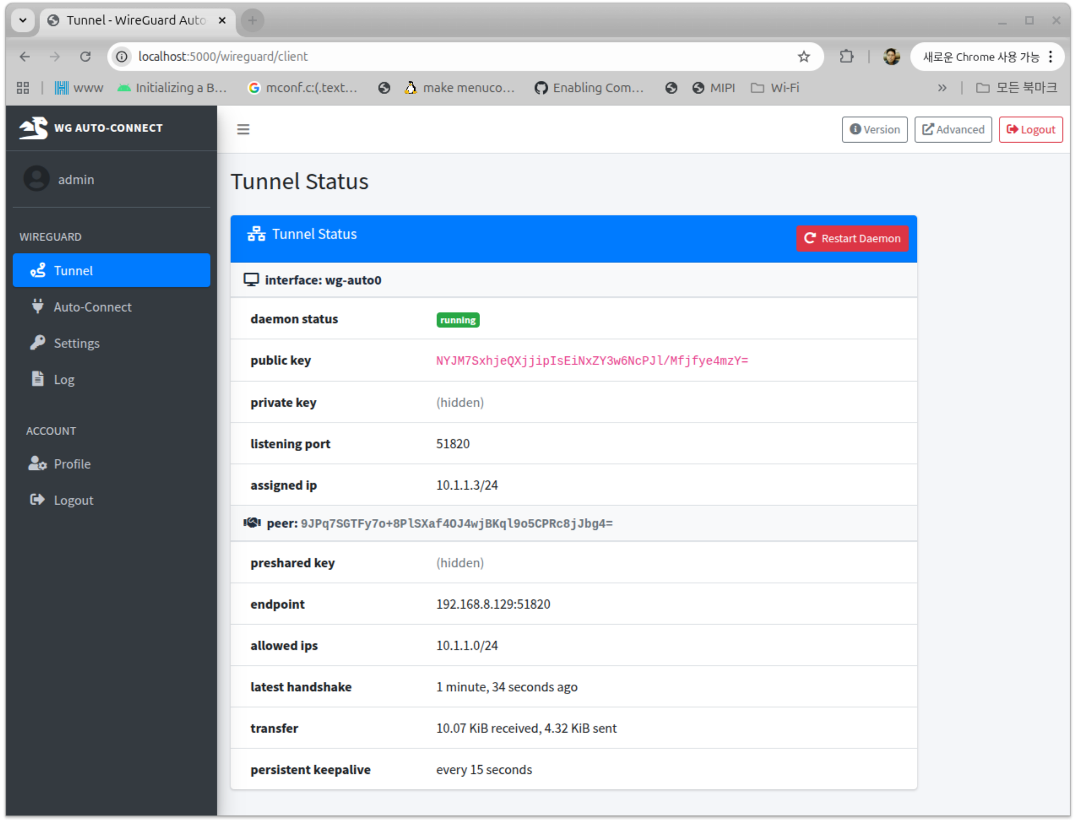



- Admin User Page

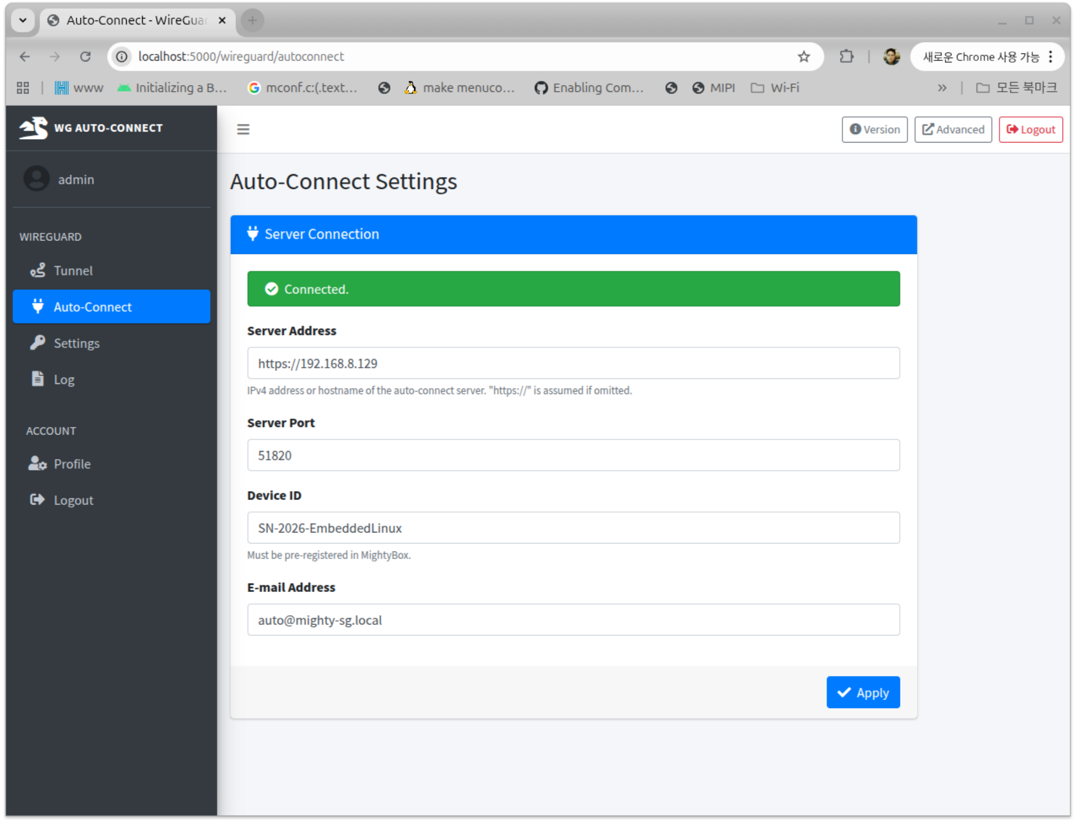


# 6.2 Mighty Linux Agent(4) – WebUI(2)

- Tunnel Status Page




- Auto Connection Page

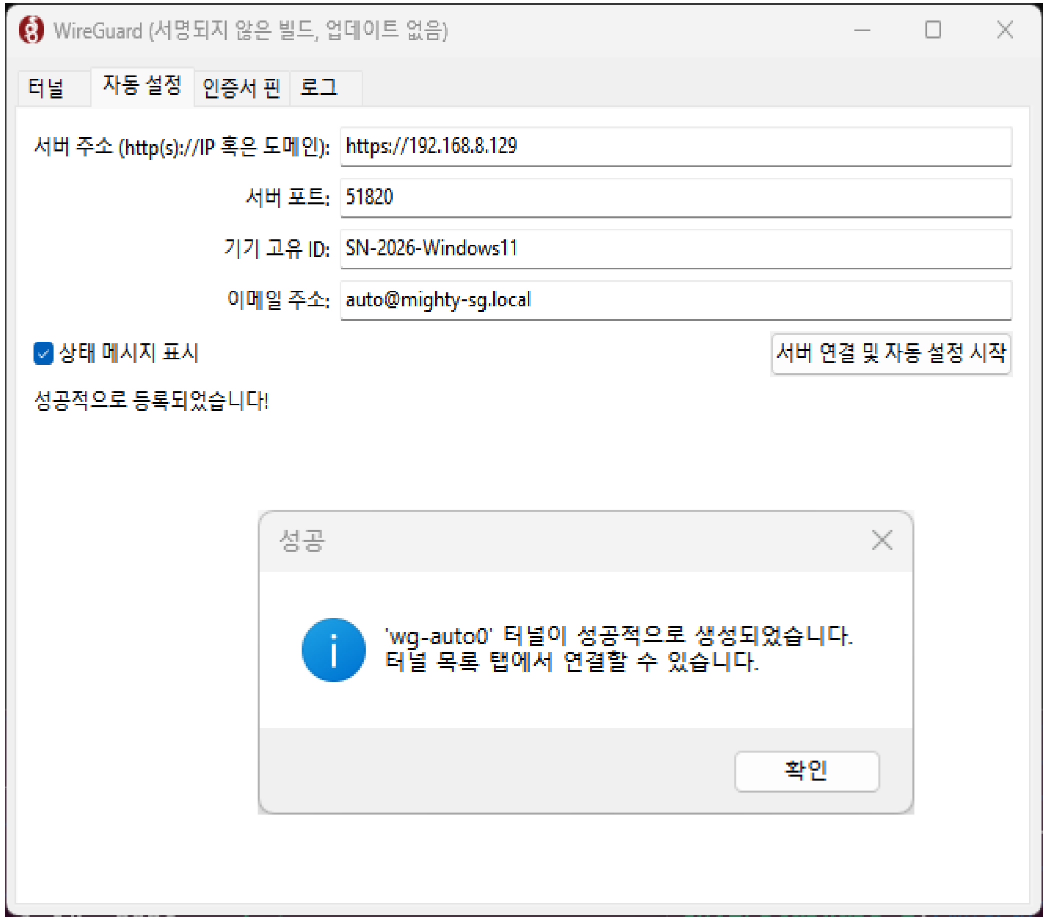


# 6.3 Mighty Windows Agent

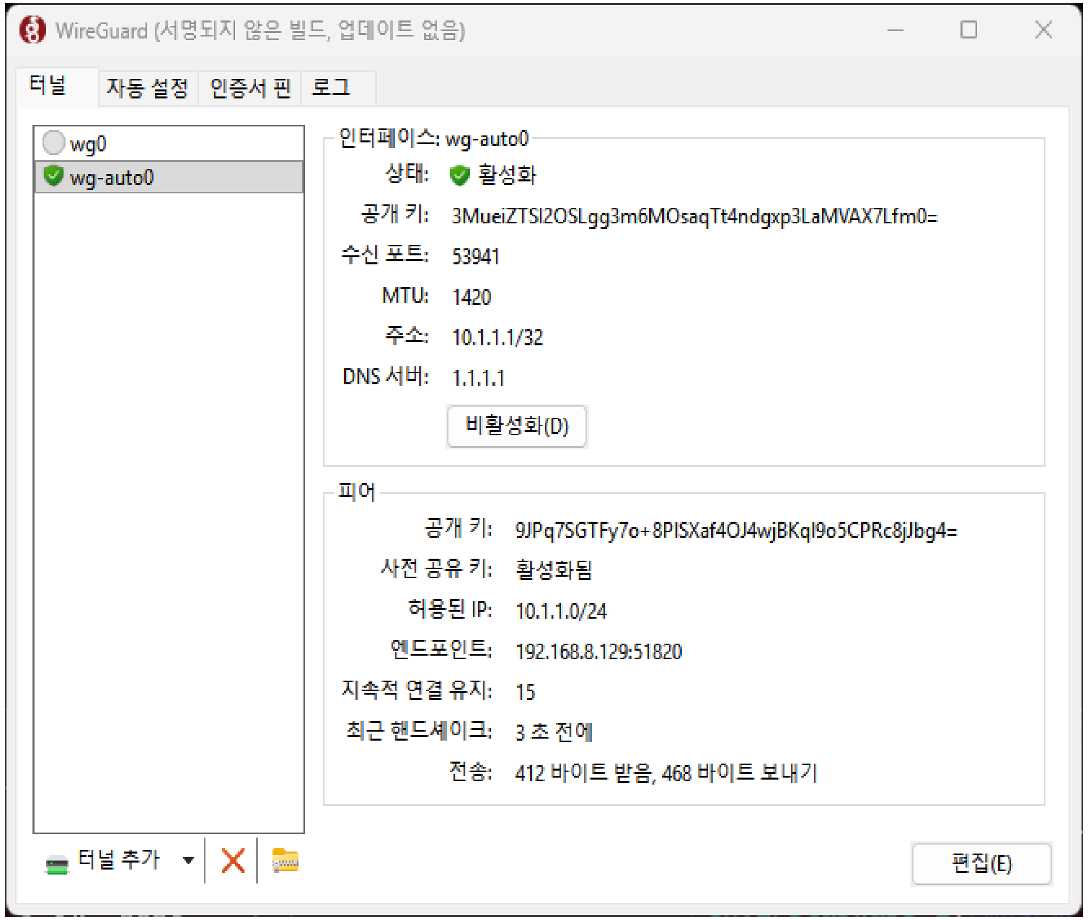
## Mighty Windows Agent

# 6.3 Mighty Windows Agent(1) – User Interface(1)

- Auto Connection Page

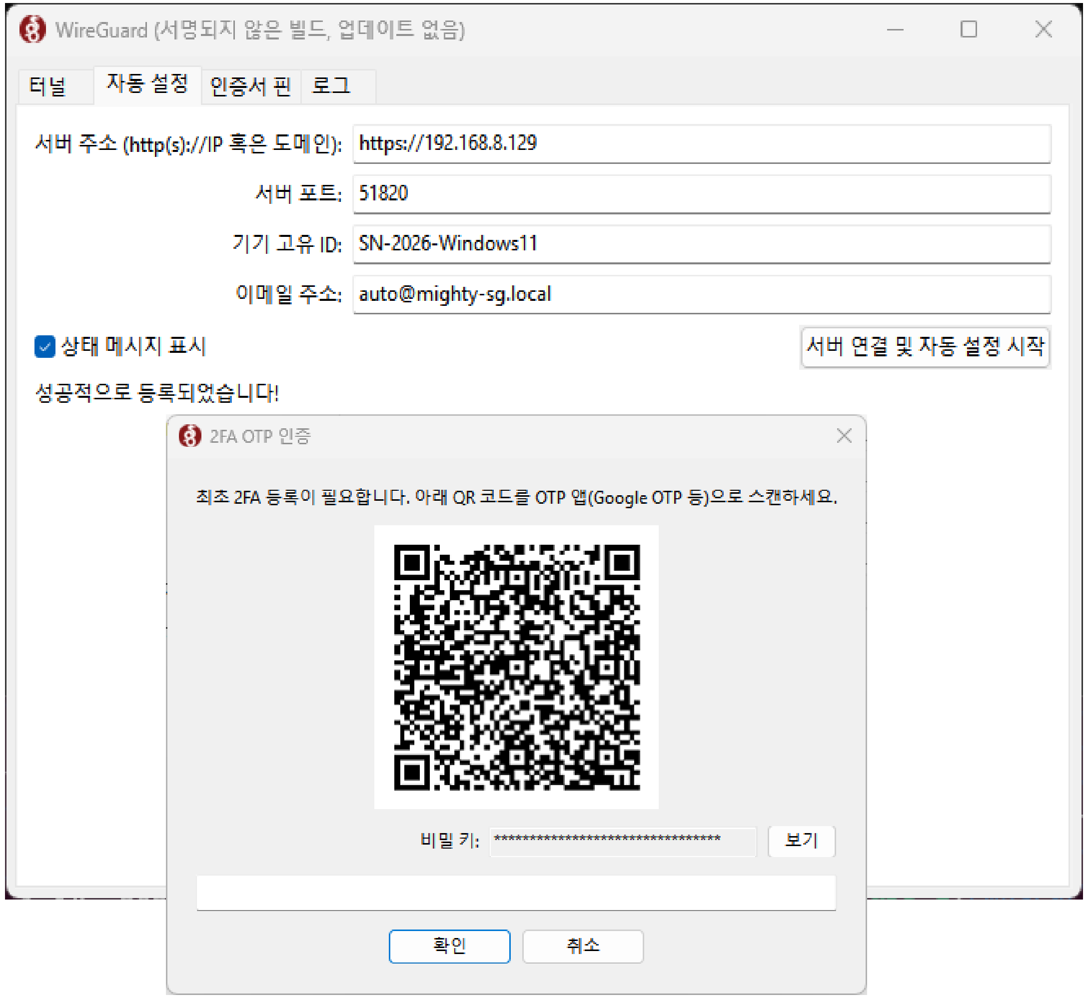



- wg-auto0 interface 생성 및 터널 활성화 Page

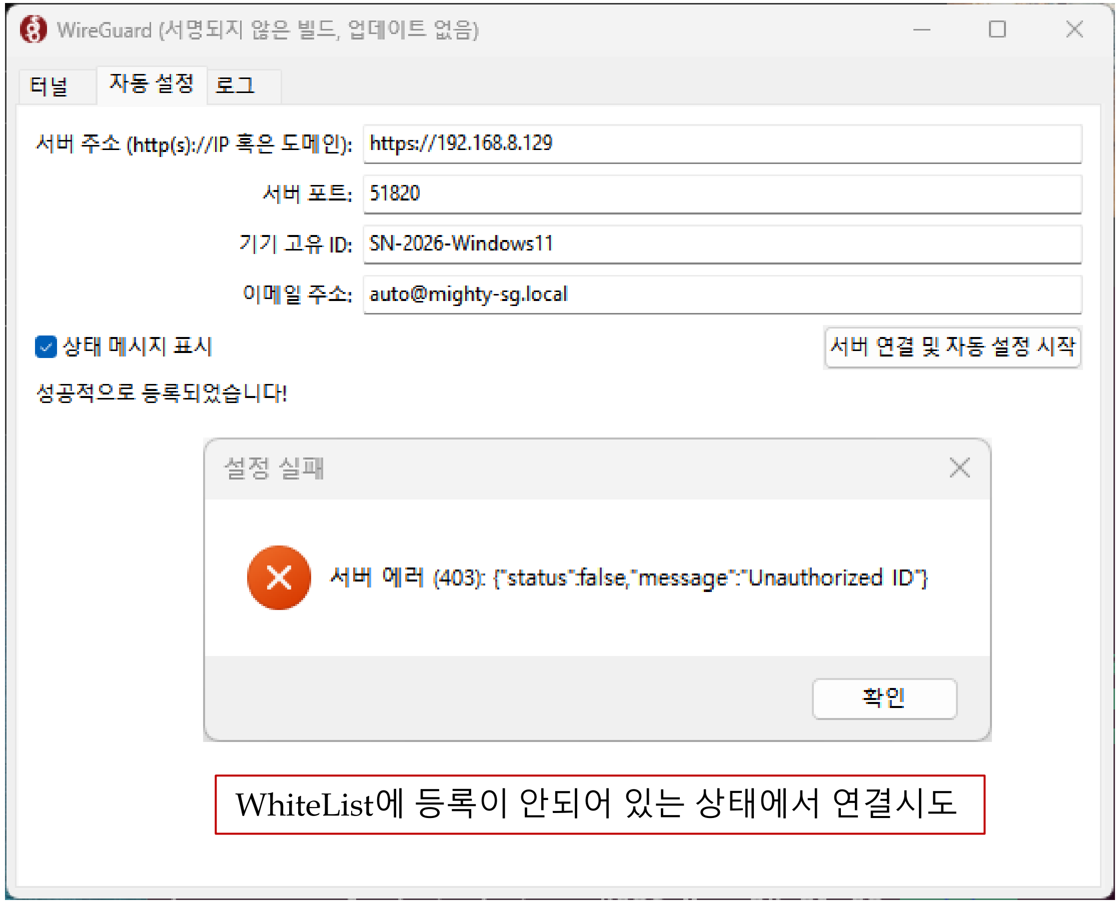


# 6.3 Mighty Windows Agent(1) – User Interface(2)

- Auto Connection – 2FA 인증 Page

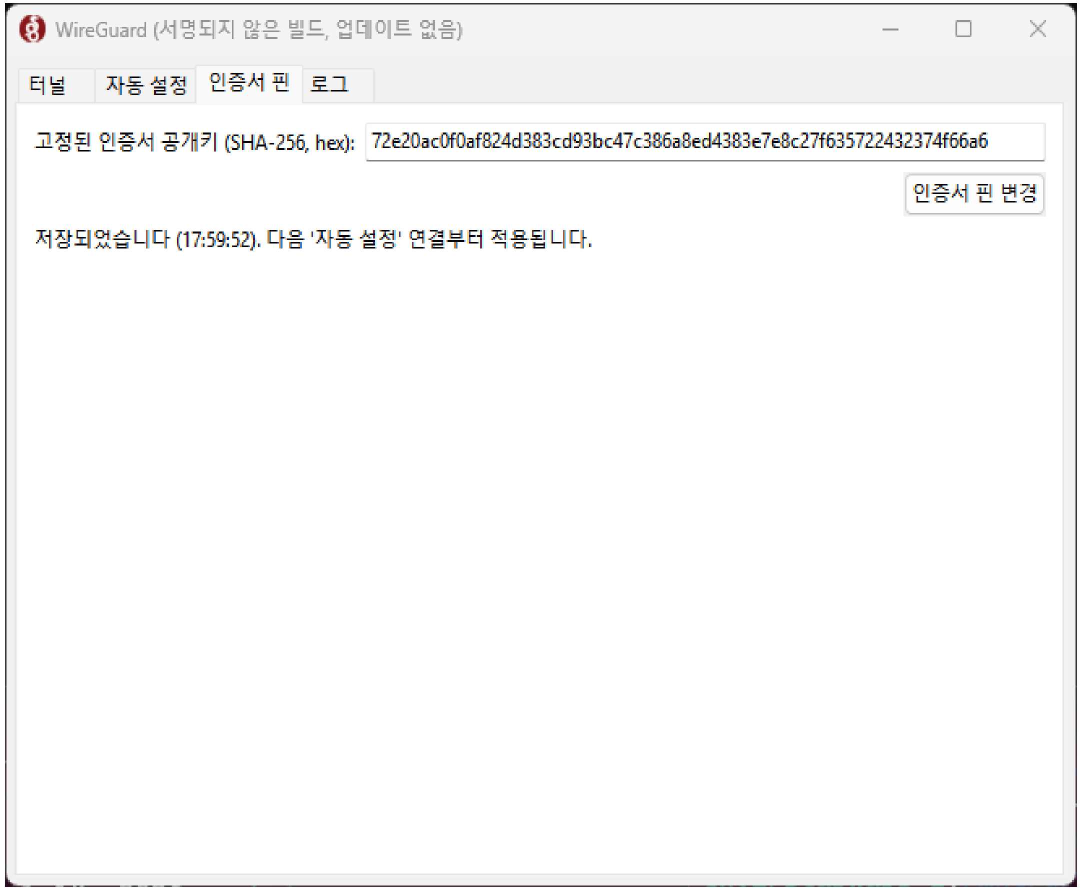



- Auto Connection – 인증 실패 예시

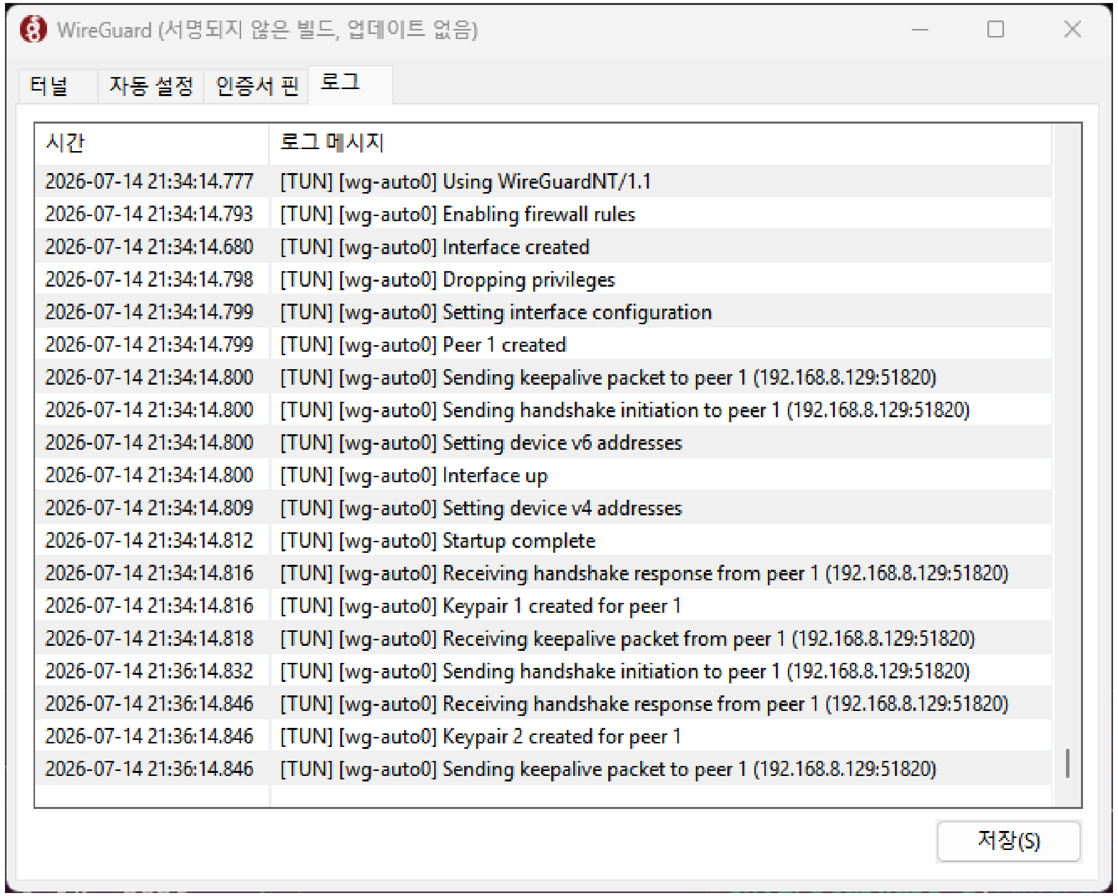


# 6.3 Mighty Windows Agent(1) – User Interface(3)

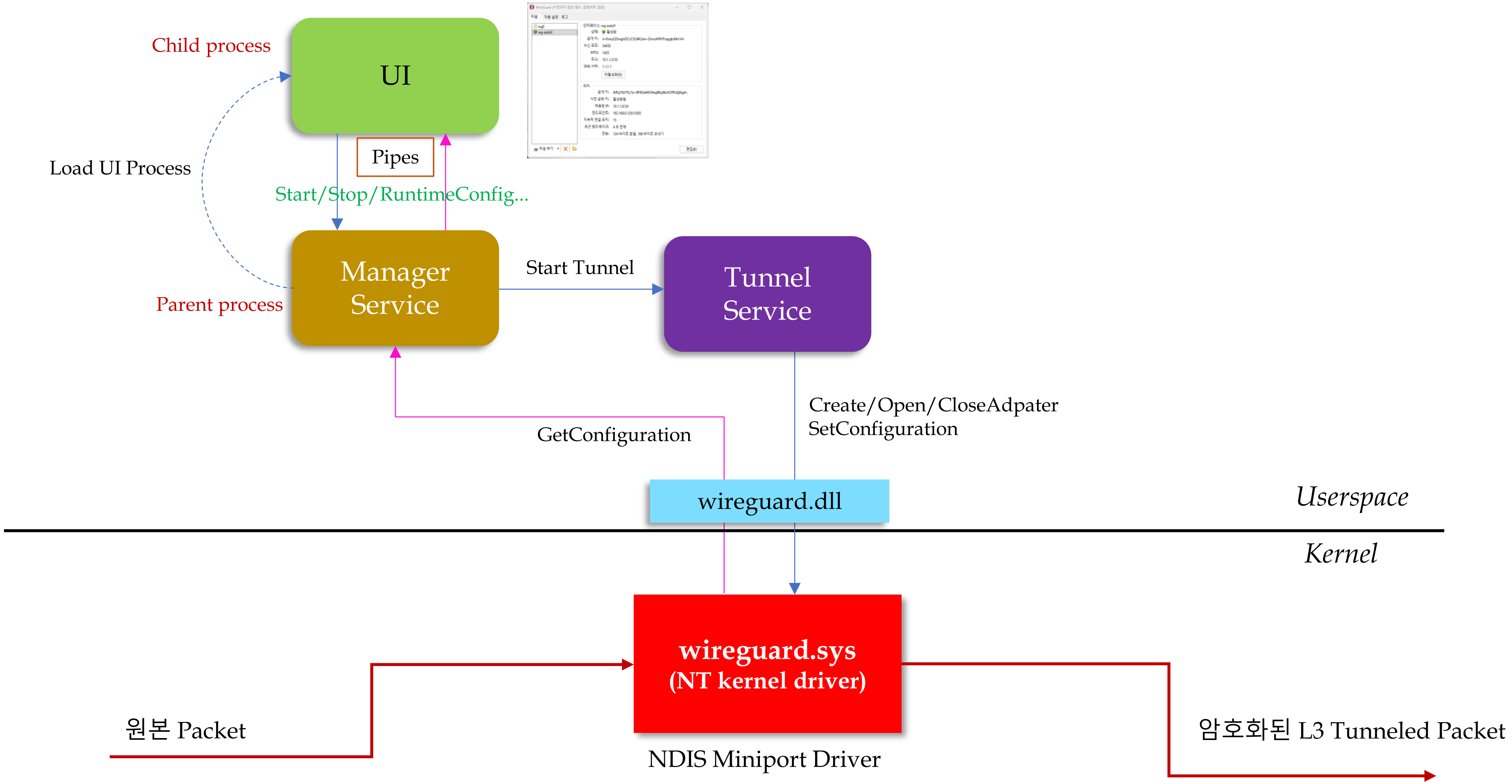
- X.509 인증서 Pin 값 변경 Page




- Log Page



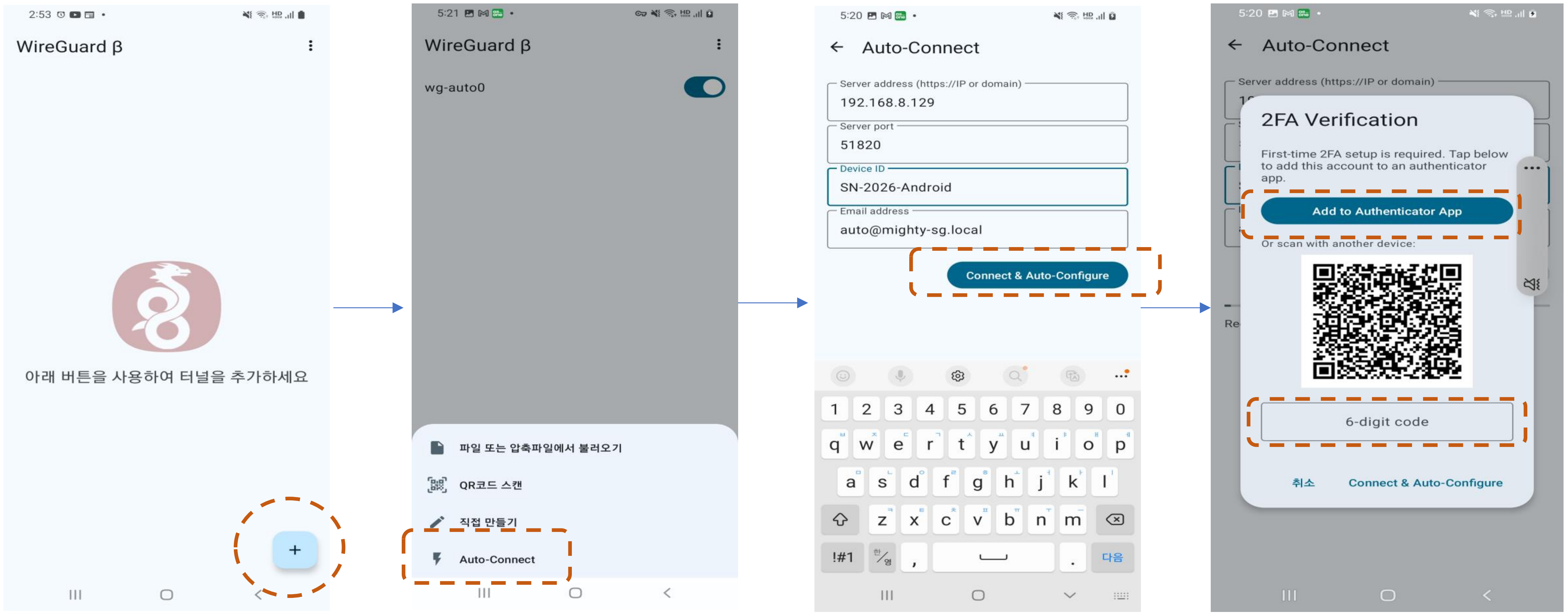
# 6.3 Mighty Windows Agent(2) – Architecture



# 6.4 Mighty Android App

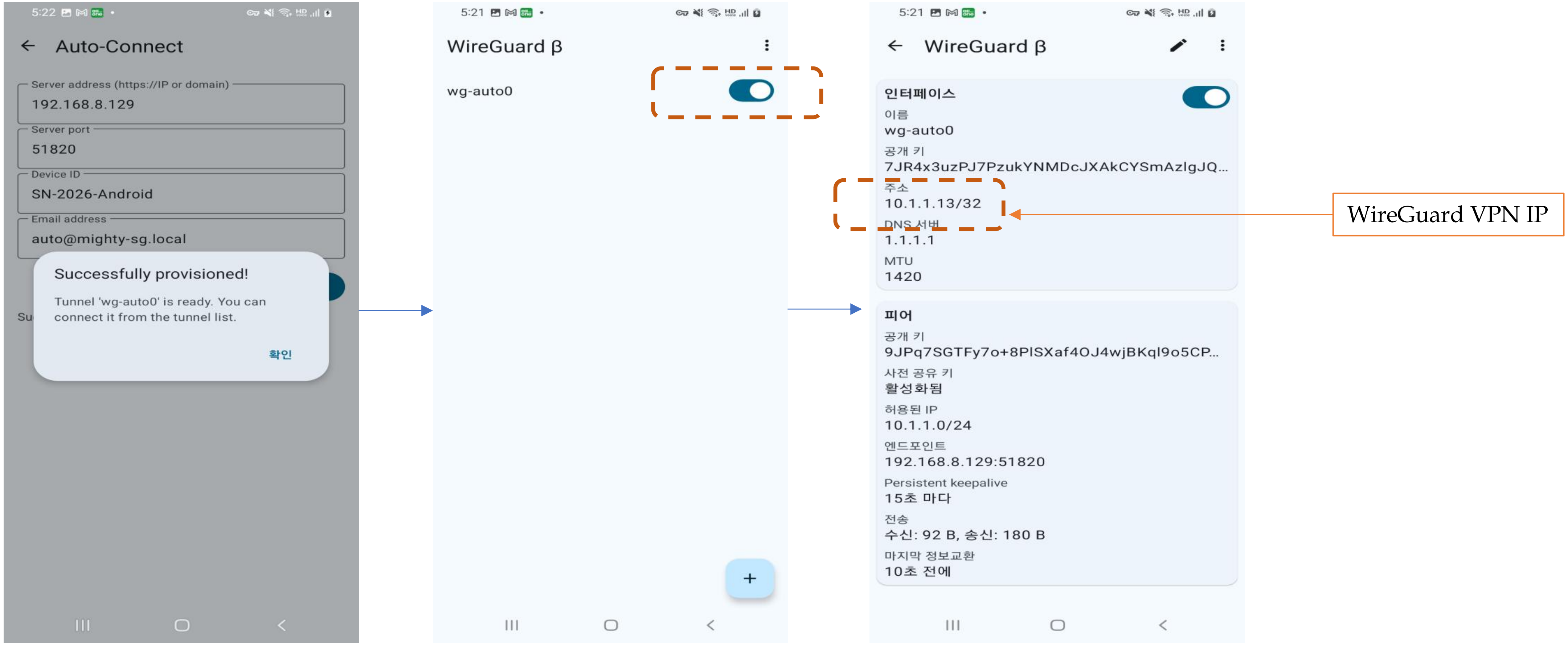
## Android App

# 6.4 Mighty Android App(1) – Auto Connect UI(1)

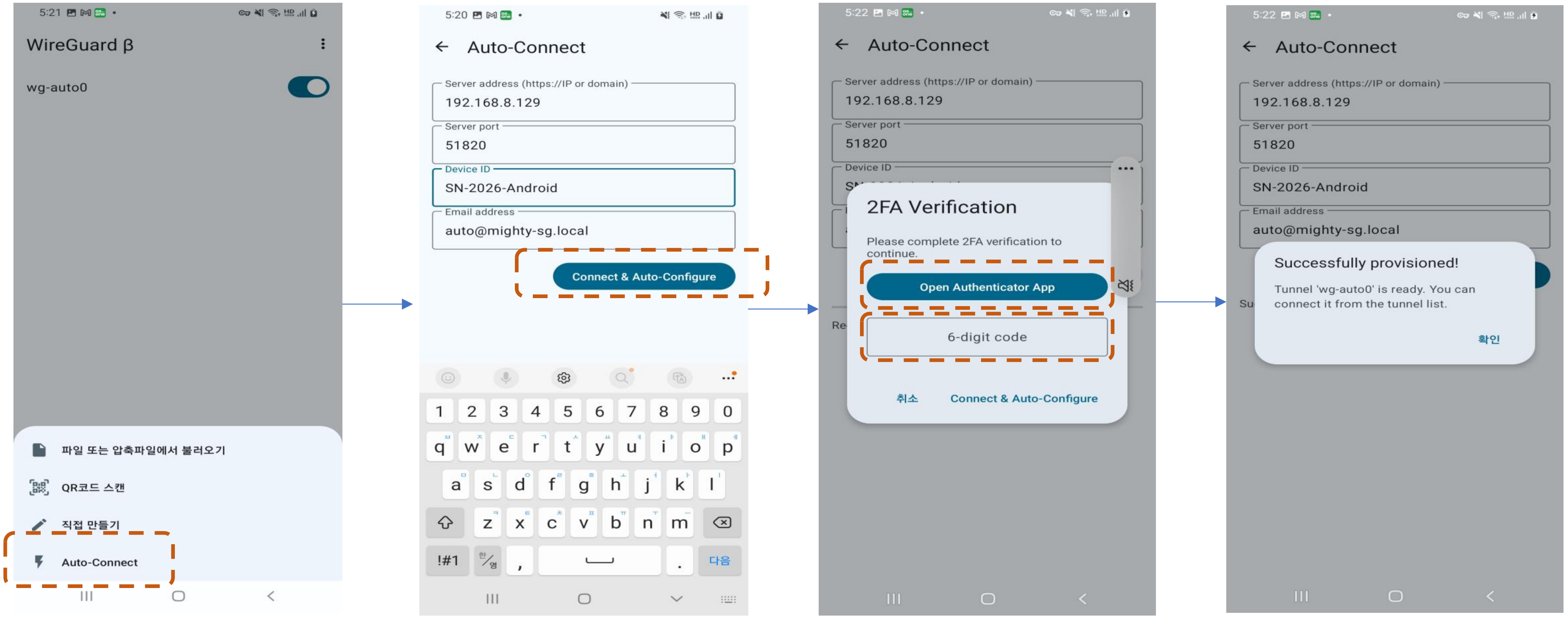



터널 추가 버튼

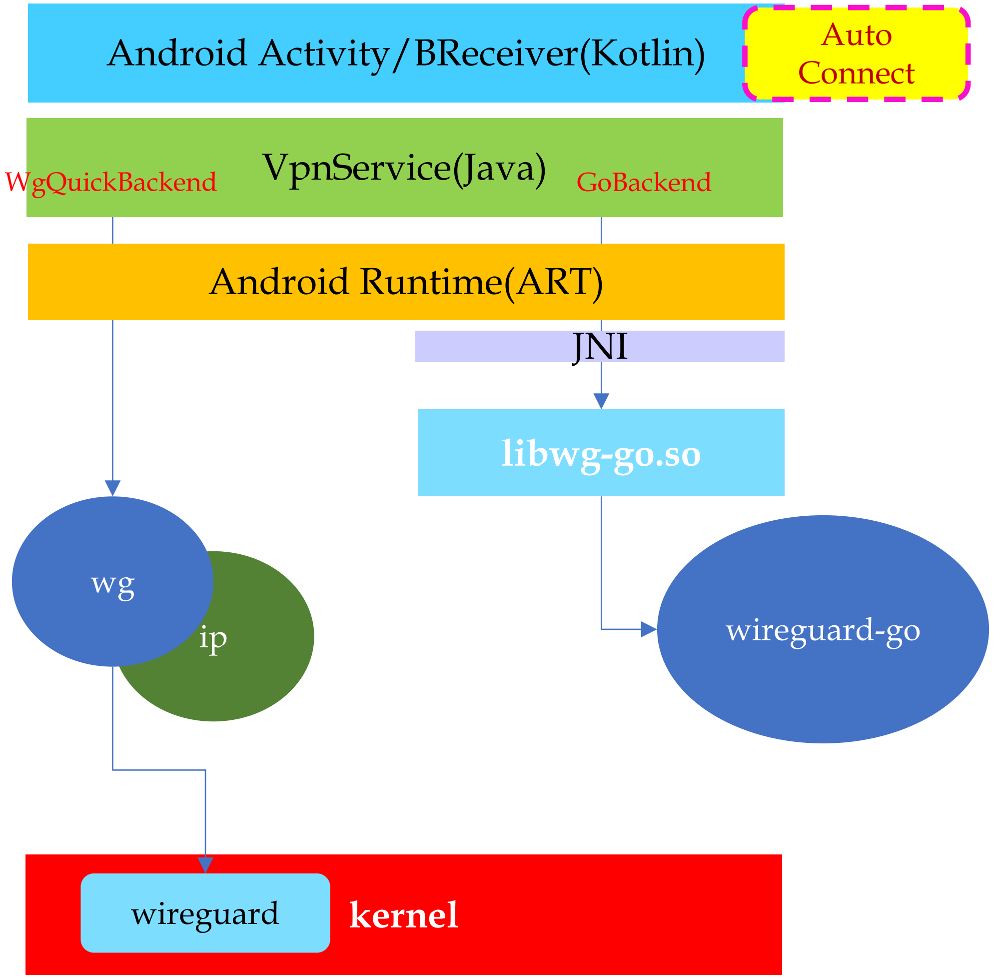
# 6.4 Mighty Android App(1) – Auto Connect UI(2)

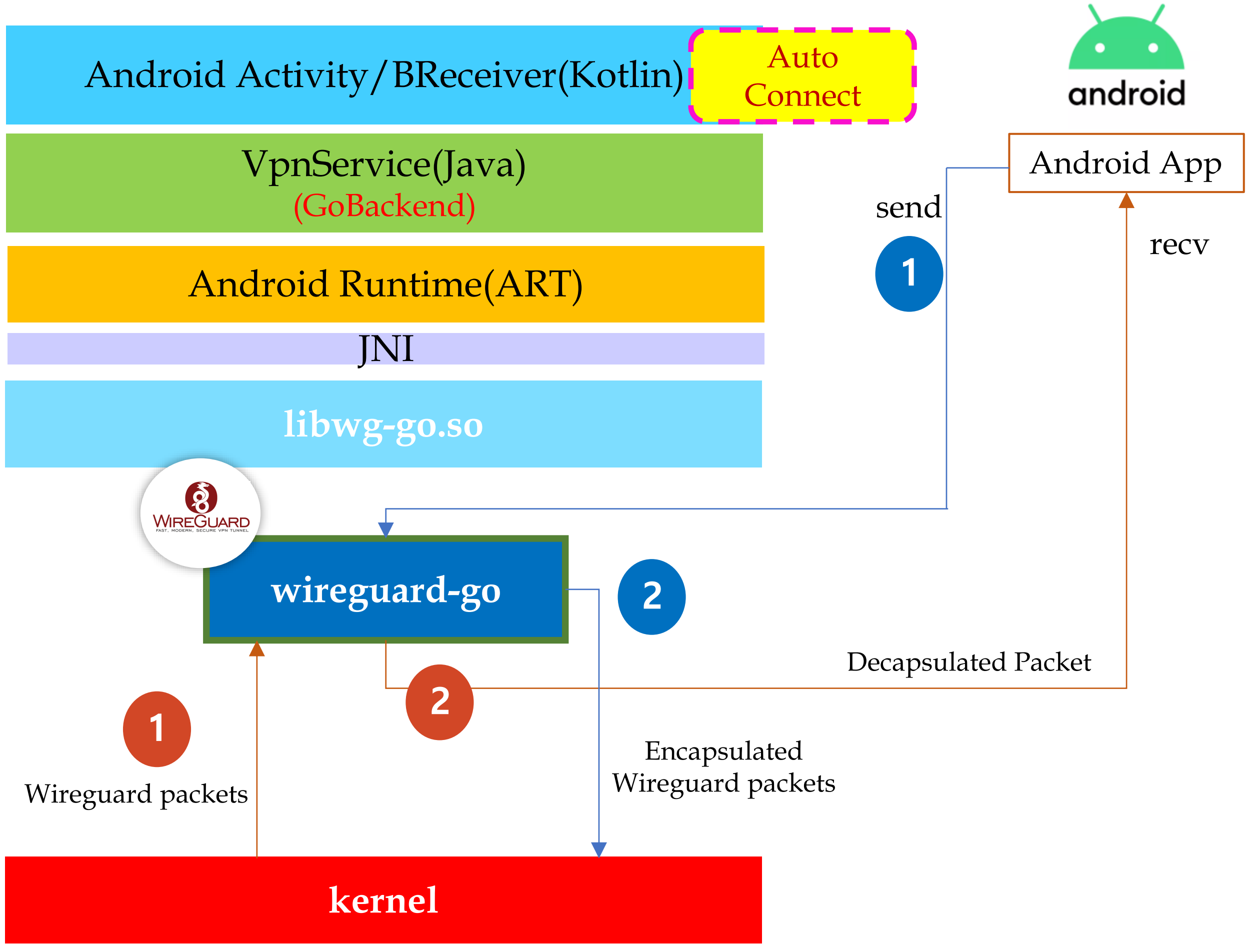


# 6.4 Mighty Android App(1) – Auto Connect UI(3)



# 6.4 Mighty Android App(2) – Architecture

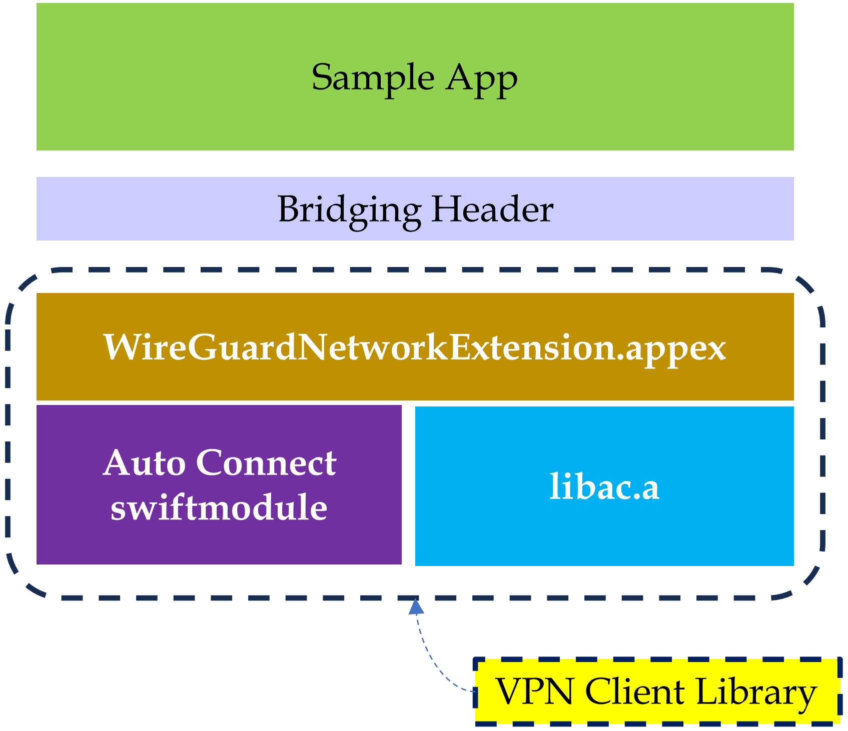


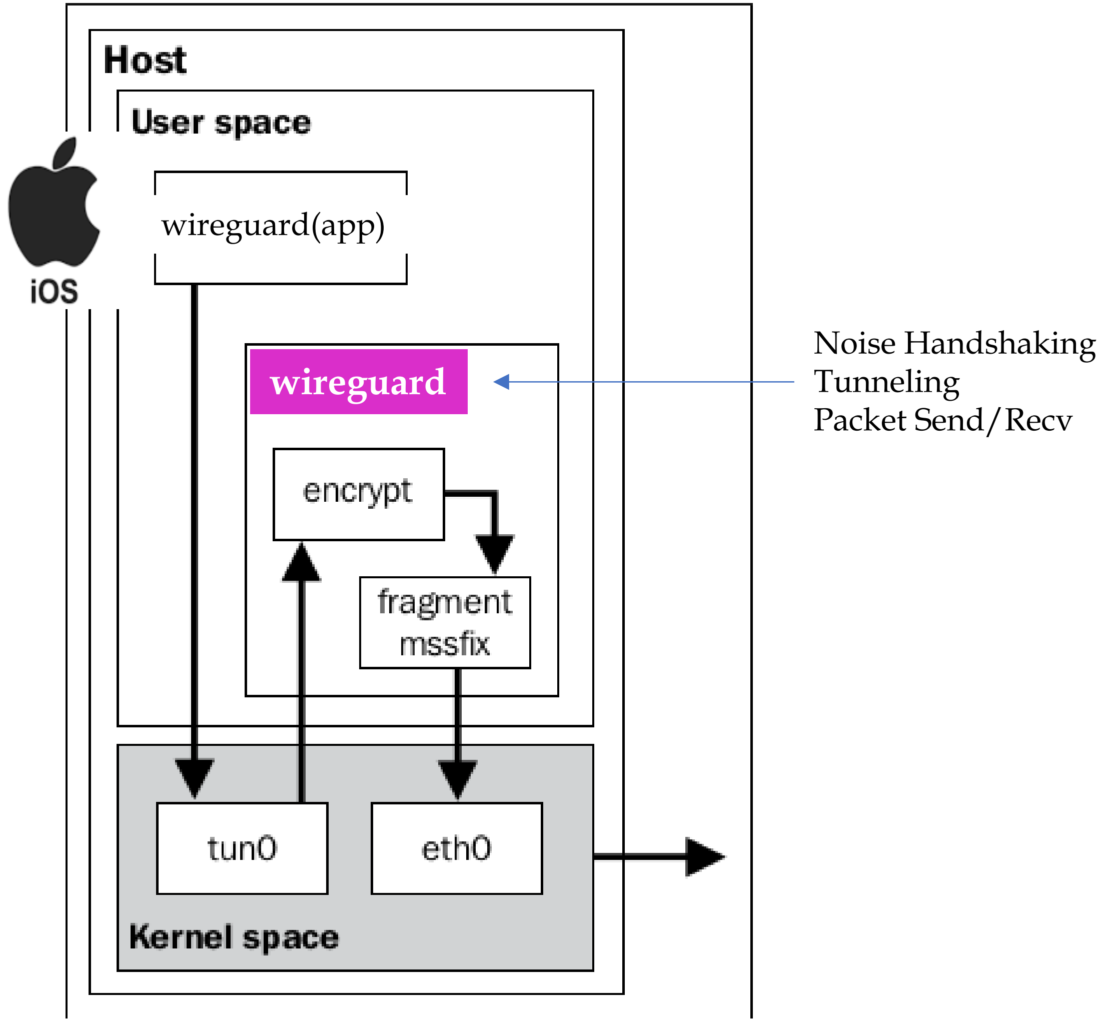



# 6.5 Mighty macOS/iOS Agent

## Mighty macOS/iOS Agent

# 6.5 Mighty macOS/iOS App – Architecture



# 7. References

# 7. References


- [1] https://www.wireguard.com
- [2] https://agentgateway.dev/
- [3] https://www.gl-inet.com/collections/all-products
- [4] https://www.friendlyelec.com/
- [5] https://ferrumgate.com/
- [6] https://tailscale.com/blog/more-throughput
- [7] https://media.frnog.org/FRnOG_28/FRnOG_28-3.pdf
- [8] Performance Comparison of VPN Solutions, Lukas Osswald
- [9] https://www.netmanias.com/
- [10] Windows is a registered trademark of Microsoft.
- [11] Android is a registered trademark of Google.
- [12] macOS & iOS is a registered trademark of Apple.
- [13] Ubuntu is a registered trademark of Canonical Ltd.
- [14] And Google~

**We Secure the AI World with Mighty AI Security Platform !**